TOÀN BỘ ĐỀ VÀ LỜI GIẢI TRONG MỘT TÀI LIỆU

LẬP TRÌNH XỬ LÝ DỮ LIỆU

Tổng hợp **5 đề** • Giải đủ **130 câu**

047 • 045 • 046 • 030 • 001 (luyện tập)

Đủ 130 đề bài, 400 lựa chọn A--D và toàn bộ code nguồn
Mỗi câu có đáp án hoặc kết luận phù hợp dữ kiện, lời giải chi tiết và phần mở rộng. Sáu câu tự luận đề 001 có thêm mẫu trả lời ngắn. Các câu lỗi hoặc thiếu dữ kiện trong đề 030 được phân tích đúng theo bản nguồn.

Nguyên tắc giữ đề

Giữ nguyên chữ, tên biến, dữ liệu, mẫu regex, các ý hỏi và cả lỗi có sẵn ở phần đề. Code sửa minh họa nằm riêng trong lời giải. Quy định dành cho thí sinh in trên nguồn là nội dung của tài liệu gốc.

Mã 046 có hai phiên bản đã nhận

Phần chính dùng PDF mã 046 mới nhất. Câu 18 đã đổi thứ tự B/C; câu 24 đã thêm tiền tố r. Phụ lục đối chiếu giữ nguyên cả các nội dung khác của ảnh cũ, không xóa mất biến thể đã gửi.

Rà soát và biên soạn: 26/10/2026.
Kiểm chứng: Python 3.12.14; pandas 3.0.1; NumPy 2.3.5; DuckDB 2.0.0.
Một tài liệu thống nhất, có mục lục và dấu trang đến từng câu.

Phạm vi đã hợp nhất và kiểm tra

Đề	Số câu	Nguồn dùng trong phần chính
047	26	Đủ bốn trang ảnh đầu tiên, câu 1--26.
045	26	Ba trang ảnh cũ và trang cuối vừa bổ sung, câu 24--26 đã giải đầy đủ.
046	26	PDF mới nhất, đủ câu 1--26; đã so với ảnh cũ theo từng câu.
030	26	PDF văn bản mã 030 đã gửi; giữ nguyên cả lỗi nguồn và ghi rõ cách hiểu.
001 luyện tập	26	PDF luyện tập 6 trang, đủ 20 câu trắc nghiệm và 6 câu tự luận.
Tổng	**130**	**100 câu trắc nghiệm, 30 câu tự luận; 400 lựa chọn A--D.**

Các điểm đã hiệu chỉnh và phân biệt rõ

- 045/20: mẫu không có ranh giới cuối nên trích được chuỗi 30 từ 30m2x.
- 045/24--26: bổ sung từ trang ảnh mới, gồm giá theo số đêm, trung bình theo khu và xử lý ký hiệu tiền.
- 046/18: PDF mới chọn C (31/08); ảnh cũ chọn B cho cùng ngày.
- 046/24: PDF mới dùng raw string nên ra ['18', nan]; ảnh cũ dùng chuỗi thường nên ra [nan, nan].
- 046/8: kết quả phụ thuộc Copy-on-Write; nêu rõ pandas 3.x và hành vi cũ.
- 030: không ép chọn đáp án cho code lỗi, câu thiếu đầu vào hoặc lựa chọn không duy nhất.
- 001/25: explode tạo 7 hàng kể cả một ô giữ chỗ cho danh sách rỗng; tỷ lệ theo chỗ ở phải chia cho 4.

Cách kiểm tra

Đã đối chiếu số câu liên tục 1--26 của từng đề, đủ bốn lựa chọn ở 100 câu trắc nghiệm, các khối code và các ý hỏi. Các đầu ra tính toán then chốt và code sửa được chạy trên ví dụ tương ứng. Các kết luận nghiệp vụ được giải thích riêng thay vì chỉ liệt kê chữ đáp án.

Lưu ý

``Đủ 5 đề" không có nghĩa mọi câu trắc nghiệm nguồn đều có một đáp án hợp lệ. Đề 030 có lỗi văn bản/code thật sự; lời giải chính xác phải chỉ ra lỗi đó. Mọi câu vẫn được giữ và được giải thích.

Mục lục theo đề

Phạm vi đã hợp nhất và kiểm tra 2

Mã đề 047 4

Mã đề 045 31

Mã đề 046 58

Mã đề 030 85

Mã đề 001 (luyện tập) 112

Đối chiếu PDF 046 mới với ảnh 046 cũ 139

Tài liệu đối chiếu và bảng kiểm cuối 141

Mã đề 047

Đáp án nhanh phần trắc nghiệm

Câu	1	2	3	4	5	6	7	8	9	10
Đáp án	B	B	A	A	D	D	C	D	C*	B
Câu	11	12	13	14	15	16	17	18	19	20
Đáp án	C	A	A	A	C	B	B	C	D	D

Câu 9: C là lựa chọn phù hợp nhất theo ý đề; phát biểu ``không sắp thì luôn KeyError cần điều kiện. Câu 14 ghi dữ liệu chuỗi; tên dtype có thể khác theo phiên bản.

Đi đến từng câu Câu 1 • Câu 2 • Câu 3 • Câu 4 • Câu 5 • Câu 6 • Câu 7 • Câu 8 • Câu 9 • Câu 10 • Câu 11 • Câu 12 • Câu 13

Câu 14 • Câu 15 • Câu 16 • Câu 17 • Câu 18 • Câu 19 • Câu 20 • Câu 21 • Câu 22 • Câu 23 • Câu 24 • Câu 25 • Câu 26

Đã có đủ câu 1--26. Phần đề nằm trước hộp đáp án; các bản sửa code chỉ nằm trong lời giải.

Phần I: Trắc nghiệm

Mã 047 • Câu 1. (2 điểm)

Việc nào bài học xếp vào nhóm AI thường làm tốt?

A. Quyết định giúp nên điền 0 cho ô trống hay không.
B. Giải thích một khái niệm còn lạ, chẳng hạn venv là gì.
C. Bịa tên tham số vẫn chạy được trên đúng bản thư viện.
D. Tự chọn quy ước ô trống đúng với nghĩa nghiệp vụ.

Đáp án: B. Giải thích một khái niệm còn lạ, chẳng hạn venv là gì.

Giải thích. AI có thể diễn giải khái niệm, tạo ví dụ và giải thích một đoạn chương trình. Chẳng hạn, venv giúp tạo môi trường Python riêng để quản lý thư viện của từng dự án.

Việc điền số 0 hay giữ giá trị thiếu phụ thuộc ý nghĩa dữ liệu: chưa có giá khác với giá bằng 0. AI không thể tự suy ra chắc chắn quy ước nghiệp vụ nếu thiếu bối cảnh. Tên tham số cũng phải có trong đúng phiên bản thư viện; một tên nghe hợp lý chưa bảo đảm chương trình chạy được. Do đó A và D giao cho AI quyền tự quyết quá mức, còn C mô tả hành vi sai.

Mở rộng / Cách nhớ. Dùng AI để hiểu và đề xuất; kiểm chứng hành vi bằng tài liệu chính thức và một ví dụ chạy được. Quy ước xử lý dữ liệu cần được người hiểu nguồn dữ liệu xác nhận.

Mã 047 • Câu 2. (2 điểm)

fullmatch không truyền flags cộng được bao nhiêu mã đúng?

```
ma = pd.Series(["P7", "p7", "P7X"])
print(int(ma.str.fullmatch(r"P\d+").sum()))
```

A. 0
B. 1
C. 3
D. 2

Đáp án: B. In ra 1.

Giải thích. Mẫu có chữ `P` hoa, theo sau là ít nhất một chữ số. `fullmatch` yêu cầu khớp toàn bộ chuỗi.

Mã	Khớp?	Lý do
`P7`	`True`	Đúng chữ P và có chữ số 7.
`p7`	`False`	Chữ p thường không trùng P hoa.
`P7X`	`False`	Dư chữ X ở cuối.

Tổng Boolean là $1 + 0 + 0 = 1$ vì `True` được tính như 1, `False` như 0 khi cộng.

Mở rộng / Cách nhớ. Với `flags=re.I`, cả `P7` và `p7` khớp nên tổng là 2. Với `contains(r"P\d+")`, `P7X` cũng chứa đoạn `P7`, nên kết quả khác `fullmatch`.

Mã 047 • Câu 3. (2 điểm)

```
s = pd.Series([0.25], index=pd.to_datetime(["2026-05-31"]))
```

f"{s.loc[...] :.1%}" cần một số, không phải Series. Nhãn nào dùng được?

A. "2026-05-31", đúng nhãn một hàng.
B. "2026", vì lấy cả năm rồi format từng tháng.
C. "2026-05", vì nhãn rút gọn luôn là một số.
D. "05", vì pct_change chỉ cần số tháng.

Đáp án: A. Dùng nhãn đầy đủ `"2026-05-31"`; kết quả định dạng là 25.0%.

Giải thích. Chỉ mục trong ví dụ có một nhãn ngày duy nhất. Truy cập đúng ngày trả về giá trị vô hướng 0.25. Định dạng `.1%` nhân giá trị với 100 để hiển thị, thêm dấu phần trăm và giữ một chữ số thập phân.

Chuỗi rút gọn `"2026-05"` chọn dữ liệu cả tháng; `"2026"` chọn cả năm. Kết quả của những phép chọn này là `Series`, kể cả trong ví dụ chỉ có một hàng, nên không thể áp trực tiếp định dạng số như trên. Chuỗi `"05"` cũng không phải cách chọn đúng một nhãn ngày trong đề.

Mở rộng / Cách nhớ. Có thể dùng `s.at[pd.Timestamp("2026-05-31")]` khi biết nhãn duy nhất. Nếu chỉ mục có nhãn ngày bị trùng, ngay cả `loc` với nhãn đầy đủ cũng có thể trả nhiều giá trị; cần kiểm tra tính duy nhất trước.

Mã 047 • Câu 4. (2 điểm)

Muốn giảm 10% giá nguyên căn trên một bản sao, rồi tính tiền ba đêm từ giá đã giảm. Thứ tự nào đúng?

A. Gán loc theo mask bằng giá cũ nhân 0,9, rồi mới tính lại cột tiền.
B. Tính tiền ba đêm, rồi gán vào Series đã lấy ra bằng bang["gia"].
C. Nhân 0,1 vào mọi hàng, kể cả phòng riêng.
D. Sửa df gốc để bản sao tự nhận giá mới.

Đáp án: A. Gán theo mặt nạ bằng giá cũ nhân 0.9, rồi tính lại cột tiền.

Giải thích. Giảm 10% nghĩa là giữ $100\% - 10\% = 90\%$, tức nhân giá với 0.9. Trước hết chọn đúng các hàng nguyên căn; sau đó cập nhật cột giá; cuối cùng tạo cột tiền từ cột giá đã cập nhật.

```
ban = df.copy()
mask = ban["loai"].eq("Nguyên căn")
ban.loc[mask, "gia"] = ban.loc[mask, "gia"] * 0.9
ban["ba_dem"] = ban["gia"] * 3
```

Nhân 0.1 chỉ giữ lại 10% giá, không phải giảm 10%. Tạo tiền trước rồi sửa giá không làm tiền đã tạo tự tính lại. Sửa bảng gốc cũng không bảo đảm một bản sao độc lập thay đổi theo.

Mở rộng / Cách nhớ. Dùng phép gán trực tiếp qua `loc` để việc cập nhật rõ ràng. Quy tắc tổng quát: sửa dữ liệu đầu vào trước, tính các cột phụ thuộc sau.

Mã 047 • Câu 5. (2 điểm)

df có một hàng giá NaN. Câu SELECT ... FROM df WHERE gia >= 90 xử lý hàng đó thế nào?

A. Báo lỗi, vì không so được với NULL.
B. Giữ hàng, vì WHERE chỉ bỏ giá bằng 0.
C. Giữ hàng, vì thiếu giá được coi là đạt ngưỡng.
D. Bỏ hàng, vì so sánh với NULL không ra đúng.

Đáp án: D. Bỏ hàng, vì phép so sánh với `NULL` không cho kết quả `TRUE`.

Giải thích. Theo ngữ nghĩa SQL trong đề, giá thiếu được biểu diễn là `NULL`. Biểu thức `NULL >= 90` có giá trị logic `UNKNOWN`, không phải `TRUE`. Mệnh đề `WHERE` chỉ giữ những hàng có điều kiện là `TRUE`; cả `FALSE` và `UNKNOWN` đều bị loại.

Không có quy tắc mặc định coi thiếu giá là đạt ngưỡng, và `WHERE` không chỉ loại những giá bằng 0. Việc so sánh với `NULL` theo cách này cũng không tự gây lỗi cú pháp.

Mở rộng / Cách nhớ. Muốn giữ thêm hàng chưa có giá, phải viết rõ:

```
SELECT * FROM df WHERE gia >= 90 OR gia IS NULL;
```

Để kiểm tra thiếu, dùng `IS NULL`, không dùng `= NULL`. Kết luận trên theo mô hình SQL `NULL`; một số hệ quản trị còn có giá trị NaN riêng với quy tắc riêng.

Mã 047 • Câu 6. (2 điểm)

Tháng 2 trong một Series giả lập có resample("ME").size() bằng 0. Câu nào đúng khi đưa số này vào báo cáo?

A. Phải đổi 0 thành NaN trước khi vẽ, nếu không biểu đồ sai.
B. Tháng 2 chắc chắn chưa được thu thập ở nguồn.
C. Tháng 2 có đánh giá, nhưng size bỏ tháng không đủ 30 ngày.
D. Series này không có hàng nào trong tháng 2; chưa đủ căn cứ nói rằng nguồn chưa thu.

Đáp án: D. Series này không có hàng nào trong tháng 2; chưa đủ căn cứ nói rằng nguồn chưa thu thập.

Giải thích. `resample("ME")` tạo các nhóm theo cuối tháng, còn `size()` đếm số hàng trong từng nhóm. Số 0 cho biết dữ liệu đang xét không chứa quan sát thuộc tháng 2.

Đây là kết luận về tập dữ liệu hiện tại. Có thể dữ liệu đã bị lọc, là dữ liệu mô phỏng, hoặc nguồn thực sự không có quan sát. Chỉ từ số 0 không thể chọn chắc chắn một nguyên nhân. Số ngày của tháng cũng không quyết định `size`: tháng 28 ngày vẫn được đếm bình thường.

Mở rộng / Cách nhớ. Phân biệt `size` đếm hàng với `count` đếm giá trị không thiếu. Một tháng có hai hàng mà cả hai giá trị đều thiếu sẽ có `size = 2` và `count = 0`. Không đổi 0 thành `NaN` chỉ vì hình thức biểu đồ.

Mã 047 • Câu 7. (2 điểm)

dot_1 và dot_2 đều có P1, khác giá và khác số đợt. pd.concat(..., ignore_index=True) xử lý P1 thế nào?

A. Xoá P1 vì mã bị trùng.
B. Gộp thành một hàng, giá là trung bình hai đợt.
C. Thành hai hàng, mỗi đợt một hàng.
D. Ghép hai đợt thành hai cột theo mã, như merge.

Đáp án: C. Thành hai hàng, mỗi đợt một hàng.

Giải thích. Mặc định `concat` nối theo hàng (`axis=0`). Hàng P1 ở đợt 1 và hàng P1 ở đợt 2 đều được giữ. `ignore_index=True` chỉ đánh lại chỉ mục kết quả thành 0, 1, 2, …; nó không kiểm tra tính duy nhất của cột mã.

Phép nối này không tự bỏ bản ghi trùng mã, không tự tính giá trung bình và không chuyển hai đợt thành hai cột. Muốn tính trung bình phải gọi thêm phép tổng hợp như `groupby`; muốn ghép các cột theo khóa mới dùng `merge` hoặc phép biến đổi thích hợp.

Mở rộng / Cách nhớ. Khi một mã có thể xuất hiện nhiều đợt, cặp `(ma, dot)` mới mô tả đầy đủ một quan sát. Đừng xóa các hàng chỉ vì `ma` bị lặp nếu mục tiêu là phân tích thay đổi theo thời gian.

Mã 047 • Câu 8. (2 điểm)

Một cột có 5 hàng, trong đó 2 ô NaN. count() và isna().sum() lần lượt là gì?

A. 5 và 2
B. 2 và 3
C. 3 và 5
D. 3 và 2

Đáp án: D. 3 và 2.

Giải thích. Có 5 hàng tất cả nhưng chỉ $5 - 2 = 3$ giá trị không thiếu. `count()` bỏ qua các ô thiếu nên trả về 3. `isna()` tạo một mặt nạ Boolean gồm hai `True` ở hai ô thiếu và ba `False` ở các ô còn lại. Tổng mặt nạ là $1 + 1 + 0 + 0 + 0 = 2$.

```
s = pd.Series([10, np.nan, 20, np.nan, 30])
print(s.count(), s.isna().sum())  # 3 2
print(len(s), s.size)             # 5 5
```

Mở rộng / Cách nhớ. Với một `Series`, luôn có `s.count() + s.isna().sum() == len(s)`. Số 0, chuỗi rỗng và dấu `?` không tự động là giá trị thiếu trong mọi tình huống; phải xét cách dữ liệu đã được đọc và chuẩn hóa.

Mã 047 • Câu 9. (2 điểm)

Chỉ mục ngày chưa sort_index, vì các hàng đang xếp theo chỗ ở. Câu nào đúng?

A. Mọi lát cắt loc đều báo KeyError, kể cả khi chỉ lấy một tháng.
B. Mọi lát cắt loc đều chạy, kể cả cắt từ năm này sang năm kia.
C. loc một tháng vẫn lấy được; cắt một khoảng năm thì cần sắp, nếu không thì KeyError.
D. sort_index xoá các đánh giá trùng ngày trước khi cắt khoảng năm.

Đáp án: C theo ý đề: chọn một tháng vẫn có thể được; nên sắp chỉ mục trước khi cắt khoảng năm. Phần nói cứ chưa sắp là luôn lỗi cần được hiểu có điều kiện.

Giải thích. Với `DatetimeIndex`, phép chọn một tháng như `s.loc["2026-03"]` có thể tìm các ngày thuộc tháng đó dù thứ tự ngày chưa tăng dần. Ngược lại, cắt khoảng bằng chuỗi ngày trên chỉ mục không đơn điệu có các hạn chế; khi mốc biên cần dùng không tồn tại, pandas có thể báo `KeyError`.

Không nên khẳng định mọi phép cắt đều lỗi hoặc mọi phép cắt đều chạy. `sort_index` chỉ sắp xếp chỉ mục, không xóa ngày trùng. Bởi vậy A, B, D không đúng; C phản ánh hướng xử lý cần chọn trong đề nhưng diễn đạt quá tuyệt đối ở vế cuối.

```
s_theo_ngay = s.sort_index()
ket_qua = s_theo_ngay.loc["2025":"2026"]
```

Mở rộng / Cách nhớ. Nếu cần giữ thứ tự hàng, có thể dùng mặt nạ ngày thay cho cắt khoảng:

```
ket_qua = s[(s.index >= "2025-01-01") & (s.index < "2027-01-01")]
```

Mã 047 • Câu 10. (2 điểm)

```
df = pd.DataFrame({"gia": [60.0, 100.0, np.nan]})
```

(df["gia"] < 80).mean() khác với tỷ lệ cùng điều kiện tính chỉ trên các hàng gia không thiếu. Vì sao hai tỷ lệ không trả lời cùng một câu?

A. Hàng thiếu giá làm cả cột không so sánh được.
B. Cách thứ nhất giữ hàng thiếu trong mẫu số vì so sánh ra False.
C. Cách thứ hai đổi ngưỡng 80 thành 0.
D. mean của bool luôn bỏ False.

Đáp án: B. Cách thứ nhất giữ hàng thiếu trong mẫu số vì phép so sánh tạo ra `False`.

Giải thích. Cột số thực thông thường trong đề cho kết quả so sánh `[True, False, False]`. Do đó tỷ lệ thứ nhất là $1/3 \approx 33.33\%$.

Nếu loại giá thiếu trước, chỉ còn 60 và 100; mặt nạ là `[True, False]` nên tỷ lệ là $1/2 = 50\%$. Hai tỷ lệ trả lời hai câu hỏi khác nhau: số hàng có giá dưới 80 trên toàn bộ hàng, hay trên những hàng biết giá.

```
ty_le_biet_gia = (df.loc[df["gia"].notna(), "gia"] < 80).mean()
```

Mở rộng / Cách nhớ. `mean` của Boolean không bỏ `False`; nó tính `False` là 0. Với dtype nullable như `Float64`, giá thiếu có thể được truyền thành `pd.NA` và bị `mean` bỏ qua, nên cần kiểm tra dtype trước khi khái quát hóa.

Mã 047 • Câu 11. (2 điểm)

nlargest cho thấy một đêm giá hàng chục triệu. Việc nên làm tiếp là gì?

A. Thay giá đó bằng trung vị của cột cho sạch phân bố.
B. Xoá hàng đó trước khi tính bất kỳ thống kê nào.
C. Giữ hàng, đối chiếu mã với nguồn, đơn vị và loại chỗ ở.
D. Đổi đơn vị của cả cột thành USD mà không xem nguồn.

Đáp án: C. Giữ hàng, đối chiếu mã với nguồn, đơn vị và loại chỗ ở.

Giải thích. Một giá trị lớn là dấu hiệu cần kiểm tra, chưa phải bằng chứng chắc chắn của dữ liệu sai. Nó có thể là biệt thự lớn, gói nhiều đêm, đồng tiền khác, sai đơn vị, hoặc lỗi nhập liệu. Cần xác định đúng bản ghi, đối chiếu nguồn và kiểm tra ý nghĩa của trường giá trước khi xử lý.

Thay ngay bằng trung vị sẽ làm mất thông tin và tự áp đặt một giá trị không có căn cứ. Xóa hàng trước mọi thống kê cũng có thể loại một quan sát hợp lệ. Đổi cả cột sang USD khi chưa biết đơn vị nguồn càng không giải quyết được nguyên nhân.

Mở rộng / Cách nhớ. Có thể lưu cờ `can_kiem_tra` và giữ giá gốc. Sau khi xác minh, mới quyết định sửa lỗi, chuyển đơn vị, loại khỏi một phân tích cụ thể hoặc báo cáo riêng. Ghi lại quy tắc để kết quả có thể được tái hiện.

Mã 047 • Câu 12. (2 điểm)

```
gia = pd.Series([40, 80, 200])  # chỉ mục 0, 1, 2
tb_khu = pd.Series({"A": 60.0, "B": 200.0})  # trung bình theo tên khu
print((gia - tb_khu).tolist())
```

Dòng in ra thế nào?

A. Mọi giá trị đều NaN vì không có nhãn chung.
B. Báo lỗi vì hai Series khác dài.
C. Ra đúng độ lệch của từng chỗ ở so với khu của nó.
D. Trừ theo vị trí hàng, bỏ qua tên khu.

Đáp án: A. Tất cả là NaN; cụ thể có năm phần tử: [nan, nan, nan, nan, nan].

Giải thích. pandas căn chỉnh hai `Series` theo nhãn trước khi trừ. Tập nhãn của `gia` là $\{0, 1, 2\}$, của `tb_khu` là $\{A, B\}$; không có nhãn chung. Kết quả dùng hợp của hai tập nhãn, gồm năm nhãn. Tại mỗi nhãn, ít nhất một vế thiếu nên phép trừ cho `NaN`.

Hai `Series` khác độ dài không tự làm phép toán báo lỗi. Chương trình cũng không biết một chỗ ở thuộc khu nào chỉ từ hai dãy số đã cho.

Mở rộng / Cách nhớ. Nếu bảng có cột khu tương ứng, hãy ánh xạ trung bình về từng hàng rồi trừ:

```
df["lech"] = df["gia"] - df["khu"].map(tb_khu)
```

Nhờ `map`, trung bình khu được đặt về đúng nhãn hàng của bảng.

Mã 047 • Câu 13. (2 điểm)

```
x = np.array([90, 127], dtype=np.int8)
print((x * 2).tolist())
```

Dòng in ra gì?

A. [-76, -2]
B. [180, 254]
C. [127, 127]
D. Báo OverflowError

Đáp án: A. [-76, -2].

Giải thích. `int8` là số nguyên có dấu 8 bit, miền biểu diễn từ -128 đến 127. Phép nhân trong ví dụ vẫn thực hiện trên dtype này. Hai kết quả toán học 180 và 254 nằm ngoài miền biểu diễn, nên bị cuộn theo modulo $2^8 = 256$:

$$180 - 256 = -76, \qquad 254 - 256 = -2.$$

Vì vậy kết quả không phải `[180, 254]`, không bị chặn ở 127, và phép nhân mảng này không tự ném `OverflowError`.

Mở rộng / Cách nhớ. Ép sang kiểu rộng hơn *trước* khi nhân:

```
print((x.astype(np.int16) * 2).tolist())  # [180, 254]
```

Ép kiểu sau phép nhân chỉ đổi cách lưu các giá trị đã sai; nó không khôi phục lại 180 và 254.

Mã 047 • Câu 14. (2 điểm)

Cột gia có các số và một ô ?, nghĩa là chưa có giá. read_csv mặc định cho cột này kiểu gì?

A. Chuỗi, vì ? không được coi là thiếu.
B. float64, và ? thành NaN.
C. int64, và ? thành 0.
D. float64, và ? thành 0,0.

Đáp án: A. Cột mang dữ liệu chuỗi vì dấu ? không nằm trong bộ ký hiệu thiếu mặc định.

Giải thích. Đọc CSV phải dựa vào quy ước của chương trình, không tự biết ý nghĩa nghiệp vụ của `?`. Khi chưa khai báo nó là giá trị thiếu, cột chứa cả các biểu diễn số và dấu hỏi nên không được suy luận thành một cột số thuần.

Ở pandas 2.x, cột kiểu này thường có dtype `object`. Từ pandas 3.0, mặc định suy luận chuỗi đã thay đổi; môi trường kiểm chứng cho dtype `str`. Dù tên dtype khác, đáp án về bản chất vẫn là A. Dấu hỏi không tự biến thành `NaN`, 0 hay 0.0.

Mở rộng / Cách nhớ. Khai báo quy ước ngay lúc đọc:

```
df = pd.read_csv("gia.csv", na_values=["?"])
# Hoặc chuẩn hóa cột đã đọc:
df["gia"] = pd.to_numeric(df["gia"], errors="coerce")
```

Cách thứ hai biến cả các chuỗi không hợp lệ khác thành thiếu, nên cần kiểm tra những hàng bị chuyển đổi.

Mã 047 • Câu 15. (2 điểm)

```
gia = pd.Series(["$8,250.00"])
print(gia.str.replace(r"[$,]", "", regex=False).iloc[0])
```

Dòng in ra gì?

A. NaN
B. 8250.00
C. $8,250.00
D. 8.250.00

Đáp án: C. Giữ nguyên chuỗi `$8,250.00`.

Giải thích. `regex=False` yêu cầu tìm nguyên văn chuỗi ký tự `[$,]`. Trong dữ liệu chỉ có ký hiệu đô la và dấu phẩy riêng lẻ, không có nguyên cụm gồm dấu ngoặc vuông, đô la, dấu phẩy, ngoặc vuông. Không có đoạn nào bị thay nên chuỗi giữ nguyên.

Tiền tố `r` chỉ làm đây là raw string của Python; nó không bật chế độ regex. Khi bật regex, `[$,]` mới là lớp ký tự có nghĩa ``một dấu đô la hoặc một dấu phẩy".

Mở rộng / Cách nhớ. Hai cách làm đúng nếu cần loại ký hiệu tiền và dấu phân cách hàng nghìn:

```
a = gia.str.replace(r"[$,]", "", regex=True)
b = (gia.str.replace("$", "", regex=False)
        .str.replace(",", "", regex=False))
# Cả a và b đều chứa chuỗi "8250.00".
so = pd.to_numeric(a)
```

Chỉ bỏ ký hiệu chưa làm dữ liệu thành số; cần bước chuyển kiểu nếu muốn tính toán.

Mã 047 • Câu 16. (2 điểm)

Sau khi tách mỗi tiện nghi thành một hàng, các hàng có Wifi là:

```
dong = [("P1", "Wifi"), ("P1", "Wifi"), ("P2", "Wifi"), ("P5", "Wifi")]
# P3 không có hàng Wifi
```

Có bao nhiêu chỗ ở liệt kê Wifi?

A. 4
B. 3
C. 2
D. 1

Đáp án: B. 3 chỗ ở: P1, P2 và P5.

Giải thích. Có bốn *hàng*, nhưng mã P1 xuất hiện hai lần. Đề hỏi số chỗ ở nên phải đếm số mã phân biệt. Tập mã là $\{P1, P2, P5\}$, có ba phần tử. P3 không được cộng vì không có hàng Wifi.

```
so_cho_o = pd.Series([ma for ma, _ in dong]).nunique()
# 3
```

Mở rộng / Cách nhớ. Với bảng đầy đủ, lọc tiện nghi rồi gọi `df.loc[mask, "ma"].nunique()`. Sau phép `explode`, một đối tượng có thể tạo nhiều hàng; trước khi đếm phải xác định đơn vị cần đếm là hàng, mã hay cặp `(ma, tien_nghi)`.

Mã 047 • Câu 17. (2 điểm)

ten là một Series. ten.strip().lower() chạy thế nào?

A. Ra Series chữ thường, đã bỏ khoảng trắng hai đầu.
B. Báo AttributeError vì Series không có strip.
C. Chỉ sửa hàng đầu tiên.
D. Bỏ qua giá trị thiếu rồi chạy tiếp.

Đáp án: B. Báo AttributeError vì Series không có phương thức strip như một chuỗi Python.

Giải thích. `strip` và `lower` là phương thức của chuỗi. Một `Series` là một tập hợp có nhãn, không phải một chuỗi đơn lẻ. Lệnh lỗi ngay tại `ten.strip()`, nên chưa tới bước `lower`.

Để xử lý từng chuỗi trong cột, dùng bộ truy cập `str`:

```
ten_sach = ten.str.strip().str.lower()
```

`str.strip` bỏ khoảng trắng ở hai đầu của mỗi phần tử; `str.lower` chuyển chữ hoa thành chữ thường. Nó không chỉ sửa hàng đầu tiên, cũng không âm thầm bỏ lỗi để chạy tiếp.

Mở rộng / Cách nhớ. `strip` không xóa khoảng trắng nằm giữa các từ. Nếu muốn rút nhiều khoảng trắng giữa từ thành một, dùng thêm `ten_sach.str.replace(r"\s+", " ", regex=True)`. Cần phân biệt chuẩn hóa cách viết với thay đổi ý nghĩa tên gốc.

Mã 047 • Câu 18. (2 điểm)

Bảng chỗ ở còn mã "001" dạng chuỗi. Bảng giá đọc mặc định có mã 1 dạng số. merge theo cột mã, chưa ép kiểu, thì sao?

A. Tự đổi cả hai về chuỗi "001" rồi ghép.
B. Bỏ hàng, vì "001" khác "1".
C. Báo lỗi vì hai cột khoá khác kiểu.
D. Ghép được, vì hai mã cùng một chỗ ở.

Đáp án: C. Báo lỗi do hai cột khóa có kiểu dữ liệu không tương thích.

Giải thích. Trong tình huống in ở đề, một phía là cột chuỗi, phía kia là cột số nguyên. pandas không tự đoán rằng mã chuỗi có số 0 ở đầu nên bằng mã số 1; phép ghép này gây `ValueError`. Vì vậy không phải cứ ghép rồi bỏ hàng không khớp như phương án B.

Mã định danh thường cần giữ nguyên số 0 đầu. Nên khai báo kiểu ngay lúc đọc cả hai nguồn:

```
a = pd.read_csv("cho_o.csv", dtype={"ma": "string"})
b = pd.read_csv("gia.csv", dtype={"ma": "string"})
ket_qua = a.merge(b, on="ma")
```

Mở rộng / Cách nhớ. Nếu mã bên giá đã bị đọc thành 1, đổi thành chuỗi chỉ tạo `"1"`, chưa trở lại `"001"`. Chỉ dùng `str.zfill(3)` khi nguồn xác nhận mã luôn dài ba ký tự. Khác kiểu gây lỗi và cùng kiểu nhưng khác giá trị là hai tình huống khác nhau.

Mã 047 • Câu 19. (2 điểm)

gia_moi có thêm P3, gia_cu không có P3. Ô P3 của gia_moi - gia_cu là gì?

A. P3 biến mất khỏi kết quả.
B. 0, vì thiếu toán hạng được coi là không đổi.
C. Đúng bằng giá mới của P3.
D. NaN, vì thiếu giá cũ; không điền 0.

Đáp án: D. NaN vì thiếu giá cũ; không tự điền 0.

Giải thích. Phép trừ hai `Series` căn theo mã. Nhãn P3 được giữ trong hợp các nhãn, nhưng ở vế `gia_cu` không có giá tương ứng. Vì thế độ chênh tại P3 chưa xác định và được biểu diễn là `NaN`.

P3 không tự biến mất; cũng không nhận kết quả 0 hoặc bằng giá mới. Kết quả 0 sẽ ngầm khẳng định giá không đổi, còn kết quả bằng giá mới ngầm coi giá cũ là 0. Cả hai đều thêm giả định mà đề không cho.

Mở rộng / Cách nhớ. Cú pháp `gia_moi.sub(gia_cu, fill_value=0)` có thể tính với 0 ở nhãn thiếu, nhưng chỉ phù hợp khi nghiệp vụ xác nhận thiếu thật sự có nghĩa là 0. Với một mã mới xuất hiện, thường nên gắn trạng thái ``mới'' thay cho tính phần trăm tăng giá chưa có cơ sở.

Mã 047 • Câu 20. (2 điểm)

contains với dấu chấm đã gạch chéo cộng được bao nhiêu hàng đúng?

```
nhan = pd.Series(["v1.2", "v1x2", "v12", "v1.2a"])
print(int(nhan.str.contains(r"v1\.2").sum()))
```

A. 4
B. 1
C. 3
D. 2

Đáp án: D. 2.

Giải thích. Dấu chấm đã được viết thành `\.` nên chỉ khớp dấu chấm thật. Mẫu tìm đoạn `v1.2`. Hai chuỗi `v1.2` và `v1.2a` đều chứa đoạn này; `v1x2` không có dấu chấm, còn `v12` thiếu ký tự ở vị trí đó.

Kết quả lần lượt là `[True, False, False, True]`, tổng bằng 2. Hậu tố `a` không làm chuỗi cuối bị loại vì `contains` tìm một đoạn con, không yêu cầu khớp toàn bộ chuỗi.

Mở rộng / Cách nhớ. Muốn chỉ chấp nhận đúng nhãn `v1.2`, dùng `nhan.str.fullmatch(r"v1\.2")`; khi đó chỉ một hàng khớp. Nếu bỏ dấu gạch chéo, dấu chấm regex đại diện cho một ký tự bất kỳ, khiến `v1x2` cũng khớp và tổng là 3.

Phần II: Tự luận

Mã 047 • Câu 21. (10 điểm)

Series chỉ có cuối tháng 1, cuối tháng 3 và cuối tháng 4. Không có hàng tháng 2.

```
s = pd.Series(
    [12, 18, 21],
    index=pd.to_datetime(["2026-01-31", "2026-03-31", "2026-04-30"]),
)
print(s.shift(1).loc["2026-03-31"])
```

a) (5 điểm) Dòng in ra số nào? Kỳ liền trước của tháng 3 trong phép shift(1) này là tháng nào? Vì sao không phải giá trị thiếu?

b) (5 điểm) Viết biểu thức chèn các cuối tháng còn thiếu rồi mới shift(1). Ô tháng 3 khi đó là gì? Hàng tháng 2 giữ số nào?

Đáp án: a) In 12.0, lấy từ hàng 31/01. b) Sau khi chèn đủ cuối tháng, hàng tháng 2 có NaN; dịch một hàng thì ô 31/03 cũng là NaN.

a) `shift(1)` không truyền `freq` nên dịch giá trị theo *vị trí hàng*, trong khi các nhãn chỉ mục được giữ nguyên. Hàng ngay trước 31/03 trong bảng hiện có là 31/01, không phải một hàng tháng 2 chưa tồn tại.

Nhãn ngày	Giá trị gốc	Sau `shift(1)`
31/01/2026	12	`NaN`
31/03/2026	18	12
30/04/2026	21	18

Do có ô thiếu ở đầu, kết quả số nguyên thông thường được nâng sang số thực, nên lệnh `print` cho `12.0`. Về giá trị số, đó là 12.

b) Chèn đủ các mốc cuối tháng, giữ tháng chưa có dữ liệu là thiếu, rồi mới dịch:

```
s_du = s.asfreq("ME")
truoc_mot_thang = s_du.shift(1)
print(truoc_mot_thang.loc["2026-03-31"])  # nan
```

Nhãn ngày	Sau `asfreq("ME")`	Sau đó `shift(1)`
31/01/2026	12	`NaN`
28/02/2026	`NaN`	12
31/03/2026	18	`NaN`
30/04/2026	21	18

Mở rộng / Cách nhớ. Tháng 2 trong `s_du` giữ `NaN`, không phải 0. Sau phép dịch, ô tháng 2 mang 12 từ tháng 1; cần nói rõ đang xét bảng nào. `asfreq` phù hợp ở đây vì dữ liệu đã là các mốc cuối tháng. `resample` dùng khi cần gom nhiều quan sát vào từng tháng; `shift(freq="ME")` lại dịch nhãn thời gian, không phải chèn lịch rồi lấy hàng trước.

Mã 047 • Câu 22. (10 điểm)

pd.concat đã nối hai đợt thành bốn hàng. Cùng một mã xuất hiện một lần ở mỗi đợt.

```
dot_1 = pd.DataFrame({"ma": ["P1", "P2"], "dot": [1, 1], "gia": [100, 80]})
dot_2 = pd.DataFrame({"ma": ["P1", "P2"], "dot": [2, 2], "gia": [140, 60]})
ca = pd.concat([dot_1, dot_2], ignore_index=True)
```

a) (5 điểm) ca.groupby("ma")["gia"].mean() cho trung bình của P1 và của P2 là bao nhiêu? Cách nhóm này làm mất điều gì?

b) (5 điểm) Viết groupby để mỗi đợt của mỗi mã giữ một trung bình riêng. Hai giá của P1 trong kết quả là bao nhiêu?

Đáp án: a) Trung bình P1 = 120, P2 = 70; mất sự phân biệt theo đợt. b) Nhóm theo cả mã và đợt; hai giá của P1 vẫn là 100 và 140.

a) Phép `ca.groupby("ma")["gia"].mean()` gom tất cả các hàng cùng mã, bất kể đợt:

$$\overline{g}_{P1} = \frac{100 + 140}{2} = 120, \qquad \overline{g}_{P2} = \frac{80 + 60}{2} = 70.$$

Kết quả chỉ có một dòng cho mỗi mã. Nó không còn cho biết P1 giá 100 ở đợt 1 và 140 ở đợt 2, cũng không giữ được chiều tăng hay giảm giữa các đợt. Đây không phải lỗi của `groupby`; khóa nhóm đã không bao gồm thông tin cần phân biệt.

b) Dùng hai cột làm khóa nhóm:

```
theo_dot = ca.groupby(["ma", "dot"])["gia"].mean()
print(theo_dot)
```

Mã	Đợt	Giá trung bình
P1	1	100.0
P1	2	140.0
P2	1	80.0
P2	2	60.0

Mỗi cặp mã--đợt chỉ có một hàng trong dữ liệu, nên trung bình bằng chính giá của hàng đó. Nếu một cặp có nhiều hàng, `mean` chỉ tổng hợp những hàng thuộc cặp ấy.

Mở rộng / Cách nhớ. Nếu muốn kết quả là bảng có cột thông thường thay vì `MultiIndex`, viết:

```
theo_dot = ca.groupby(["ma", "dot"], as_index=False)["gia"].mean()
```

Trước khi tổng hợp, xác định đơn vị quan sát: ``mỗi mã" hay ``mỗi mã ở mỗi đợt". Đây là quyết định về ý nghĩa phân tích, không chỉ là lựa chọn cú pháp.

Mã 047 • Câu 23. (10 điểm)

Bản sao bảng giá. Cần giảm 10% mỗi chỗ nguyên căn, rồi tính tiền ba đêm từ giá đã giảm.

```
df = pd.DataFrame({
    "loai": ["Nguyên căn", "Phòng riêng", "Nguyên căn"],
    "gia": [200.0, 50.0, np.nan],
})
```

a) (5 điểm) Viết hai dòng: giảm đúng các hàng nguyên căn, rồi tạo ba_dem = gia * 3. Tổng các ô ba_dem đã có số là bao nhiêu? Ô thiếu giá thành gì?

b) (5 điểm) Nếu tạo ba_dem trước rồi mới giảm cột gia, tổng ba_dem là bao nhiêu? Viết lại đúng thứ tự hai dòng.

Đáp án: a) Tổng ba đêm của các ô có số là 690.0; ô thiếu vẫn NaN. b) Nếu tạo ba đêm trước rồi chỉ giảm giá, tổng ba đêm vẫn là 750.0.

a) Hai dòng cần viết, giả sử `df` là bản sao đang chỉnh:

```
df.loc[df["loai"].eq("Nguyên căn"), "gia"] *= 0.9
df["ba_dem"] = df["gia"] * 3
```

Giá 200 giảm còn $200 \times 0.9 = 180$. Phòng riêng giữ giá 50. Giá thiếu của hàng nguyên căn cuối vẫn thiếu vì `NaN` nhân một số vẫn là `NaN`.

Loại	Giá gốc	Giá sau giảm	Ba đêm
Nguyên căn	200	180	540
Phòng riêng	50	50	150
Nguyên căn	NaN	NaN	NaN

Mặc định `df["ba_dem"].sum()` bỏ qua giá trị thiếu, nên tổng các ô đã biết là $540 + 150 = 690$; không có nghĩa tổng tiền của cả ba chỗ ở đã được biết đầy đủ.

b) Nếu tính ba đêm từ giá gốc, cột được tạo là `[600.0, 150.0, NaN]`, tổng $600 + 150 = 750$. Sau đó giảm cột `gia` không làm `ba_dem` tự cập nhật: một cột pandas chứa kết quả tính tại thời điểm gán, không phải công thức sống liên kết như ô bảng tính.

Viết lại đúng thứ tự chính là hai dòng ở ý a. Nếu đã lỡ tính trước, sau khi sửa giá phải gán lại cột tiền:

```
df["ba_dem"] = df["gia"] * 3
```

Mở rộng / Cách nhớ. Nếu cần giữ nguồn, bắt đầu bằng `df = bang_goc.copy()`. Với yêu cầu chỉ chấp nhận tổng khi *mọi* hàng đều có giá, có thể dùng `df["ba_dem"].sum(min_count=len(df))`; dữ liệu này sẽ cho `NaN`. Không điền 0 cho giá thiếu nếu chưa có căn cứ nghiệp vụ.

Mã 047 • Câu 24. (10 điểm)

Nguồn ghi ngày theo ngày/tháng/năm. Đoạn dưới chưa khóa cách đọc.

```
dat = pd.DataFrame({
    "vao": ["05/01/2026", "04/06/2026"],
    "ra": ["02/02/2026", "09/06/2026"],
})
vao = pd.to_datetime(dat["vao"])
ra = pd.to_datetime(dat["ra"])
print((ra - vao).dt.days.sum())
```

a) (5 điểm) Với cách đọc mặc định, hai ngày vào được hiểu là ngày nào, và tổng số đêm in ra là bao nhiêu?

b) (5 điểm) Sửa hai lệnh to_datetime để đọc đúng ngày/tháng/năm. Tổng số đêm của hai lượt là bao nhiêu?

Đáp án: a) Hai ngày vào bị hiểu là 01/05 và 06/04/2026; tổng in ra 65. b) Khóa định dạng ngày/tháng/năm thì tổng đúng là 33 đêm.

a) Không chỉ định định dạng, `dayfirst` mặc định là `False`. Với bốn chuỗi mơ hồ đã cho, pandas hiểu theo tháng/ngày/năm:

Lượt	Ngày vào hiểu được	Ngày ra hiểu được	Chênh lệch
1	01/05/2026	02/02/2026	-88 ngày
2	06/04/2026	06/09/2026	153 ngày

Lượt 1 có ngày ra trước ngày vào, nên hiệu âm. Tổng máy in là $-88 + 153 = 65$, dù cách hiểu này sai so với nguồn. Riêng lượt 2: từ 06/04 tới 06/09 là $30 + 31 + 30 + 31 + 31 = 153$ ngày.

b) Hai lệnh đọc ngày nên sửa thành:

```
vao = pd.to_datetime(dat["vao"], format="%d/%m/%Y")
ra = pd.to_datetime(dat["ra"], format="%d/%m/%Y")
print((ra - vao).dt.days.sum())  # 33
```

Lượt	Ngày vào đúng	Ngày ra đúng	Số đêm
1	05/01/2026	02/02/2026	28
2	04/06/2026	09/06/2026	5

Lượt 1: $02/02 - 05/01 = 28$ ngày; lượt 2: $9 - 4 = 5$ ngày; tổng $28 + 5 = 33$ đêm. Với lưu trú, không cộng thêm 1 vào hiệu ngày.

Mở rộng / Cách nhớ. `dayfirst=True` cũng cho kết quả đúng ở bộ dữ liệu này, nhưng `format` mô tả rõ hợp đồng dữ liệu. Nên kiểm tra `(ra >= vao).all()` sau khi đọc. Một tổng dương không bảo đảm mọi lượt đều hợp lệ: ở ý a, hiệu âm bị che bởi hiệu dương lớn hơn.

Mã 047 • Câu 25. (10 điểm)

Phản hồi đã đọc được. Trường cod là 200. temp_f là nhiệt độ cao nhất theo độ F.

```
payload = {
    "cod": 200,
    "daily": {"ngay": ["2026-11-05", "2026-11-06"],
              "temp_f": [77, 59]}
}
```

a) (5 điểm) 77°F và 59°F bằng bao nhiêu °C? Vì sao cod bằng 200 chưa đủ để cộng các số trong temp_f như nhiệt độ °C?

b) (5 điểm) Viết code tạo Series từ temp_f và đổi sang °C bằng (f - 32) * 5 / 9. Tổng hai ngày theo °C là bao nhiêu?

Đáp án: a) 77°F = 25°C; 59°F = 15°C. Trường cod bằng 200 không xác nhận đơn vị là °C. b) Series đổi sang °C có hai giá trị 25 và 15; tổng là 40.0.

a) Công thức đổi đơn vị là

$$C = (F - 32)\frac{5}{9}.$$

Thay từng giá trị vào:

$$(77 - 32)\frac{5}{9} = 45\frac{5}{9} = 25, \qquad (59 - 32)\frac{5}{9} = 27\frac{5}{9} = 15.$$

Trường `cod = 200` trong nội dung phản hồi thể hiện một mã trạng thái do API quy định, không biến trường `temp_f` thành độ C. Hơn nữa, mã trạng thái trong payload và mã HTTP `response.status_code` là hai trường cần phân biệt. Dù phản hồi thành công, vẫn phải kiểm tra ý nghĩa, đơn vị, cấu trúc và kiểu dữ liệu.

b) Tạo `Series`, đổi từng phần tử rồi tính tổng:

```
f = pd.Series(payload["daily"]["temp_f"], dtype="float64")
c = (f - 32) * 5 / 9
print(c.tolist())  # [25.0, 15.0]
print(c.sum())     # 40.0
```

Để giữ ngày đi kèm nhiệt độ, có thể dùng ngày làm chỉ mục:

```
f = pd.Series(payload["daily"]["temp_f"],
              index=pd.to_datetime(payload["daily"]["ngay"]),
              dtype="float64")
c = (f - 32) * 5 / 9
```

Mở rộng / Cách nhớ. Đổi Fahrenheit sang Celsius là phép biến đổi affine, có phần trừ 32. Vì vậy không được cộng hai số F rồi chỉ đổi một lần:

$$\sum_{i=1}^{n} C_i = \frac{5}{9}\left(\sum_{i=1}^{n} F_i - 32n\right).$$

Với $n = 2$, tổng là $\frac{5}{9}(136 - 64) = 40$. Đây là tổng số học theo yêu cầu đề; nếu báo cáo mức nhiệt điển hình, trung bình hai ngày là 20°C và thường có ý nghĩa hơn tổng nhiệt độ.

Mã 047 • Câu 26. (10 điểm)

Năm tên chỗ ở. Mẫu lấy số mét vuông đứng thành một từ, không phân biệt hoa thường.

```
ten = pd.Series([
    "Depto 20 m2", "Studio 15m²", "100 M2 luxury",
    "Torre 2024", "12m2x",
])
pat = r"\b(\d+)\s*m(?:2|²)\b"
so = pd.to_numeric(ten.str.extract(pat, flags=re.I, expand=False), errors="coerce")
```

a) (5 điểm) so.sum() bằng bao nhiêu? Hai tên nào không có diện tích trích được, và vì sao không được cộng 0 cho hai tên đó khi người ta muốn trung bình của các tên khớp?

b) (5 điểm) Bỏ flags=re.I thì sum còn bao nhiêu? Viết lại dòng extract để lấy đủ như ý a.

Đáp án: a) Tổng = 135.0; hai tên không khớp là Torre 2024 và 12m2x. Trung bình trên các tên khớp = 45.0. b) Bỏ re.I thì tổng = 35.0.

a) Phân tích mẫu: `\b` là ranh giới từ; `(\d+)` bắt ít nhất một chữ số; `\s*` cho phép không có hoặc có nhiều khoảng trắng; `m(?:2²)|` nhận `m2` hoặc `m²`. `(?:...)` không tạo nhóm trích xuất. `re.I` cho phép `M` hoa khớp với `m` thường.

Tên	Giá trị	Lý do
`Depto 20 m2`	20	Đúng số, khoảng trắng và đơn vị.
`Studio 15m²`	15	Khoảng trắng có thể vắng; nhận ký tự `²`.
`100 M2 luxury`	100	Chữ M hoa được nhận nhờ `re.I`.
`Torre 2024`	`NaN`	Không có đơn vị diện tích.
`12m2x`	`NaN`	Sau 2 là x; cả hai là ký tự từ, nên không có ranh giới cuối.

Tổng là $20 + 15 + 100 = 135$; `sum` mặc định bỏ giá trị thiếu. Nếu muốn trung bình của *các tên khớp*, mẫu số phải là 3:

$$\overline{x} = \frac{135}{3} = 45.$$

Điền 0 cho hai tên không khớp biến kết quả thành $135/5 = 27$, ngầm nói hai tên đó có diện tích bằng 0. ``Không trích được'' không đồng nghĩa ``diện tích 0''.

b) Khi bỏ `re.I`, tên `100 M2 luxury` không khớp nữa; còn $20 + 15 = 35$. Sửa dòng trích xuất như sau (giả sử đã `import re`):

```
trich = ten.str.extract(pat, flags=re.IGNORECASE, expand=False)
so = pd.to_numeric(trich, errors="coerce")
```

Mở rộng / Cách nhớ. Có thể đặt cờ trong mẫu bằng `r"(?i)\b(\d+)\s*m(?:2²)"`⊥. Mẫu hiện tại chỉ nhận số nguyên; để nhận số thập phân cần thiết kế thêm dấu thập phân và ranh giới đầu phù hợp, tránh vô tình bắt riêng phần đuôi của một số như `12.5m2`.

Mã đề 045

Đáp án nhanh phần trắc nghiệm

Câu	1	2	3	4	5	6	7	8	9	10
Đáp án	B	C	A	C	B	C	B	D	A	D
Câu	11	12	13	14	15	16	17	18	19	20
Đáp án	B	C	D	B	D	D	A	A	C	A

Đã đủ câu 24--26 từ trang ảnh bổ sung. Câu 1 được kiểm chứng theo lời gọi có dayfirst=True; câu 20 giữ nguyên mẫu không có ranh giới cuối.

Đi đến từng câu Câu 1 • Câu 2 • Câu 3 • Câu 4 • Câu 5 • Câu 6 • Câu 7 • Câu 8 • Câu 9 • Câu 10 • Câu 11 • Câu 12 • Câu 13

Câu 14 • Câu 15 • Câu 16 • Câu 17 • Câu 18 • Câu 19 • Câu 20 • Câu 21 • Câu 22 • Câu 23 • Câu 24 • Câu 25 • Câu 26

Đã có đủ câu 1--26. Phần đề nằm trước hộp đáp án; các bản sửa code chỉ nằm trong lời giải.

Phần I: Trắc nghiệm

Mã 045 • Câu 1. (2 điểm)

```
pd.to_datetime("2026-04-08", dayfirst=True)
```

Biểu thức này cho ngày nào?

A. 2026-08-08
B. 2026-08-04
C. NaT
D. 2026-04-08

Đáp án: B. 2026-08-04, tức ngày 04 tháng 08 năm 2026.

Giải thích. Trong lời gọi không có `format` này, `dayfirst=True` ưu tiên hiểu phần ngày trước phần tháng. Với chuỗi đang xét, năm vẫn là 2026; hai trường còn lại được hiểu là ngày 04, tháng 08. Kết quả kiểm chứng là `Timestamp('2026-08-04 00:00:00')`.

Hình dạng có năm ở đầu không đủ để bỏ qua tham số `dayfirst`. Tuy nhiên, `dayfirst` là ưu tiên của bộ phân tích, không phải một hợp đồng định dạng nghiêm ngặt cho mọi chuỗi có thể gặp.

Mở rộng / Cách nhớ. Nếu ý định là đọc đúng năm-tháng-ngày để lấy ngày 08/04/2026, khóa định dạng:

```
pd.to_datetime("2026-04-08", format="%Y-%m-%d")
# Timestamp('2026-04-08 00:00:00')
```

Quy ước của nguồn nên được ghi rõ; không để bộ đọc tự đoán trong các bài toán ngày có thể bị đảo.

Mã 045 • Câu 2. (2 điểm)

pd.Series(["a.b", "axb"]).str.contains(".", regex=False) cho hai giá trị nào?

A. True, True
B. False, False
C. True, False
D. False, True

Đáp án: C. True, False.

Giải thích. Khi `regex=False`, dấu chấm được tìm như một ký tự văn bản thông thường. Chuỗi `a.b` chứa dấu chấm nên trả `True`; chuỗi `axb` không chứa dấu chấm nên trả `False`.

Nếu dùng regex, dấu chấm không escape thường khớp một ký tự bất kỳ, nên cả hai chuỗi đều có thể khớp. Đề đã tắt regex nên không áp dụng ý nghĩa đặc biệt đó.

Mở rộng / Cách nhớ. Ba cách tương đương để tìm dấu chấm thật:

```
s.str.contains(".", regex=False)
s.str.contains(r"\.", regex=True)
s.str.contains(r"[.]", regex=True)
```

Chọn cách thể hiện rõ ý định nhất; với một đoạn văn bản cố định, `regex=False` thường dễ đọc.

Mã 045 • Câu 3. (2 điểm)

pd.Timestamp("2026-01-31 21:30", tz="UTC") cần được hiển thị theo Asia/Ho_Chi_Minh. Dùng hàm nào?

A. tz_convert, vì mốc này đã mang múi giờ.
B. Không đổi gì, vì UTC và giờ Việt Nam luôn cùng ngày.
C. tz_localize, vì mốc UTC chưa có múi giờ.
D. tz_convert rồi tz_localize để gắn thêm giờ Việt Nam.

Đáp án: A. Dùng tz_convert vì mốc đã có múi giờ. Kết quả là 01/02/2026 lúc 04:30, UTC+07:00.

Giải thích. `tz="UTC"` cho biết thời điểm đã được gắn múi giờ. Muốn giữ nguyên thời điểm thực nhưng đổi đồng hồ hiển thị sang giờ Việt Nam, dùng:

```
t = pd.Timestamp("2026-01-31 21:30", tz="UTC")
print(t.tz_convert("Asia/Ho_Chi_Minh"))
# 2026-02-01 04:30:00+07:00
```

Giờ Việt Nam đi trước UTC 7 giờ; $21{:}30 + 7$ giờ bằng $04{:}30$ ngày kế tiếp. Vì thế không thể khẳng định ngày UTC và ngày Việt Nam luôn giống nhau.

`tz_localize` dùng để gắn múi giờ cho mốc chưa có múi giờ; gọi trực tiếp nó để gắn thêm một múi giờ vào mốc đã có múi giờ không phải thao tác đúng trong đề.

Mở rộng / Cách nhớ. Nếu chuỗi nguồn chưa có múi giờ nhưng được xác nhận là giờ UTC, làm hai bước: `t.tz_localize("UTC").tz_convert("Asia/Ho_Chi_Minh")`. Nếu nguồn vốn là giờ địa phương, phải gắn đúng múi giờ nguồn trước.

Mã 045 • Câu 4. (2 điểm)

```
df = pd.DataFrame({
    "loai": ["Nguyên căn", "Phòng riêng", "Nguyên căn"],
    "gia": [100.0, 50.0, np.nan],
}, index=["a", "b", "c"])
print((df["gia"] * 2).loc["c"])
```

Giá trị in ra là gì?

A. Báo lỗi vì gặp giá trị thiếu
B. 0.0
C. NaN
D. 100.0

Đáp án: C. NaN (lệnh print hiển thị nan).

Giải thích. Cột giá có nhãn `c` mang `np.nan`. Nhân cả cột với 2 tạo các giá trị `[200.0, 100.0, NaN]` và giữ chỉ mục `["a", "b", "c"]`. Lấy `.loc["c"]` trả giá trị thiếu của hàng cuối.

Phép nhân không tự điền 0, không lấy giá của hàng khác và không báo lỗi chỉ vì có `NaN` trong cột số thực.

Mở rộng / Cách nhớ. Dùng `pd.isna(x)` hoặc `s.isna()` để kiểm tra thiếu. Không kiểm tra bằng `x == np.nan`, vì `NaN` không bằng chính nó theo quy tắc số thực dấu phẩy động.

Mã 045 • Câu 5. (2 điểm)

Chỉ cần hai cột của một bảng rất rộng. Một lần đọc CSV với usecols, một lần đọc Parquet và chỉ lấy hai cột đó. Khác nhau nào đúng?

A. Parquet luôn nhỏ hơn và luôn nhanh hơn CSV.
B. CSV vẫn quét cả dòng; Parquet có thể không đọc cột không cần.
C. Cả CSV lẫn Parquet đều đọc nguyên mọi cột vào bộ nhớ.
D. CSV bỏ qua được phần văn bản của cột không cần.

Đáp án: B. CSV vẫn phải quét dữ liệu theo dòng; Parquet có thể tránh đọc các cột không cần.

Giải thích. CSV là văn bản tổ chức theo dòng. Bộ đọc phải tìm dấu phân cách, dấu nháy và ranh giới dòng để nhận biết các trường; `usecols` giúp chỉ giữ các cột cần vào kết quả nhưng không biến CSV thành định dạng có thể truy cập từng cột độc lập.

Parquet tổ chức dữ liệu theo cột và có siêu dữ liệu về cấu trúc. Với tùy chọn `columns`, bộ đọc có thể chỉ đọc dữ liệu của các cột cần thiết. Điều này thường tiết kiệm I/O với bảng rất rộng.

```
a = pd.read_csv("bang.csv", usecols=["ma", "gia"])
b = pd.read_parquet("bang.parquet", columns=["ma", "gia"])
```

Mở rộng / Cách nhớ. Không khẳng định Parquet luôn nhỏ hơn hoặc luôn nhanh hơn CSV trong mọi trường hợp. Kích thước, nén, bộ đọc, bộ nhớ đệm và cách chọn dữ liệu đều ảnh hưởng. Điểm phân biệt trong câu hỏi là khả năng chỉ đọc cột nhờ cấu trúc lưu trữ.

Mã 045 • Câu 6. (2 điểm)

```
cu = pd.Series([10, 4], index=["P1", "P2"])
moi = pd.Series([7, 12], index=["P2", "P1"])
print((moi.to_numpy() - cu.to_numpy()).tolist())
```

Dòng in ra gì?

A. [2, 8]
B. [2, 3]
C. [-3, 8]
D. [3, 2]

Đáp án: C. [-3, 8].

Giải thích. `to_numpy` lấy các giá trị theo thứ tự hiện tại và bỏ thông tin nhãn. Mảng mới là `[7, 12]`, mảng cũ là `[10, 4]`. NumPy trừ theo vị trí:

$$[7-10,\ 12-4] = [-3,\ 8].$$

Kết quả không phải chênh lệch đúng theo mã: vị trí đầu đã trừ giá mới của P2 với giá cũ của P1.

Mở rộng / Cách nhớ. Nếu muốn so đúng từng mã, giữ nguyên hai `Series`:

```
print(moi - cu)
# P1    2  (12 - 10)
# P2    3  ( 7 -  4)
```

Chỉ chuyển sang mảng sau khi đã xác nhận các vị trí ở hai vế tương ứng đúng với nhau.

Mã 045 • Câu 7. (2 điểm)

```
s = pd.Series([10, 20, 30], index=["a", "b", "c"])
print(int(s.loc["a":"b"].sum()), int(s.iloc[0:1].sum()))
```

Hai số in ra là gì?

A. 10 10
B. 30 10
C. 30 30
D. 60 10

Đáp án: B. 30 và 10.

Giải thích. `loc` cắt theo nhãn và trong phép cắt nhãn ở đây lấy cả hai đầu: từ `a` tới `b` bao gồm 10 và 20, tổng 30.

`iloc` cắt theo vị trí nguyên với quy ước Python: lấy đầu, bỏ cuối. Đoạn `0:1` chỉ lấy vị trí 0, nên tổng là 10. Kết quả không phụ thuộc việc các nhãn là chữ hay số nếu ta đã dùng đúng bộ truy cập.

Mở rộng / Cách nhớ. Muốn lấy hai vị trí đầu, dùng `s.iloc[0:2]`. Muốn lấy nhãn `a` và `b` một cách tường minh, dùng `s.loc[["a", "b"]]`. Luôn phân biệt một nhãn có giá trị 0 với vị trí số 0.

Mã 045 • Câu 8. (2 điểm)

Theo chính sách bài học, trường hợp nào vừa không bị coi là gian lận, vừa không bị 0 điểm cá nhân ở phần code đó?

A. Chỉ khai báo sau khi giảng viên hỏi tới đoạn đó.
B. Nhờ AI viết, không khai báo vì đoạn code ngắn.
C. Có khai báo, nhưng vấn đáp không giải thích được hàm đó.
D. Hiểu từng dòng, đã kiểm trên mẫu tự tính, và có khai báo.

Đáp án: D. Hiểu từng dòng, đã kiểm trên mẫu tự tính và có khai báo.

Giải thích. Trong bốn lựa chọn, D đồng thời thể hiện ba yêu cầu: công khai việc dùng AI, tự hiểu chương trình, và tự kiểm tra đầu ra trên một mẫu có thể tính độc lập.

A chỉ khai báo sau khi bị hỏi nên không đáp ứng yêu cầu minh bạch ngay từ đầu. B dùng AI nhưng không khai báo. C có khai báo nhưng không giải thích được hành vi của code; việc khai báo một mình chưa chứng minh năng lực cá nhân ở phần đó.

Mở rộng / Cách nhớ. Khi dùng AI học lập trình, hãy giữ một ví dụ nhỏ tự tính được, dự đoán kết quả trước khi chạy và kiểm tra các trường hợp thiếu dữ liệu hoặc khóa không khớp. Đây là cách hiểu và kiểm chứng code; các quy định đánh giá cụ thể vẫn theo chính sách của lớp học được câu hỏi viện dẫn.

Mã 045 • Câu 9. (2 điểm)

Ngân sách: nguyên căn ≤ 100, phòng riêng ≤ 60. Bảng có a nguyên căn 100, b phòng riêng 60, c nguyên căn 101, d phòng riêng 100. Những hàng nào được giữ?

A. a và b
B. b và d
C. a và c
D. chỉ b

Đáp án: A. a và b.

Giải thích. Mỗi loại có ngưỡng riêng, và điều kiện dùng dấu $\leq$ nên nhận cả giá bằng ngưỡng:

Hàng	Loại	Giá	Ngưỡng	Đạt?
a	Nguyên căn	100	100	Có
b	Phòng riêng	60	60	Có
c	Nguyên căn	101	100	Không
d	Phòng riêng	100	60	Không

Hàng d không được dùng ngưỡng 100 của nguyên căn chỉ vì giá của nó là 100.

Mở rộng / Cách nhớ. Mặt nạ biểu diễn đúng hai trường hợp là:

```
mask = ((df["loai"].eq("Nguyên căn") & df["gia"].le(100))
        | (df["loai"].eq("Phòng riêng") & df["gia"].le(60)))
ket_qua = df.loc[mask]
```

Với `Series`, dùng `&` và `|` cho từng phần tử, đồng thời đặt ngoặc quanh mỗi điều kiện; không dùng `and`/`or` của một Boolean đơn lẻ.

Mã 045 • Câu 10. (2 điểm)

extract dưới đây in ra list nào?

```
ma = pd.Series(["ID-7a", "ID-12", "7"])
print(ma.str.extract(r"^ID-(\d+)$")[0].tolist())
```

A. ["7", "12", NaN]
B. ["7", "12", "7"]
C. [NaN, NaN, NaN]
D. [NaN, "12", NaN]

Đáp án: D. [nan, "12", nan].

Giải thích. Dấu `^` yêu cầu bắt đầu chuỗi, `$` yêu cầu kết thúc chuỗi. Phần `ID-` phải đúng nguyên dạng; nhóm `(\d+)` bắt một hoặc nhiều chữ số.

`ID-7a` bị loại vì có chữ a sau số; `ID-12` khớp và nhóm bắt trả *chuỗi* `"12"`; `7` thiếu tiền tố `ID-`. Các chuỗi không khớp cho giá trị thiếu. `[0]` chọn cột của nhóm trích xuất duy nhất.

Mở rộng / Cách nhớ. Nếu cần dùng kết quả để tính toán, chuyển bằng `pd.to_numeric`. Nếu chỉ muốn kiểm tra mã hợp lệ mà không lấy số, `ma.str.fullmatch(r"ID-\d+")` trả trực tiếp mặt nạ Boolean.

Mã 045 • Câu 11. (2 điểm)

Với một mảng số dài, vì sao x * 2 thường nhanh hơn vòng for Python làm cùng phép nhân?

A. NumPy bỏ hẳn vòng lặp, CPU không còn nhân từng số.
B. Phép nhân chạy trong mã đã biên dịch sẵn, đỡ xử lý đối tượng Python từng lượt.
C. Python biên dịch vòng for của người viết thành mã máy ngay lúc gọi.
D. x * 2 luôn chia việc cho mọi lõi CPU.

Đáp án: B. Phép nhân chạy trong mã đã biên dịch sẵn, giảm xử lý đối tượng Python ở từng lượt.

Giải thích. Mảng NumPy có dtype cụ thể và phép toán được triển khai bằng các vòng lặp mức thấp trong mã đã biên dịch. Python chỉ cần yêu cầu thực hiện một phép toán mảng, thay vì điều khiển từng lượt lặp và xử lý từng phần tử như đối tượng Python.

Vẫn có công việc nhân từng phần tử và vẫn có vòng lặp ở mức triển khai. Vì vậy A sai khi nói CPU không còn nhân từng số. C sai vì Python không tự động biên dịch mọi vòng `for` của người dùng thành mã máy ngay lúc gọi. D sai vì `x * 2` không bảo đảm chia việc cho tất cả lõi CPU.

Mở rộng / Cách nhớ. Lợi thế phụ thuộc kích thước mảng, dtype và tác vụ. Với mảng rất nhỏ, chi phí gọi hàm hoặc cấp phát có thể đáng kể; với dtype `object`, lợi ích có thể giảm vì vẫn phải xử lý đối tượng Python. Vector hóa cũng không tự loại được tràn số của dtype hẹp.

Mã 045 • Câu 12. (2 điểm)

Mã trạng thái là 404 và nội dung vẫn là JSON báo lỗi. Gọi r.json() mà không gọi raise_for_status(). Kết quả nào đúng?

A. json() luôn ném lỗi vì mã khác 200.
B. json() tự đổi mã trạng thái thành 200.
C. Có thể nhận dict của thông báo lỗi, không có lỗi HTTP.
D. json() trả None với mọi mã từ 400 trở lên.

Đáp án: C. Có thể nhận dict thông báo lỗi, không tự phát sinh lỗi HTTP chỉ từ r.json().

Giải thích. `r.json()` giải mã phần thân phản hồi thành đối tượng Python. Nếu phần thân là một JSON object hợp lệ, nó có thể trả về `dict` dù HTTP là 404.

`raise_for_status()` mới là lời gọi dùng để phát sinh ngoại lệ cho trạng thái HTTP lỗi. Giải mã JSON thành công không đổi trạng thái 404 thành 200 và không chứng minh yêu cầu đã thành công. Nếu phần thân không phải JSON hợp lệ, `r.json()` có thể báo lỗi giải mã JSON, là một loại lỗi khác.

Mở rộng / Cách nhớ. Một luồng xử lý phổ biến là:

```
r = requests.get(url, timeout=10)
r.raise_for_status()
payload = r.json()
```

Nếu cần lấy chi tiết thông báo lỗi từ JSON, có thể đọc phần thân khi xử lý ngoại lệ, đồng thời vẫn giữ thông tin trạng thái HTTP để quyết định bước tiếp theo.

Mã 045 • Câu 13. (2 điểm)

time và temperature_2m_max đang cùng thứ tự. Chỉ sắp danh sách nhiệt độ giảm dần trước khi ghép cột thì sao?

A. Pandas tự ghép lại theo ngày.
B. Lệnh ghép báo lỗi vì hai danh sách lệch nhau.
C. Mỗi nhiệt độ vẫn đi với đúng ngày của nó.
D. Nhiệt độ lớn nhất dính vào ngày đang đứng đầu.

Đáp án: D. Nhiệt độ lớn nhất bị gán vào ngày đang đứng đầu.

Giải thích. Quan hệ giữa hai danh sách được xác định theo vị trí: ngày thứ i đi cùng nhiệt độ thứ i. Sắp riêng nhiệt độ làm thay đổi vị trí của nó trong khi danh sách ngày giữ nguyên. Khi ghép theo vị trí, các cặp ngày--nhiệt độ bị thay đổi.

Ví dụ ngày `["ngày 1", "ngày 2"]` và nhiệt độ `[20, 30]` ban đầu cho ngày 1 là 20, ngày 2 là 30. Sắp riêng nhiệt độ thành `[30, 20]` sẽ gán 30 vào ngày 1. pandas không tự biết nhiệt độ đã từng thuộc ngày nào, và hai danh sách vẫn dài bằng nhau nên không nhất thiết báo lỗi.

Mở rộng / Cách nhớ. Hãy ghép thành bảng trước rồi sắp cả các hàng:

```
df = pd.DataFrame({"time": time,
                   "temperature_2m_max": temperature_2m_max})
df = df.sort_values("temperature_2m_max", ascending=False)
```

Khi sắp theo hàng, mỗi nhiệt độ luôn đi cùng ngày của chính nó.

Mã 045 • Câu 14. (2 điểm)

2026-02-28 có dayofweek bằng 5, tức thứ Bảy. 2026-03-01 có dayofweek bằng bao nhiêu? Thứ Hai là 0.

A. 0
B. 6
C. 1
D. 5

Đáp án: B. 6.

Giải thích. Quy ước là thứ Hai 0, thứ Ba 1, thứ Tư 2, thứ Năm 3, thứ Sáu 4, thứ Bảy 5, Chủ nhật 6. Ngày 01/03 là ngày liền sau 28/02 trong năm 2026, nên là Chủ nhật và có giá trị 6.

$$\text{dayofweek ngày sau} = (\text{dayofweek ngày trước} + 1) \bmod 7.$$

Mở rộng / Cách nhớ. Có thể lấy bằng `pd.Timestamp("2026-03-01").dayofweek`. Đừng nhầm với `isoweekday()`, có quy ước thứ Hai là 1 và Chủ nhật là 7. Với cột ngày, dùng `df["ngay"].dt.dayofweek`.

Mã 045 • Câu 15. (2 điểm)

```
moi = co_gia.assign(gia_nghin=co_gia["price"] / 1000)
```

Sau lệnh này, co_gia["price"] thế nào?

A. Bị chia cho 1000.
B. Thành NaN vì assign xoá cột cũ.
C. Chỉ đổi sau lần chạy thứ hai của cùng ô.
D. Giữ nguyên đơn vị cũ; cột mới nằm ở moi.

Đáp án: D. Cột price ở co_gia giữ nguyên đơn vị cũ; cột gia_nghin nằm trong moi.

Giải thích. `assign` trả một `DataFrame` kết quả có thêm hoặc thay cột được nêu tên. Ở đây cột được gán là `gia_nghin`, không phải `price`. Phép chia tạo ra các giá trị cho cột mới; không gán ngược vào cột `price` của bảng nguồn.

Ví dụ giá gốc 100000 tạo `gia_nghin = 100` trong `moi`. Bảng `moi` vẫn có `price = 100000`; bảng `co_gia` cũng giữ `price = 100000`. Gọi lại cùng dòng không làm giá nguồn bị chia tiếp.

Mở rộng / Cách nhớ. Cách viết thuận tiện cho chuỗi thao tác:

```
moi = co_gia.assign(gia_nghin=lambda d: d["price"] / 1000)
```

Đặt tên cột kèm đơn vị giúp tránh nhầm. Muốn thay cột giá gốc, phải có phép gán chỉ rõ cột đó; đó là một thao tác khác với dòng của đề.

Mã 045 • Câu 16. (2 điểm)

extract với lượng từ * in ra list nào?

```
don_vi = pd.Series(["m2", "12m2"])
print(don_vi.str.extract(r"(\d*)m2")[0].tolist())
```

A. ["m2", "12m2"]
B. ["0", "12"]
C. [NaN, "12"]
D. ["", "12"]

Đáp án: D. ["", "12"].

Giải thích. `\d*` cho phép không có hoặc có nhiều chữ số. Với chuỗi `m2`, nhóm số bắt *chuỗi rỗng*; phần `m2` phía sau vẫn khớp, nên đây là một lần khớp thành công. Với `12m2`, nhóm bắt được `"12"`.

Chuỗi rỗng khác `NaN`. Chỉ khi toàn bộ mẫu không khớp thì `extract` mới tạo giá trị thiếu cho hàng đó.

Mở rộng / Cách nhớ. Muốn bắt buộc có ít nhất một chữ số, thay `\d*` bằng `\d+`. Khi ấy chuỗi `m2` không khớp và cho `NaN`, chuỗi `12m2` vẫn cho `"12"`. Khi chuyển chuỗi rỗng sang số bằng `to_numeric(errors="coerce")`, chuỗi rỗng sẽ thành giá trị thiếu.

Mã 045 • Câu 17. (2 điểm)

```
ten = pd.Series(["metro center", pd.NA], dtype="string")
mask = ten.str.contains("metro", na=False)
ket_qua = ten[~mask]
```

Hàng tên thiếu có nằm trong ket_qua không?

A. Có, vì False bị phủ định thành True.
B. Có, và ô đó biến thành chuỗi "metro".
C. Không, vì giá trị thiếu bị xoá đi từ trước.
D. Không, vì ~ bỏ qua ô thiếu.

Đáp án: A. Có, vì False ở hàng thiếu bị phủ định thành True.

Giải thích. Chuỗi đầu có `metro` nên nhận `True`. Hàng thiếu được tham số `na=False` gán kết quả tìm kiếm là `False`. Mặt nạ là `[True, False]`; phủ định bằng ~ tạo `[False, True]`, nên hàng thiếu được giữ.

`na=False` chỉ quy định giá trị của *mặt nạ*, không thay tên thiếu thành chuỗi `"metro"` hay thành `False`. Giá trị trong `ket_qua` vẫn là `pd.NA` ở nhãn 1.

Mở rộng / Cách nhớ. Nếu muốn chỉ giữ các tên *có dữ liệu* và không chứa `metro`, thêm điều kiện:

```
ket_qua = ten[ten.notna() & ~mask]
```

Khi phủ định một điều kiện có xử lý thiếu, nên kiểm tra lại hàng thiếu được phân vào nhóm nào.

Mã 045 • Câu 18. (2 điểm)

Khu C có trong bảng vùng nhưng ô vùng của C là NaN. Sau left join với indicator=True, hàng P3 thuộc khu C có _merge là gì?

A. both
B. right_only
C. NaN
D. left_only

Đáp án: A. both.

Giải thích. Cột `_merge` báo tình trạng *khớp khóa*, không báo thuộc tính sau ghép có đầy đủ hay không. Hàng P3 có khóa C ở bảng trái; bảng phải cũng có khóa C, nên đây là hàng khớp từ cả hai bảng: `both`.

Ô vùng vẫn có thể là `NaN` vì giá trị thuộc tính vốn thiếu ở bản ghi C của bảng phải. Chỉ khi khóa C không có trong bảng phải thì hàng này mới là `left_only`. Với `left join`, các hàng chỉ có bên phải không được đưa thêm vào kết quả.

Mở rộng / Cách nhớ. Cần hai kiểm tra riêng: `ket_qua["_merge"].eq("left_only")` để tìm khóa không ghép được, và `ket_qua["vung"].isna()` để tìm thuộc tính vùng thiếu. Một hàng `both` vẫn có thể cần bổ sung dữ liệu.

Mã 045 • Câu 19. (2 điểm)

df có nhãn hàng a, b, c và hai cột loai, gia. df.shape là gì?

A. (3, 3)
B. (3, 1)
C. (3, 2)
D. (2, 3)

Đáp án: C. (3, 2).

Giải thích. `DataFrame.shape` trả một tuple theo thứ tự `(số hàng, số cột)`. Có ba nhãn hàng và hai tên cột nên kết quả là `(3, 2)`. Chỉ mục hàng không được tính như một cột dữ liệu bổ sung trong `shape`.

Các ô thiếu vẫn nằm trong hình dạng bảng. Một giá trị `NaN` không làm số hàng hoặc số cột giảm đi; `shape` đo cấu trúc, không đếm số ô có dữ liệu.

Mở rộng / Cách nhớ. `len(df)` cho 3, `len(df.columns)` cho 2 và `df.size` cho $3 \times 2 = 6$ ô. Còn `df.count()` trả số ô không thiếu theo từng cột, nên không đồng nghĩa với `shape`.

Mã 045 • Câu 20. (2 điểm)

Mẫu dưới đây không có \b ở cuối. extract trên "30m2x" trả gì?

```
ten = pd.Series(["30m2x"])
print(ten.str.extract(r"\b(\d+)\s*m(?:2|²)")[0].iloc[0])
```

A. "30"
B. "30m2x"
C. NaN
D. "x"

Đáp án: A. "30" (chuỗi).

Giải thích. Mẫu in trong đề là `r"\b(\d+)\s*m(?:2²)"|`, chỉ có ranh giới từ ở đầu và *không có* ranh giới từ ở cuối. Vì vậy, mẫu khớp được đoạn `30m2` nằm ở đầu chuỗi `30m2x`; chữ x phía sau không làm lần khớp bị loại.

Nhóm bắt `(\d+)` lấy hai chữ số 30. `extract` trả nội dung nhóm bắt, nên kết quả là chuỗi `"30"`, không phải toàn bộ `"30m2x"`, chữ x hay giá trị thiếu. Sau khi chọn cột nhóm bằng `[0]` rồi lấy phần tử đầu bằng `iloc[0]`, lệnh `print` hiển thị `30` không kèm dấu nháy; đáp án A dùng dấu nháy để biểu diễn giá trị chuỗi.

Mở rộng / Cách nhớ. Nếu thêm `\b` ở cuối thành `r"\b(\d+)\s*m(?:2²)"┴` thì chuỗi này mới không khớp, vì giữa 2 và x không có ranh giới từ. Đây là điểm khác với mẫu của câu 26 mã đề 047: một ký hiệu thêm hoặc bớt có thể đổi đáp án.

Phần II: Tự luận

Mã 045 • Câu 21. (10 điểm)

Tệp gia.csv có nội dung:

```
khu,gia
A,100
B,?
C,50
```

Một bạn đọc mặc định rồi ghi Parquet ngay:

```
df = pd.read_csv("gia.csv")
df.to_parquet("gia.parquet")
lai = pd.read_parquet("gia.parquet")
print(lai["gia"].mean())
```

a) (5 điểm) Vì sao dòng mean báo lỗi? Dấu ? đang nằm trong cột giá dưới dạng gì?

b) (5 điểm) Sửa cách đọc CSV để sau khi ghi và đọc lại Parquet, mean chạy được. Kết quả là bao nhiêu?

Đáp án: a) Dấu ? chưa được khai báo là thiếu; cột chứa chuỗi, nên mean báo lỗi kiểu dữ liệu. b) Đọc CSV với na_values=["?"], ghi rồi đọc lại Parquet thì trung bình = 75.0.

a) Với cách đọc mặc định, `?` được giữ như văn bản. Cột giá vì thế mang các chuỗi `["100", "?", "50"]`, thường có dtype `object` ở pandas 2.x hoặc `str` ở pandas 3.x. `mean` yêu cầu dữ liệu số; nó không tự biết dấu hỏi đại diện cho giá thiếu nên gây `TypeError` trong môi trường kiểm chứng. Thông điệp lỗi cụ thể có thể khác theo phiên bản.

Parquet lưu kiểu dữ liệu đã đưa vào, không tự chuyển một cột chuỗi thành cột số. Việc chuyển định dạng tệp không thay thế bước làm sạch dữ liệu.

b) Khai báo dấu hỏi là thiếu ngay lúc đọc, rồi ghi lại:

```
df = pd.read_csv("gia.csv", na_values=["?"])
df.to_parquet("gia.parquet", index=False)
lai = pd.read_parquet("gia.parquet")
print(lai["gia"].mean())  # 75.0
```

Cột giá trở thành `[100.0, NaN, 50.0]`. Giá thiếu bị bỏ qua khi tính trung bình, nên:

$$\overline{g} = \frac{100 + 50}{2} = 75.$$

Không chia cho 3 vì giá của khu B chưa biết, không phải bằng 0.

Mở rộng / Cách nhớ. Nếu đã có bảng đọc sai, có thể chuyển cột giá sang số trước khi ghi Parquet:

```
df["gia"] = pd.to_numeric(df["gia"], errors="coerce")
```

Cần theo dõi những chuỗi bị chuyển thành thiếu để phát hiện lỗi nhập liệu. Thao tác đọc/ghi Parquet cần một bộ máy hỗ trợ như `pyarrow` hoặc `fastparquet`; đó là phụ thuộc kỹ thuật, khác với nguyên nhân sai kiểu của câu hỏi.

Mã 045 • Câu 22. (10 điểm)

Chuỗi JSON của một chỗ ở là:

```
raw = '["Wifi", "Washer, dryer"]'
```

a) (5 điểm) raw.split(",") cho bao nhiêu mảnh? Vì sao không dùng kết quả đó để đếm tiện nghi?

b) (5 điểm) Viết code đọc chuỗi thành danh sách Python và cho biết có bao nhiêu tiện nghi.

Đáp án: a) raw.split(",") cho 3 mảnh, không phải số tiện nghi. b) json.loads(raw) cho danh sách ["Wifi", "Washer, dryer"], gồm 2 tiện nghi.

a) `split` tách ở *mọi* dấu phẩy, kể cả dấu phẩy nằm bên trong tên tiện nghi thứ hai. Chuỗi có hai dấu phẩy nên cho ba mảnh:

```
['["Wifi"', ' "Washer', ' dryer"]']
```

Các mảnh còn chứa dấu ngoặc hoặc dấu nháy của cú pháp JSON. Kết quả đã chia đôi tên `Washer, dryer`, nên dùng độ dài của nó sẽ đếm sai thành ba tiện nghi.

b) Bộ giải mã JSON hiểu đâu là dấu phân cách giữa hai phần tử, đâu là ký tự bên trong một chuỗi:

```
import json

tien_nghi = json.loads(raw)
print(tien_nghi)       # ['Wifi', 'Washer, dryer']
print(len(tien_nghi))  # 2
```

Hai phần tử thật là `Wifi` và toàn bộ tên `Washer, dryer`. Dấu phẩy bên trong cặp dấu nháy của tên thứ hai được giữ nguyên như một phần nội dung.

Mở rộng / Cách nhớ. `json.loads` đọc JSON từ một chuỗi; `json.load` đọc từ đối tượng tệp. Với nhiều hàng, có thể áp dụng `json.loads` cho từng chuỗi JSON hợp lệ rồi `explode` nếu cần một hàng cho mỗi tiện nghi. Sau khi tách, để đếm chỗ ở có một tiện nghi, nhớ đếm mã chỗ ở phân biệt thay vì chỉ đếm hàng.

Lưu ý

Nếu dữ liệu là JSON, dùng bộ phân tích JSON. Tách chuỗi theo ký tự chỉ phù hợp khi nguồn quy định rõ một định dạng phẳng không có ký tự phân cách xuất hiện trong nội dung.

Mã 045 • Câu 23. (10 điểm)

Ba kỳ cuối tháng có 40, 50 và 80 đánh giá.

```
s = pd.Series(
    [40, 50, 80],
    index=pd.to_datetime(["2026-01-31", "2026-02-28", "2026-03-31"]))
```

a) (5 điểm) Tỷ lệ thay đổi của kỳ 31/03 so với kỳ liền trước là bao nhiêu? Kỳ liền trước là ngày nào?

b) (5 điểm) s.pct_change(periods=2) ở kỳ 31/03 lấy mốc là kỳ nào, tỷ lệ bao nhiêu? Viết biểu thức lấy đúng tỷ lệ so với kỳ liền trước.

Đáp án: a) 31/03 so với kỳ trước là 28/02: tăng 60%. b) periods=2 ở 31/03 lấy mốc 31/01: tăng 100%.

a) Tỷ lệ thay đổi so với kỳ liền trước:

$$r_t = \frac{s_t - s_{t-1}}{s_{t-1}} = \frac{s_t}{s_{t-1}} - 1.$$

Ở 31/03, giá trị hiện tại là 80 và giá trị kỳ trước 28/02 là 50:

$$\frac{80 - 50}{50} = \frac{30}{50} = 0.6 = 60\%.$$

Đây là tỷ lệ tăng, không phải mức hiện tại bằng 160% của trước đó, dù hai cách nói có liên quan.

b) `s.pct_change(periods=2)` lùi hai *hàng quan sát*. Tại hàng thứ ba 31/03, hàng lùi hai vị trí là 31/01, giá trị 40. Do đó:

$$\frac{80 - 40}{40} = 1 = 100\%.$$

Biểu thức lấy đúng tỷ lệ so với kỳ liền trước là:

```
ty_le = s.pct_change(periods=1, fill_method=None).loc["2026-03-31"]
print(ty_le)        # xấp xỉ 0.6
print(f"{ty_le:.1%}")  # 60.0%
```

Ngày	Giá trị	So với 1 kỳ trước	So với 2 kỳ trước
31/01	40	NaN	NaN
28/02	50	25%	NaN
31/03	80	60%	100%

Mở rộng / Cách nhớ. `pct_change` trả tỷ lệ dạng thập phân, không tự nhân 100. Vì thế 0.6 mới là đầu vào phù hợp cho định dạng `:.1%`. `periods` mặc định đếm hàng; nếu thiếu một tháng, phải chèn đủ lịch trước khi muốn diễn giải là thay đổi đúng một tháng. Nếu giá trị kỳ gốc bằng 0, phép chia cần được xử lý riêng vì tỷ lệ thông thường không xác định.

Mã 045 • Câu 24. (10 điểm)

Ba chỗ ở. Tổng tiền của n đêm bằng giá một đêm nhân n, cộng phí dọn một lần.

```
cho = pd.DataFrame({
    "ma": ["a", "b", "c"],
    "gia_dem": [90, 70, 50],
    "phi": [20, 40, 180],
})
```

a) (5 điểm) Với 7 đêm, tổng tiền của a, b và c là bao nhiêu? Khi đó c đã rẻ hơn cả a và b chưa?

b) (5 điểm) Viết code tìm số đêm nguyên nhỏ nhất để tổng tiền của c nhỏ hơn nghiêm ngặt tổng tiền của a và của b. Số đó là bao nhiêu?

Đáp án: a) a: 650; b: 530; c: 530. c chưa rẻ hơn cả hai vì vẫn bằng b. b) Số đêm nguyên nhỏ nhất là 8.

a) Phí dọn chỉ cộng một lần cho mỗi lượt, không nhân với số đêm:

$$T_a(n) = 90n + 20, \qquad T_b(n) = 70n + 40, \qquad T_c(n) = 50n + 180.$$

Thay $n = 7$ được $T_a = 630 + 20 = 650$, $T_b = 490 + 40 = 530$, $T_c = 350 + 180 = 530$. c rẻ hơn a nhưng *bằng* b, nên chưa thỏa điều kiện rẻ hơn nghiêm ngặt cả a và b.

b) Giải đồng thời hai bất đẳng thức:

$$50n + 180 < 90n + 20 \iff n > 4, \qquad 50n + 180 < 70n + 40 \iff n > 7.$$

Điều kiện gộp là $n > 7$. Với n là số đêm nguyên dương, nhỏ nhất là 8. Khi ấy a = 740, b = 600, c = 580.

```
bang = cho.set_index("ma")
n = 1
while True:
    tong = bang["gia_dem"] * n + bang["phi"]
    if (tong.loc["c"] < tong.loc[["a", "b"]]).all():
        break
    n += 1
print(n)  # 8
```

Vòng lặp xét số đêm theo thứ tự tăng nên lần đầu thỏa là số nhỏ nhất. Ở dữ liệu này c có giá mỗi đêm thấp hơn cả hai phương án còn lại nên chắc chắn có lúc phí dọn cao hơn được bù lại.

Mở rộng / Cách nhớ. Phải phân biệt dấu $<$ với $\leq$: nếu cho phép bằng cả a và b thì n = 7 đã đủ. Với bài toán tổng quát, kiểm tra sự tồn tại của lời giải trước khi dùng vòng lặp không giới hạn; không phải mọi bảng giá đều có một phương án trở nên rẻ nhất khi ở lâu.

Mã 045 • Câu 25. (10 điểm)

Bảng giá theo khu. Ô thiếu không đưa vào trung bình.

```
df = pd.DataFrame({
    "khu": ["A", "A", "A", "B"],
    "gia": [40, None, 80, 200],
})
tb = df.groupby("khu")["gia"].transform("mean")
print((df["gia"] - tb).iloc[0])
```

a) (5 điểm) Dòng in ra số nào? Trung bình của khu A được tính từ những giá nào?

b) (5 điểm) Một bạn đổi dòng tạo tb thành tb = df["gia"].mean() rồi in cùng hiệu ở hàng đầu. Số in ra là bao nhiêu? Viết lại dòng tạo tb để mỗi hàng được so với trung bình khu của chính nó.

Đáp án: a) -20.0; trung bình khu A là 60, tính từ 40 và 80. b) Khoảng -66.66666666666667 nếu dùng trung bình toàn bảng.

a) Giá None được biểu diễn là thiếu và bị bỏ qua khi tính mean. Khu A có hai giá đã biết 40, 80:

$$\overline{g}_A = \frac{40 + 80}{2} = 60.$$

`transform("mean")` đặt trung bình của khu về từng hàng ban đầu, nên tb là `[60.0, 60.0, 60.0, 200.0]`. Hàng đầu có giá 40; độ lệch là $40 - 60 = -20$. Hàng thiếu vẫn có trung bình khu 60 để đối chiếu, nhưng hiệu giá thiếu trừ 60 vẫn là NaN.

b) Nếu dùng `df["gia"].mean()`, tb là một số trung bình chung:

$$\overline{g} = \frac{40 + 80 + 200}{3} = \frac{320}{3}, \qquad 40 - \frac{320}{3} = -\frac{200}{3} \approx -66.66666666666667.$$

Kết quả này so hàng đầu với cả bảng, không phải với khu A. Khôi phục dòng tạo tb đúng:

```
tb = df.groupby("khu")["gia"].transform("mean")
```

Mở rộng / Cách nhớ. `agg`/`mean` sau groupby thường cho một kết quả mỗi nhóm, chỉ mục theo khu; `transform` cho kết quả có cùng số hàng và chỉ mục với đầu vào. Dùng transform khi cần trừ, so sánh hoặc lọc từng hàng theo thống kê của nhóm mà hàng đó thuộc về.

Mã 045 • Câu 26. (10 điểm)

Ba mức giá đang là chuỗi. Mẫu r"[$,]" đã được viết sẵn trong cả hai lệnh.

```
gia = pd.Series(["$2,400.00", "$60.50", "hỏi chủ"])
giu = gia.str.replace(r"[$,]", "", regex=False)
bot = gia.str.replace(r"[$,]", "", regex=True)
so = pd.to_numeric(bot, errors="coerce")
print(giu.tolist())
print(float(so.sum()))
```

a) (5 điểm) List của giu là gì? Vì sao dấu $ và dấu phẩy vẫn còn?

b) (5 điểm) Ba giá trị trong so là gì? Tổng in ra là bao nhiêu? Ô "hỏi chủ" có được tính là 0 không?

Đáp án: a) giu = ['2, 400.00′,′60.50', 'hỏi chủ']. b) so = [2400.0, 60.5, NaN]; tổng in ra 2460.5. Ô hỏi chủ vẫn thiếu, không được chuyển thành 0.

a) Tiền tố r chỉ điều khiển cách Python đọc chuỗi mẫu. Chính `regex=False` làm bộ thay thế tìm nguyên văn cả cụm `[$,]`, gồm hai ngoặc vuông, dấu đô la và dấu phẩy. Cụm nguyên văn đó không xuất hiện trong ba giá đầu vào, nên giu giữ nguyên toàn bộ.

b) Với `regex=True`, mẫu `[$,]` là một lớp ký tự: khớp một dấu đô la *hoặc* một dấu phẩy. bot trở thành `['2400.00', '60.50', 'hỏi chủ']`. `to_numeric(errors="coerce")` đọc được hai số đầu; chuỗi hỏi chủ không đọc được nên thành NaN.

$$\text{Tổng các giá đã đọc được} = 2400 + 60.5 = 2460.5.$$

Sum mặc định bỏ qua giá trị thiếu. Việc ô hỏi chủ không làm thay đổi tổng không có nghĩa nó đã trở thành 0: giá trị trong Series vẫn là NaN, biểu thị chưa biết giá.

Mở rộng / Cách nhớ. Nếu cần phân biệt ``toàn bộ đều không đọc được" với ``tổng giá thật bằng 0", dùng `so.sum(min_count=1)`. Việc giữ NaN cũng bảo vệ mẫu số khi tính mean: bỏ qua ô không biết khác với cộng thêm một giá bằng 0.

Mã đề 046

Đáp án nhanh phần trắc nghiệm

Câu	1	2	3	4	5	6	7	8	9	10
Đáp án	B	D	D	C	C	C	A	B*	D	B
Câu	11	12	13	14	15	16	17	18	19	20
Đáp án	D	A	A	B	D	A	C	C	C	A

Phần này chép nguyên văn PDF mới nhất. Dấu * ở câu 8: B với pandas 3.x/Copy-on-Write; hành vi cũ khi tắt chế độ này có thể cho D. Bảng đối chiếu ảnh cũ ở phụ lục.

Đi đến từng câu Câu 1 • Câu 2 • Câu 3 • Câu 4 • Câu 5 • Câu 6 • Câu 7 • Câu 8 • Câu 9 • Câu 10 • Câu 11 • Câu 12 • Câu 13

Câu 14 • Câu 15 • Câu 16 • Câu 17 • Câu 18 • Câu 19 • Câu 20 • Câu 21 • Câu 22 • Câu 23 • Câu 24 • Câu 25 • Câu 26

Đã có đủ câu 1--26. Phần đề nằm trước hộp đáp án; các bản sửa code chỉ nằm trong lời giải.

Phần I: Trắc nghiệm

Mã 046 • Câu 1. (2 điểm)

18.534 chỗ ở gom thành 9.712 host, mỗi host một trung vị giá. Câu nào đúng khi đưa số này vào báo cáo?

A. size theo host_id đếm số host trong thành phố.
B. Phải nói rõ số này là trung vị theo host, không phải theo chỗ ở.
C. Bảng host vẫn có 18.534 hàng.
D. Trung vị đó chính là trung vị của 18.534 chỗ ở.

Đáp án: B. Phải nói rõ số này là trung vị theo host, không phải theo chỗ ở.

Giải thích. Trước khi gom, đơn vị quan sát là một chỗ ở và có 18.534 hàng. Sau khi nhóm theo `host_id` và tính trung vị giá, mỗi host chỉ còn một giá trị đại diện; kết quả có 9.712 hàng nếu đúng 9.712 host được giữ trong phép nhóm. Vì thế cần mô tả rõ cách tổng hợp và đơn vị quan sát khi báo cáo.

`size` của từng nhóm host đếm số hàng chỗ ở thuộc host đó, không phải tổng số host trong thành phố. Số host là số nhóm. Trung vị giá của từng host cũng không thể được gọi ngay là trung vị của toàn bộ 18.534 chỗ ở.

Mở rộng / Cách nhớ. Trung vị của các trung vị nhìn chung khác trung vị toàn bộ dữ liệu. Ví dụ host H1 có giá 10, 10, 10 và host H2 có giá 100: hai trung vị theo host là 10 và 100, trung vị của chúng là 55, trong khi trung vị bốn chỗ ở là 10. Việc gom nhóm đã thay đổi trọng số: mỗi host góp một giá trị thay vì mỗi chỗ ở góp một giá trị.

Mã 046 • Câu 2. (2 điểm)

Với mẫu r"P\d+" , "P7x" cho str.match và str.fullmatch thế nào?

A. False và False
B. True và True
C. False và True
D. True và False

Đáp án: D. True và False.

Giải thích. Mẫu `P\d+` yêu cầu chữ P rồi ít nhất một chữ số. `str.match` kiểm tra một lần khớp bắt đầu từ đầu chuỗi; nó tìm được đoạn `P7` trong `P7x`, nên cho `True`. Nó không bắt buộc phần còn lại của chuỗi phải hết.

`str.fullmatch` yêu cầu toàn bộ chuỗi khớp mẫu. Chữ x cuối không thuộc phần chữ số, nên chuỗi còn thừa ký tự và cho `False`.

Mở rộng / Cách nhớ. Phân biệt ba thao tác: `contains` tìm ở bất kỳ vị trí nào; `match` tìm từ đầu; `fullmatch` khớp toàn chuỗi. Với chuỗi `xP7`, `contains` có thể cho `True`, còn hai thao tác sau đều cho `False`.

Mã 046 • Câu 3. (2 điểm)

duckdb.sql("SELECT ...").df() trả về đối tượng nào?

A. Một file CSV tạm, chưa có cột.
B. Một chuỗi JSON nguyên văn của câu SQL.
C. Số hàng duy nhất, không còn bảng.
D. Một DataFrame của pandas.

Đáp án: D. Một DataFrame của pandas.

Giải thích. `duckdb.sql(...)` thực hiện truy vấn và tạo đối tượng quan hệ của DuckDB. Lời gọi `.df()` lấy kết quả quan hệ đó về dạng `pandas.DataFrame`, có hàng và cột tương ứng với kết quả SQL.

Đây không phải thao tác xuất CSV, không trả một chuỗi JSON mô tả SQL và cũng không chỉ trả số hàng. Ngay cả truy vấn `SELECT COUNT(*) ...` có một số đếm thì gọi `.df()` vẫn cho bảng một ô, không phải một số vô hướng.

```
ket_qua = duckdb.sql("SELECT 1 AS x").df()
print(type(ket_qua).__name__)  # DataFrame
```

Mở rộng / Cách nhớ. Chọn cách lấy kết quả theo mục tiêu: `.df()` để tiếp tục xử lý bằng pandas; `.fetchall()` lấy danh sách các tuple; `.fetchone()` lấy một hàng. Kết quả đưa vào DataFrame sẽ cần bộ nhớ để lưu bảng đã lấy về.

Mã 046 • Câu 4. (2 điểm)

```
A = np.array([[10, 12, 11],
              [20, 21, 24],
              [30, 33, 31],
              [40, 44, 42]], dtype=np.int64)
R = A.reshape(2, 6)
print(R.itemsize, R.nbytes)
```

Hai số in ra là gì?

A. 4 96
B. 8 48
C. 8 96
D. 16 96

Đáp án: C. 8 96.

Giải thích. `np.int64` dùng 64 bit cho mỗi phần tử. Vì 8 bit bằng 1 byte, mỗi phần tử chiếm $64/8 = 8$ byte, nên `R.itemsize = 8`.

Mảng A có $4 \times 3 = 12$ phần tử. Sau khi đổi hình dạng thành 2×6, số phần tử và dtype đều giữ nguyên. Do đó:

$$\texttt{R.nbytes} = \texttt{R.size} \times \texttt{R.itemsize} = 12 \times 8 = 96 \text{ byte}.$$

Mở rộng / Cách nhớ. `reshape` không làm mỗi số nguyên nhỏ đi hay giảm số phần tử. Trong ví dụ mảng liên tục này, R có thể dùng chung vùng dữ liệu với A; `nbytes` mô tả dung lượng dữ liệu phần tử của R, không khẳng định có thêm 96 byte mới được cấp phát. Nó cũng không cộng phần bộ nhớ quản lý đối tượng Python.

Mã 046 • Câu 5. (2 điểm)

DataFrame đã đọc đúng: cột khu còn chuỗi "NA" , cột gia có NaN . Đưa bảng này vào DuckDB. Câu nào đúng?

A. "NA" thành NULL , còn NaN của giá được giữ để tính trung bình.
B. Cả "NA" lẫn NaN đều thành NULL .
C. NaN của giá thành NULL và bị AVG bỏ qua; "NA" vẫn là một mã khu.
D. DuckDB tính NaN như số 0 và nhóm "NA" biến mất.

Đáp án: C. NaN của giá thành NULL và bị AVG bỏ qua; "NA" vẫn là một mã khu.

Giải thích. Đề nói DataFrame *đã đọc đúng*: chuỗi `"NA"` được giữ là một mã khu hợp lệ, còn `NaN` trong cột số biểu diễn giá thiếu. Khi DuckDB quét bảng pandas này, giá thiếu được xử lý như SQL `NULL`; chuỗi `"NA"` vẫn là văn bản.

`AVG(gia)` bỏ qua những giá `NULL` và chia tổng cho số giá không thiếu. Nó không thay `NULL` bằng 0. Việc mã khu có hai chữ N, A không làm nhóm đó biến mất.

Mở rộng / Cách nhớ. Phân biệt hai tầng: `read_csv` có thể nhận `"NA"` là ký hiệu thiếu nếu đọc mặc định; nhưng câu này đã khẳng định nó vẫn là chuỗi trong DataFrame. Không áp lại quy tắc đọc CSV vào một bảng đã đọc đúng. Trong SQL, `COUNT(*)` đếm hàng còn `COUNT(gia)` đếm giá không NULL.

Mã 046 • Câu 6. (2 điểm)

```
df = pd.DataFrame({"gia": [100, 50, np.nan]})
```

dtype của cột gia là gì?

A. int64 , vì các giá gốc là số nguyên.
B. str , vì NaN là một chữ.
C. float64 , vì cột số có giá trị thiếu.
D. object , vì bảng vừa có chữ vừa có số.

Đáp án: C. float64, vì cột số có giá trị thiếu.

Giải thích. Ba phần tử đầu vào gồm hai số nguyên và `np.nan`. Với cách tạo bảng mặc định đang xét, `np.nan` là giá trị số thực thiếu; cột được suy luận thành `float64`, lưu các giá trị tương ứng `100.0`, `50.0`, `NaN`.

Tên `np.nan` trong code là một giá trị đặc biệt, không phải chuỗi chữ `"NaN"`. Vì vậy không có lý do để kết luận cột là `str` hay `object` từ đoạn mã này.

Mở rộng / Cách nhớ. Nếu muốn cột số nguyên có hỗ trợ thiếu của pandas, có thể chỉ định kiểu nullable:

```
s = pd.Series([100, 50, pd.NA], dtype="Int64")
```

`Int64` có chữ I hoa là kiểu mở rộng của pandas, khác `int64` NumPy thông thường. Đây là biến thể chủ động đổi dtype, không phải đầu ra của code đề.

Mã 046 • Câu 7. (2 điểm)

```
df = pd.DataFrame({
    "loai": ["Nguyên căn", "Phòng riêng", "Nguyên căn"],
    "gia": [100.0, 50.0, np.nan],
}, index=["a", "b", "c"])
mask = df["loai"] == "Nguyên căn"
print(int(mask.sum()))
```

Số in ra là gì?

A. 2
B. 3
C. 0
D. 1

Đáp án: A. 2.

Giải thích. Điều kiện chỉ kiểm tra cột `loai`. Hai hàng a và c có loại `Nguyên căn`; hàng b là `Phòng riêng`. Mặt nạ tương ứng là `[True, False, True]`.

Khi cộng Boolean, `True` đóng góp 1 còn `False` đóng góp 0. Vậy `mask.sum()` bằng $1 + 0 + 1 = 2$; `int` đưa kết quả về số nguyên để in. Giá của hàng c thiếu không ảnh hưởng vì điều kiện này không kiểm tra giá.

Mở rộng / Cách nhớ. Nếu mục tiêu là đếm nguyên căn *có giá*, phải viết thêm điều kiện:

```
mask = df["loai"].eq("Nguyên căn") & df["gia"].notna()
print(int(mask.sum()))  # 1
```

Đếm số hàng đúng điều kiện và đếm số giá không thiếu là hai câu hỏi khác nhau.

Mã 046 • Câu 8. (2 điểm)

```
df = pd.DataFrame({
    "loai": ["Nguyên căn", "Phòng riêng", "Nguyên căn"],
    "gia": [100.0, 50.0, np.nan],
}, index=["a", "b", "c"])
bang = df.copy()
cot = bang["gia"]
cot.loc["a"] = 80
```

Sau các dòng này, giá a trong bang và trong cot là gì?

A. bang là 80, cot vẫn 100.
B. bang vẫn 100, cot là 80.
C. Cả hai đều 100.
D. Cả hai đều 80.

Đáp án: B với pandas 3.x hoặc khi bật Copy-on-Write: bang vẫn 100, cot là 80. Với hành vi cũ khi tắt Copy-on-Write, cùng đoạn mã có thể cho D.

Giải thích. `bang = df.copy()` tạo bản sao của df. Sau đó `cot = bang["gia"]` lấy một Series từ bảng bang. Lệnh cuối sửa đối tượng `cot`, không trực tiếp gán vào `bang`.

Trong pandas 3.x, Copy-on-Write luôn được áp dụng: sửa Series đã lấy ra không truyền thay đổi ngược về DataFrame cha. Chạy trên pandas 3.0.1 cho:

Ô được xét	Giá trị sau lệnh
`df.loc["a", "gia"]`	100.0
`bang.loc["a", "gia"]`	100.0
`cot.loc["a"]`	80.0

Điểm phụ thuộc phiên bản: ở pandas 2.x với Copy-on-Write tắt, Series lấy từ cột có thể chia sẻ dữ liệu và phép sửa này có thể làm giá trong bang cũng thành 80; với đoạn mã và hành vi cũ đó chọn D. Đề không ghi phiên bản hay tùy chọn này, nên cần nêu giả định khi chốt đáp án. Bảng tra cứu của tài liệu dùng pandas 3.0.1.

Mở rộng / Cách nhớ. Muốn sửa bang một cách tường minh, dùng trực tiếp:

```
bang.loc["a", "gia"] = 80
```

`df.copy()` chỉ tách bang khỏi df; tự nó không giải quyết câu hỏi cot có truyền thay đổi về bang trong mọi phiên bản.

Mã 046 • Câu 9. (2 điểm)

extract có re.I và \b hai đầu in ra list nào?

```
import re
ten = pd.Series(["80 M2", "12m2x"])
print(ten.str.extract(r"\b(\d+)\s*m2\b", flags=re.I)[0].tolist())
```

A. [NaN, NaN]
B. ["80", "12"]
C. ["M2", "m2"]
D. ["80", NaN]

Đáp án: D. ["80", NaN].

Giải thích. Nhóm `(\d+)` bắt phần số; `\s*` cho phép có hoặc không có khoảng trắng; `re.I` cho phép m thường khớp M hoa. Vì vậy `80 M2` khớp và nhóm bắt trả chuỗi `"80"`.

Với `12m2x`, phần cuối mẫu yêu cầu ranh giới từ sau 2. Cả 2 và x đều là ký tự từ, nên giữa chúng không có ranh giới. Mẫu không khớp và cho giá trị thiếu. Kết quả nhóm bắt là số dưới dạng chuỗi, không phải đơn vị `M2` hay `m2`.

Mở rộng / Cách nhớ. Nếu bỏ `re.I`, tên đầu cũng không khớp. Nếu chỉ bỏ ranh giới cuối, chuỗi thứ hai sẽ khớp đoạn `12m2` và trả `"12"`. Mỗi cờ và mỗi ký hiệu đều có vai trò riêng.

Mã 046 • Câu 10. (2 điểm)

Tệp ma.csv chỉ có nội dung sau, không khai báo dtype :

```
id
007
```

```
raw = pd.read_csv("ma.csv")
print(raw.loc[0, "id"])
```

Ô đó được in ra dưới dạng nào?

A. Chuỗi "007" , còn đủ số 0.
B. Số 7, kiểu số nguyên.
C. Giá trị thiếu, vì 007 không hợp lệ.
D. Chuỗi "7" , mất hai số 0.

Đáp án: B. Số 7, kiểu số nguyên.

Giải thích. `read_csv` mặc định suy luận dtype từ dữ liệu. Cột chỉ có `007`, là một biểu diễn số nguyên hợp lệ, nên được đọc thành số nguyên 7. Các số 0 đầu là cách viết trong tệp, không được giữ khi biểu diễn dưới dạng số.

Đầu ra là `7`, không phải chuỗi `"7"`; cũng không phải thiếu. Trong môi trường kiểm chứng, dtype của cột là `int64`.

Mở rộng / Cách nhớ. Nếu id là mã định danh cần giữ độ dài, khai báo kiểu chuỗi ngay lúc đọc:

```
raw = pd.read_csv("ma.csv", dtype={"id": "string"})
print(raw.loc[0, "id"])  # 007
```

Đổi số 7 thành chuỗi sau khi đã đọc chỉ cho `"7"`. Chỉ thêm lại 0 bằng `zfill(3)` nếu biết chắc quy ước mã dài ba ký tự.

Mã 046 • Câu 11. (2 điểm)

```
df = pd.DataFrame({
    "loai": ["Nguyên căn", "Phòng riêng", "Nguyên căn"],
    "gia": [100.0, 50.0, np.nan],
}, index=["a", "b", "c"])
mot = df.loc[["a", "b"], "gia"]
hai = df.loc[["a", "b"], ["gia"]]
print(type(mot).__name__, type(hai).__name__)
```

Hai tên kiểu in ra là gì?

A. DataFrame DataFrame
B. Series Series
C. DataFrame Series
D. Series DataFrame

Đáp án: D. Series DataFrame.

Giải thích. Hai phép chọn cùng lấy các hàng a và b. Khác biệt nằm ở bộ chọn cột:

- `"gia"` là một tên cột đơn: chiều cột được rút gọn, nên `mot` là Series.
- `["gia"]` là một danh sách cột: giữ chiều bảng, nên `hai` là DataFrame một cột.

`type(...).__name__` lấy tên lớp, nên in hai tên cách nhau bằng khoảng trắng như đáp án D.

Mở rộng / Cách nhớ. Với bảng có tên cột duy nhất như đề, `df["gia"]` có `ndim = 1`, còn `df[["gia"]]` có `ndim = 2`. Hai đối tượng có thể chứa cùng các giá trị nhưng khác kiểu và các phương thức hoặc cách căn chỉnh có thể khác nhau.

Mã 046 • Câu 12. (2 điểm)

```
s = pd.Series([10, 30], index=pd.to_datetime(["2026-01-31", "2026-03-31"]))
print(s.asfreq("ME").shift(1).loc["2026-03-31"])
```

Ô in ra là gì?

A. NaN, vì hàng tháng 2 vừa được thêm đang thiếu.
B. Đúng bằng giá trị của tháng 3, vì shift không đổi hàng.
C. Số 0, vì tháng 2 không có hàng trong Series gốc.
D. Đúng bằng giá trị của tháng 1, tháng liền trước trên lịch.

Đáp án: A. NaN, vì hàng tháng 2 vừa được thêm đang thiếu.

Giải thích. `asfreq("ME")` tạo đủ các mốc cuối tháng từ tháng 1 đến tháng 3. Hàng 28/02 được thêm với giá trị thiếu, không tự điền 0. Sau đó `shift(1)` dịch giá trị một hàng và giữ nhãn chỉ mục:

Nhãn	Sau asfreq	Sau shift
31/01/2026	10	`NaN`
28/02/2026	`NaN`	10
31/03/2026	30	`NaN`

Ô 31/03 lấy giá trị của hàng 28/02 vừa được chèn, tức `NaN`. Lệnh print hiển thị `nan`.

Mở rộng / Cách nhớ. Nếu bỏ bước `asfreq`, bảng gốc chỉ có hai hàng; `shift(1)` tại 31/03 sẽ lấy 10 từ 31/01. Chèn đủ lịch trước làm rõ sự khác nhau giữa ``hàng trước" và ``tháng trước".

Mã 046 • Câu 13. (2 điểm)

Cột khu là Categorical với các mức A, B, C, nhưng bảng chỉ có hàng A và B. groupby(..., observed=True)["gia"] .mean() khác observed=False ở điểm nào?

A. observed=True chỉ còn A và B; observed=False thêm khu C là NaN.
B. Cả hai cách đều thêm khu C với trung bình 0.
C. Cả hai cách đều bỏ khu C vì không có hàng.
D. observed=True thêm khu C là NaN; observed=False bỏ khu C.

Đáp án: A. observed=True chỉ còn A và B; observed=False thêm khu C là NaN.

Giải thích. Categorical có danh sách mức cho phép A, B, C, nhưng chỉ A và B có quan sát. `observed=True` trả kết quả cho những mức thật sự xuất hiện. `observed=False` giữ cả mức C đã khai báo nhưng không có hàng.

Trung bình nhóm C không có giá trị nào để tính, nên là `NaN`, không phải 0. Trung bình của A và B được tính như thường trong cả hai cách; sự khác nhau nằm ở việc có trả nhóm chưa quan sát C hay không.

Mở rộng / Cách nhớ. `observed` khác `dropna`: `observed` liên quan mức Categorical chưa xuất hiện, còn `dropna` liên quan khóa nhóm bị thiếu. Dù mặc định của thư viện thay đổi giữa các phiên bản, truyền rõ `observed=True/False` giúp chương trình thể hiện đúng ý định.

Mã 046 • Câu 14. (2 điểm)

Quy trình có trách nhiệm gồm tự phác việc cần làm, hỏi AI, đọc code, kiểm trên mẫu, rồi khai báo. Việc nào phải xong trước khi giữ số vào báo cáo?

A. Khai báo AI_USAGE.md xong rồi mới cần đọc code.
B. Tự tính tay trên một mẫu nhỏ để đối chiếu với code.
C. Chạy lại thấy không có thông báo lỗi là đủ.
D. Hỏi lại AI xem quy ước ô trống có ổn hay không.

Đáp án: B. Tự tính tay trên một mẫu nhỏ để đối chiếu với code.

Giải thích. Trước khi dùng con số trong báo cáo, cần kiểm tra chương trình có trả lời đúng câu hỏi hay không. Một mẫu nhỏ với đầu ra tự tính được giúp phát hiện sai mẫu số, sai điều kiện lọc, mất nhãn hoặc hiểu nhầm giá thiếu.

Code chạy không báo lỗi chỉ chứng minh chưa gặp lỗi thực thi trong lần chạy đó, không bảo đảm cách tính đúng. Khai báo việc dùng AI là yêu cầu minh bạch riêng, không thay cho đọc hiểu và kiểm chứng. Hỏi lại AI cũng chưa tạo ra phép đối chiếu độc lập với dữ liệu.

Mở rộng / Cách nhớ. Một cách kiểm tra hữu ích: ghi dữ liệu mẫu, tự tính đáp án mong đợi, chạy code rồi so sánh. Thêm ít nhất một trường hợp có ô thiếu hoặc khóa không khớp nếu bài toán có những tình huống đó. Quy định `AI_USAGE.md` trong câu hỏi là nội dung đề, không phải yêu cầu tạo thêm tệp cho tài liệu này.

Mã 046 • Câu 15. (2 điểm)

```
goc = pd.Series(["18/08/2026", "31/06/2026", None])
doc = pd.to_datetime(goc, format="%d/%m/%Y", errors="coerce")
print(int((goc.notna() & doc.isna()).sum()))
```

Dòng in ra bao nhiêu?

A. 3
B. 0
C. 2
D. 1

Đáp án: D. 1.

Giải thích. Mặt nạ cần đếm đồng thời hai điều: dữ liệu nguồn không thiếu và sau chuyển ngày lại bị thiếu. Xét từng hàng:

Nguồn	Sau đọc	Nguồn có giá trị?	Được đếm?
`18/08/2026`	18/08/2026	Có	Không
`31/06/2026`	`NaT`	Có	Có
`None`	`NaT`	Không	Không

Tháng 6 chỉ có 30 ngày nên `31/06/2026` không hợp lệ; `errors="coerce"` đổi nó thành `NaT`. Hàng `None` đã thiếu từ đầu nên không được coi là lỗi đọc một giá trị có sẵn. Tổng chỉ có 1 hàng.

Mở rộng / Cách nhớ. Chỉ dùng `doc.isna().sum()` sẽ đếm 2, trộn dữ liệu thiếu ban đầu với dữ liệu không đọc được. Mặt nạ trong đề giúp thống kê riêng lỗi chuyển đổi và có thể dùng để lấy các chuỗi cần kiểm tra.

Mã 046 • Câu 16. (2 điểm)

Ghép bảng mỗi host một dòng trở lại bảng chỗ ở bằng how="left" . Việc validate="many_to_one" thêm được gì so với chỉ tin left join?

A. Dừng nếu host_id bên bảng host bị lặp, trước khi hàng bị nhân im lặng.
B. Tự cộng số chỗ ở của các dòng host trùng thành một dòng.
C. Thay cho việc đếm lại số hàng của bảng sau khi ghép.
D. Bảo đảm mọi chỗ ở đều tìm thấy host, kể cả host thiếu.

Đáp án: A. Dừng nếu host_id bên bảng host bị lặp, trước khi hàng bị nhân im lặng.

Giải thích. `many_to_one` mô tả quan hệ nhiều hàng chỗ ở ở bảng trái ứng với tối đa một hàng của cùng host ở bảng phải. pandas kiểm tra khóa ở bảng phải phải duy nhất. Nếu một host xuất hiện nhiều lần ở bảng phải, lệnh ghép báo `MergeError` thay vì âm thầm nhân hàng bên trái.

Ví dụ một chỗ ở có host H1, nhưng bảng host có hai hàng H1; nếu không kiểm tra, hàng chỗ ở có thể sinh hai hàng kết quả. `how="left"` tự nó chỉ quy định giữ các hàng phía trái, không bảo đảm số hàng kết quả giữ nguyên khi khóa phải bị trùng.

Mở rộng / Cách nhớ. `validate` không tự gộp các hàng host trùng và không bảo đảm mọi khóa trái đều có đối tác. Để kiểm tra mức độ khớp, thêm `indicator=True` và xem các hàng `left_only`; vẫn nên đối chiếu số hàng trước và sau ghép.

Mã 046 • Câu 17. (2 điểm)

Hai nhóm mà extract lấy từ "4 bedrooms" là gì?

```
ten = pd.Series(["4 bedrooms"])
print(ten.str.extract(r"(\d+)\s*(BR|bedrooms?)").iloc[0].tolist())
```

A. ["4 bedrooms"]
B. [NaN, "bedrooms"]
C. ["4", "bedrooms"]
D. ["4", "bedroom"]

Đáp án: C. ["4", "bedrooms"].

Giải thích. Mẫu có hai nhóm bắt: `(\d+)` bắt số phòng và `(BRbedrooms?)|` bắt cách viết đơn vị. Trong `4 bedrooms`, nhóm đầu lấy `"4"`, phần `\s*` nhận dấu cách, nhóm sau nhận `"bedrooms"`.

Dấu hỏi chỉ làm chữ s cuối có thể có hoặc không. Với chuỗi đang có chữ s, lần khớp nhận cả nó; không tự cắt về dạng số ít `"bedroom"`. `extract` trả hai cột nhóm; `iloc[0]` lấy hàng đầu rồi `tolist()` tạo danh sách hai chuỗi.

Mở rộng / Cách nhớ. Nếu chỉ muốn lấy số và không cần trả đơn vị thành nhóm riêng, đổi nhóm đơn vị sang không bắt:

```
so = ten.str.extract(r"(\d+)\s*(?:BR|bedrooms?)", expand=False)
```

Sau đó chuyển sang số nếu cần tính. Việc nhóm có được trả ra hay không phụ thuộc dấu ngoặc thường so với `(?:...)`.

Mã 046 • Câu 18. (2 điểm)

Đánh giá cuối cùng trong tháng 8 là ngày 18/08/2026. Nhãn tháng 8 của resample("ME") là ngày nào?

A. 2026-09-01
B. 2026-08-18
C. 2026-08-31
D. 2026-08-01

Đáp án: C. 2026-08-31.

Giải thích. `ME` nghĩa là tần suất cuối tháng. Các quan sát của tháng 8 được gom vào kỳ có nhãn ngày cuối tháng, tức 31/08/2026. Nhãn này được xác định bởi lịch, không phải ngày có quan sát cuối cùng.

Do đó, dù dữ liệu mới nhất là 18/08, kỳ vẫn mang nhãn 31/08. Tên nhãn không chứng minh dữ liệu đã được thu thập tới hết tháng 8.

Mở rộng / Cách nhớ. Khi báo cáo tháng, cần kiểm tra cả nhãn kỳ và khoảng ngày nguồn thật sự phủ. Một kỳ kết thúc sau ngày quan sát cuối có thể là kỳ chưa đủ dữ liệu; câu 25 yêu cầu lọc rõ những kỳ có nhãn không vượt quá ngày cuối cùng.

Mã 046 • Câu 19. (2 điểm)

```
khu = pd.Series([" Đà Lạt ", "ĐÀ LẠT", "đà lạt", "Da Lat"])
print(khu.str.strip().str.lower().nunique())
```

Dòng cuối in ra gì?

A. 1
B. 3
C. 2
D. 4

Đáp án: C. 2.

Giải thích. `str.strip()` xóa khoảng trắng hai đầu từng tên; `str.lower()` chuyển chữ hoa thành chữ thường. Ba chuỗi đầu đều trở thành `đà lạt`. Chuỗi cuối trở thành `da lat`.

Hai dạng `đà lạt` và `da lat` vẫn khác nhau vì lower không bỏ dấu tiếng Việt và không đổi đ thành d. `nunique()` đếm hai giá trị phân biệt nên in 2.

Mở rộng / Cách nhớ. Chuẩn hóa hoa thường, khoảng trắng, Unicode và dấu là các thao tác khác nhau. Nếu muốn gộp tên có dấu và không dấu, phải xác định quy tắc chuẩn hóa hoặc dùng bảng ánh xạ được kiểm tra; không mặc định coi mọi tên bỏ dấu là cùng một địa điểm.

Mã 046 • Câu 20. (2 điểm)

```
raw = '["Pocket wifi"]'
ds = ["Pocket wifi"]
```

So sánh "wifi" in ds với pd.Series([raw]).str.contains("wifi", case=False) .

A. Phần tử thì False ; tìm trên chuỗi gốc, không phân biệt hoa thường, thì True .
B. Phần tử thì True ; tìm trên chuỗi JSON gốc thì lại False .
C. Cả hai đều True , vì cả hai cách đều thấy đoạn chữ wifi .
D. Cả hai đều False , vì không có đúng phần tử wifi .

Đáp án: A. Phép kiểm tra phần tử cho False; tìm đoạn chữ trên chuỗi gốc cho True.

Giải thích. Toán tử `in` trên danh sách kiểm tra có phần tử nào *bằng đúng* đối tượng đang tìm hay không. Danh sách chỉ có phần tử `"Pocket wifi"`; phần tử đó không bằng `"wifi"`, nên `"wifi" in ds` cho `False`.

`str.contains("wifi", case=False)` tìm đoạn chữ wifi bên trong từng chuỗi, không phân biệt hoa thường. Chuỗi JSON gốc chứa đoạn đó nên phần tử đầu của Series kết quả là `True`. Hai biểu thức cũng có kiểu đầu ra khác nhau: một Boolean đơn lẻ và một Series Boolean.

Mở rộng / Cách nhớ. Nếu mục tiêu là xem một tiện nghi có đúng tên `wifi`, hãy giải mã JSON rồi so khớp tên. Nếu mục tiêu là tên tiện nghi có chứa chữ wifi, có thể tìm đoạn chữ trên từng phần tử. Đừng dùng tìm kiếm trên chuỗi JSON để thay thế việc hiểu cấu trúc danh sách khi cần thống kê tiện nghi chính xác.

Phần II: Tự luận

Mã 046 • Câu 21. (10 điểm)

Tệp gia_theo_khu.csv . NA là mã một khu, ô trống là chưa có giá, 0 là giá đã ghi.

```
khu,gia
NA,200
TB,50
NA,
TB,20
NA,40
NA,0
```

```
df = pd.read_csv(
    "gia_theo_khu.csv",
    dtype={"gia": "float64"},
    keep_default_na=False,
    na_values={"gia": [""]},
)
print(df.loc[df["khu"] == "NA", "gia"].mean())
```

a) (5 điểm) Dòng in ra bao nhiêu? Ô giá trống và giá 0 của khu NA được xử lý thế nào?

b) (5 điểm) Bỏ keep_default_na=False thì cùng dòng in ra gì? Nếu vẫn giữ cờ đó nhưng coi cả chuỗi "0" là thiếu, trung bình khu NA thành bao nhiêu?

Đáp án: a) 80.0. Ô trống là NaN và bị bỏ qua; giá 0 vẫn được tính. b) Bỏ keep_default_na=False: NaN. Giữ cờ nhưng coi cả "0" là thiếu: 120.0.

a) `keep_default_na=False` tắt bộ ký hiệu thiếu mặc định, nhờ đó mã khu `"NA"` vẫn là chuỗi. Tham số `na_values` trong đề chỉ khai báo ô rỗng ở cột giá là thiếu. Dtype cột giá được khóa là `float64`.

Bốn hàng khu NA có các giá `[200.0, NaN, 40.0, 0.0]`. `mean` bỏ qua giá thiếu nhưng giữ số 0 hợp lệ:

$$\overline{g}_{NA} = \frac{200 + 40 + 0}{3} = 80.$$

Ô trống không được tính vào mẫu số, còn giá 0 có đóng góp vào mẫu số. Đây là lý do không thể coi cả hai là cùng một loại dữ liệu.

b) Nếu bỏ `keep_default_na=False`, giá trị mặc định là `True`; chuỗi `"NA"` ở cột khu sẽ bị nhận là thiếu. Điều kiện `df["khu"] == "NA"` không còn chọn được các hàng đó. Trung bình của tập rỗng là `NaN`, lệnh print hiển thị `nan`, không phải 0.

Nếu giữ cờ nhưng khai báo thêm `"0"` là thiếu ở cột giá:

```
df = pd.read_csv(
    "gia_theo_khu.csv",
    dtype={"gia": "float64"},
    keep_default_na=False,
    na_values={"gia": ["", "0"]},
)
```

Các giá được tính ở khu NA chỉ còn 200 và 40, nên trung bình trở thành $(200 + 40)/2 = 120$. Đó là kết quả của quy ước thay đổi; theo ý nghĩa nguồn đã nêu, coi 0 là thiếu là sai và làm trung bình tăng từ 80 lên 120.

Mở rộng / Cách nhớ. Nên khai báo ký hiệu thiếu theo từng cột. Một chuỗi có thể là mã hợp lệ trong cột này nhưng là ký hiệu thiếu trong cột khác. Sau đọc, kiểm tra số hàng, số giá thiếu và sự tồn tại của các mã quan trọng trước khi tổng hợp.

Mã 046 • Câu 22. (10 điểm)

Hai lần ghi giá của cùng ba mã. Lần 2 chưa có giá của c.

```
lan1 = pd.Series({"a": 100, "b": 200, "c": 300})
lan2 = pd.Series({"a": 130, "b": 210, "c": np.nan})
```

a) (5 điểm) lan2.mean() - lan1.mean() bằng bao nhiêu? Vì sao hiệu này chưa phải mức thay đổi của cùng một tập chỗ ở?

b) (5 điểm) Viết code tính hiệu trung bình lần 2 trừ trung bình lần 1 chỉ trên những mã có giá ở cả hai lần. Kết quả là bao nhiêu?

Đáp án: a) -30.0, do hai trung bình dùng hai tập mã khác nhau. b) Trên các mã có giá ở cả hai lần, hiệu trung bình là +20.0.

a) Lần 1 có đủ a, b, c nên:

$$\overline{g}_1 = \frac{100 + 200 + 300}{3} = 200.$$

Lần 2 chỉ có a và b có giá, vì mean bỏ qua NaN:

$$\overline{g}_2 = \frac{130 + 210}{2} = 170, \qquad \overline{g}_2 - \overline{g}_1 = 170 - 200 = -30.$$

Hiệu âm này không mô tả mức giảm của cùng một tập: lần 1 có mã c giá cao 300, lần 2 mã c không được tính. Thay đổi thành phần quan sát có thể làm trung bình giảm dù cả a và b đều tăng giá.

b) Căn theo mã rồi chỉ giữ các mã có dữ liệu ở cả hai lần. Một cách rõ ràng:

```
cap = pd.concat([lan1.rename("lan1"), lan2.rename("lan2")], axis=1)
cap = cap.dropna(subset=["lan1", "lan2"])
hieu_tb = cap["lan2"].mean() - cap["lan1"].mean()
print(hieu_tb)  # 20.0
```

Hai mã được giữ là a và b. Trung bình lần 1 là $(100 + 200)/2 = 150$, lần 2 là 170; hiệu $170 - 150 = 20$.

Cũng có thể tính trung bình của các hiệu ghép đúng mã:

```
print((cap["lan2"] - cap["lan1"]).mean())  # 20.0
```

Mở rộng / Cách nhớ. Trên cùng một tập cặp quan sát và cùng trọng số, hiệu hai trung bình bằng trung bình các hiệu: $\overline{y} - \overline{x} = \overline{y - x}$. Ở đây hai mức tăng là 30 và 10, trung bình 20. Không điền giá thiếu của c bằng 0 nếu nguồn chưa xác nhận giá thực sự bằng 0.

Mã 046 • Câu 23. (10 điểm)

Hai lượt ở. Số đêm ở đây là số ngày trọn: phần giờ lẻ không làm tròn thành thêm một ngày.

```
luot = pd.DataFrame({
    "vao": pd.to_datetime(["2026-03-01 20:00", "2026-03-10 08:00"]),
    "ra": pd.to_datetime(["2026-03-03 02:00", "2026-03-10 20:00"]),
})
hieu = luot["ra"] - luot["vao"]
print((luot.loc[0, "ra"] - luot.loc[0, "vao"]).days)
print(hieu.dt.days.tolist())
```

a) (5 điểm) Số ngày trọn của lượt đầu là bao nhiêu? Lượt đó dài 1 ngày 6 giờ. Vì sao không phải 2?

b) (5 điểm) hieu.days chạy thế nào? List số ngày trọn của cả hai lượt là gì, và tổng là bao nhiêu?

Đáp án: a) 1 ngày trọn. b) hieu.days báo AttributeError; phải dùng hieu.dt.days. Danh sách là [1, 0], tổng bằng 1.

a) Lượt đầu từ 01/03 lúc 20:00 đến 03/03 lúc 02:00 dài 30 giờ, tức 1 ngày 6 giờ. Một ngày trọn là 24 giờ; 6 giờ dư chưa đủ một ngày mới. Theo quy ước đề đã nêu:

$$\left\lfloor \frac{30}{24} \right\rfloor = 1.$$

Hai mốc thuộc ba ngày lịch có tên khác nhau không làm số ngày trọn thành 2. `Timedelta.days` lấy phần ngày của khoảng thời gian, không làm tròn lên theo lịch hay số ngày có chạm tới.

b) `hieu` là Series chứa các Timedelta. Nó không có thuộc tính `days` dùng trực tiếp như một Timedelta đơn lẻ; `hieu.days` báo `AttributeError`. Phải dùng bộ truy cập `dt` để lấy phần ngày của từng phần tử:

```
ngay_tron = hieu.dt.days
print(ngay_tron.tolist())  # [1, 0]
print(ngay_tron.sum())     # 1
```

Lượt hai từ 08:00 đến 20:00 cùng ngày chỉ dài 12 giờ nên có 0 ngày trọn. Tổng số ngày trọn của hai lượt là $1 + 0 = 1$.

Mở rộng / Cách nhớ. `hieu.dt.total_seconds() / 86400` cho số ngày dạng thực, ở đây là `[1.25, 0.5]`. Không cộng các giờ dư của các lượt độc lập rồi làm tròn tổng nếu yêu cầu đếm ngày trọn *từng lượt*. Trong dữ liệu thực tế, cần kiểm tra ngày ra không trước ngày vào; đối với khoảng âm, cách lấy phần ngày có thể cho giá trị âm khác với cắt về 0.

Mã 046 • Câu 24. (10 điểm)

Hai tên cần lấy số mét vuông đứng thành một từ riêng.

```
ten = pd.Series(["18m2", "18m2x"])
print(ten.str.extract(r"\b(\d+)m2\b")[0].tolist())
```

a) (5 điểm) Dòng in ra list nào? Lệnh có dừng vì lỗi không? Vì sao 18m2x không cho số 18?

b) (5 điểm) Viết lại mẫu bằng chuỗi thô để ranh giới từ còn hiệu lực. List mới là gì?

Đáp án: a) ['18', nan]; lệnh chạy bình thường. 18m2x không khớp vì không có ranh giới từ giữa 2 và x. b) Mẫu trong PDF mới đã là chuỗi thô; kết quả không đổi.

a) Bản PDF 046 mới dùng `r"\b(\d+)m2\b"`, có sẵn tiền tố r. Regex nhận các ranh giới từ thật. Tên `18m2` có ranh giới ở đầu và cuối nên nhóm số lấy được chuỗi `'18'`. Với `18m2x`, chữ số 2 và chữ x đều là ký tự từ; giữa chúng không có `\b`, nên mẫu không khớp và cho NaN.

b) Có thể viết rõ mẫu ra biến rồi trích; vẫn là cùng một mẫu chuỗi thô:

```
pat = r"\b(\d+)m2\b"
print(ten.str.extract(pat)[0].tolist())  # ['18', nan]
```

Ý b của PDF mới yêu cầu viết lại bằng chuỗi thô dù code đã có r; không cần bỏ r rồi thêm lại để tạo lỗi giả định. **Đối chiếu ảnh cũ.** Ảnh mã 046 trước đây dùng `"\b(\d+)m2\b"` không có r và hỏi vì sao `18m2` không cho 18. Với chuỗi thường ấy, Python hiểu `\b` là backspace U+0008, nên kết quả cũ là `[nan, nan]`. Đây là hai phiên bản code khác nhau, không phải hai lời giải mâu thuẫn cho cùng một code.

Mở rộng / Cách nhớ. Chuỗi thường tương đương đúng phải nhân đôi gạch chéo: `"\\b(\\d+)m2\\b"`. Trong bản tổng hợp này phần chính dùng nguyên văn PDF 046 mới; phụ lục lưu cả câu 24 theo ảnh cũ để giữ đầy đủ nội dung đã nhận.

Mã 046 • Câu 25. (10 điểm)

Mỗi ngày từ 01/07/2026 đến 18/08/2026 có đúng một đánh giá. resample("ME") gắn nhãn 31/07 cho tháng 7 và nhãn 31/08 cho tháng 8.

```
ngay = pd.date_range("2026-07-01", "2026-08-18", freq="D")
s = pd.Series(1, index=ngay)
theo = s.resample("ME").sum()
```

a) (5 điểm) Hai tổng trong theo là bao nhiêu? Vì sao không kết luận tháng 8 ít đánh giá hơn tháng 7 khi so cùng độ dài một tháng?

b) (5 điểm) Viết biểu thức chỉ giữ những kỳ có nhãn không vượt quá ngày đánh giá cuối cùng. Còn lại kỳ nào và tổng bao nhiêu?

Đáp án: a) Kỳ 31/07 có 31, kỳ 31/08 có 18. Tháng 8 mới được quan sát 18 ngày. b) Chỉ giữ kỳ 31/07/2026, tổng bằng 31.

a) `date_range` với hai mốc và tần suất ngày tạo đủ các ngày từ 01/07 đến hết 18/08. Mỗi ngày có giá trị 1, nên tổng tháng bằng số ngày có dữ liệu:

Nhãn kỳ	Phạm vi quan sát thật	Tổng
31/07/2026	01/07--31/07: 31 ngày	31
31/08/2026	01/08--18/08: 18 ngày	18

Không thể từ $18 < 31$ kết luận lượng đánh giá của *một tháng đầy đủ* tháng 8 thấp hơn tháng 7. Hai tổng có độ dài quan sát khác nhau; nhãn 31/08 không làm xuất hiện dữ liệu từ 19/08 đến 31/08. Với số liệu đã cho, tốc độ trung bình trong những ngày có dữ liệu của cả hai kỳ đều là 1 đánh giá/ngày.

b) Lấy ngày quan sát cuối rồi so với nhãn kỳ:

```
du_ky = theo.loc[theo.index <= s.index.max()]
print(du_ky)
# 2026-07-31    31
```

Ngày cuối là 18/08/2026. Nhãn 31/07 không vượt mốc đó nên được giữ; 31/08 vượt nên bị loại. Tổng của phần giữ lại là 31.

Mở rộng / Cách nhớ. Quy tắc này phù hợp ở đây vì nguồn bắt đầu đúng 01/07 và mỗi ngày trong khoảng đều có một đánh giá. Với nguồn bắt đầu giữa tháng, chỉ kiểm tra nhãn không vượt ngày cuối chưa đủ chứng minh kỳ đầu đầy đủ; cần kiểm tra cả mốc bắt đầu và độ phủ quan sát.

Mã 046 • Câu 26. (10 điểm)

Bảng có khu và giá.

```
df = pd.DataFrame({"khu": ["A", "A", "B"], "gia": [10, 30, 40]})
tb = df.groupby("khu")["gia"].mean()
tb.to_csv("tb.csv", index=False)
```

a) (5 điểm) Tệp tb.csv còn những gì? Vì sao không còn tên khu?

b) (5 điểm) Viết dòng ghi tệp để còn cột khu và cột trung bình, nhưng không ghi thêm một cột chỉ mục số.

Đáp án: a) CSV chỉ còn cột gia và hai trung bình 20.0, 40.0; tên khu nằm ở index đã bị bỏ. b) Chuyển index thành cột rồi ghi với index=False.

a) `groupby("khu")["gia"].mean()` trả một Series: tên Series là `gia`, còn A và B nằm ở chỉ mục có tên `khu`. Giá trung bình là:

$$\overline{g}_A = \frac{10+30}{2} = 20, \qquad \overline{g}_B = 40.$$

`to_csv(index=False)` bỏ chỉ mục khi ghi. Vì mã khu nằm trong chỉ mục, chúng cũng bị bỏ. Nội dung tệp còn:

```
gia
20.0
40.0
```

Giá trị vẫn còn nhưng tệp không còn nói trung bình nào thuộc khu nào.

b) Chuyển chỉ mục khu thành một cột thông thường trước khi ghi:

```
tb.reset_index().to_csv("tb.csv", index=False)
```

Tệp mới là:

```
khu,gia
A,20.0
B,40.0
```

`reset_index()` đưa mã khu về cột `khu`; `index=False` lúc này chỉ ngăn ghi thêm chỉ mục số mới 0, 1. Nếu muốn tên cột thể hiện rõ trung bình:

```
tb.rename("gia_trung_binh").reset_index().to_csv("tb.csv", index=False)
```

Mở rộng / Cách nhớ. Có thể tránh đưa khóa vào chỉ mục ngay từ đầu:

```
tb = df.groupby("khu", as_index=False)["gia"].mean()
```

Trước khi xuất bảng, luôn kiểm tra các khóa định danh đang là cột hay chỉ mục; bỏ index chỉ an toàn nếu index không chứa thông tin cần giữ.

Mã đề 030

Đáp án nhanh phần trắc nghiệm

Câu	1	2	3	4	5	6	7	8	9	10
Đáp án	D*	B	B	X	X	C	X	X	X	X
Câu	11	12	13	14	15	16	17	18	19	20
Đáp án	X	X*	A/C	C	D	B	C/D	X	A*	D

X: code gốc lỗi, thiếu dữ kiện hoặc không có lựa chọn đúng; xem từng câu. Câu 1 D gần ý định nhưng lỗi chữ. Câu 12 không in, giá trị gán là 3 (C nếu thêm print). Câu 19 A do AND hai số đếm, không phải giao mặt nạ theo hàng. A/C và C/D biểu thị hai lựa chọn cùng đúng.

Đi đến từng câu Câu 1 • Câu 2 • Câu 3 • Câu 4 • Câu 5 • Câu 6 • Câu 7 • Câu 8 • Câu 9 • Câu 10 • Câu 11 • Câu 12 • Câu 13

Câu 14 • Câu 15 • Câu 16 • Câu 17 • Câu 18 • Câu 19 • Câu 20 • Câu 21 • Câu 22 • Câu 23 • Câu 24 • Câu 25 • Câu 26

Đã có đủ câu 1--26. Phần đề nằm trước hộp đáp án; các bản sửa code chỉ nằm trong lời giải.

Phần I: Trắc nghiệm

Mã 030 • Câu 1. (2 điểm)

Ghép bảng mỗi host một dòng trở lại bảng chờ ở bảng how="left". Việc validate="many_to_one" đòi bảng như thế nào?

A. Thay cho việc điền lại số hàng của bảng sau khi ghép.
B. Thay cho việc đo độ đều nhau giữa các host.
C. Bỏ qua số cột "host" của bảng và chuyển thành một dòng.
D. Dùng dấu host_id bên bảng host bị lặp, trước khi bảng bị nhân lên.

Đáp án: D là lựa chọn gần với ý định của câu hỏi, nhưng bản PDF viết sai ở cụm ``Dùng dấu". Quy tắc chính xác: khóa host_id ở bảng host phải duy nhất; trùng khóa sẽ gây MergeError.

Giải thích. `validate="many_to_one"` kiểm tra quan hệ nhiều hàng ở bảng trái ứng với tối đa một hàng cùng khóa ở bảng phải. Khi ghép bảng chỗ ở với bảng mỗi host một dòng, nhiều chỗ ở có thể cùng host, nhưng bảng host không được có nhiều hàng cùng `host_id`.

Nếu bảng host có khóa lặp, lệnh ghép dừng bằng `MergeError`. Nếu không kiểm tra, một hàng trái gặp hai hàng phải cùng khóa có thể sinh hai hàng kết quả. `how="left"` quy định giữ hàng bên trái, không tự bảo đảm khóa bên phải duy nhất.

Mở rộng / Cách nhớ. A, B và C không mô tả tác dụng của validate. Nội dung D có vẻ muốn diễn đạt việc *dừng khi phát hiện khóa lặp*; tài liệu giữ nguyên chữ nguồn và không coi câu chữ lỗi đó là một mô tả API chuẩn. Muốn kiểm tra khóa chưa tìm thấy đối tác, dùng thêm `indicator=True`; đó là kiểm tra khác với tính duy nhất của khóa.

Mã 030 • Câu 2. (2 điểm)

```
df = pd.DataFrame({'giá': [100, 50, np.nan]})
```

dtype của cột giá là gì?

A. object, vì có giá trị thiếu.
B. float64, vì dữ liệu số.
C. int64, vì không có dữ liệu số nguyên.
D. str, vì NaN là một chuỗi.

Đáp án: B. float64.

Giải thích. Đầu vào cột `giá` có hai số nguyên và `np.nan`. Với cách tạo DataFrame mặc định, pandas suy luận cột số thực `float64`, chứa `100.0`, `50.0` và `NaN`.

`np.nan` là giá trị thiếu kiểu số thực, không phải chuỗi `"NaN"`. Vì vậy không có chữ trong cột như phương án D nói; A cũng không đúng vì có thiếu không đồng nghĩa bắt buộc phải dùng `object`. `int64` NumPy thông thường không biểu diễn NaN.

Mở rộng / Cách nhớ. Muốn số nguyên có giá trị thiếu, có thể chủ động dùng dtype nullable `Int64` của pandas:

```
s = pd.Series([100, 50, pd.NA], dtype="Int64")
```

Đây là một cách tạo dữ liệu khác; nó không làm thay đổi đáp án của dòng code đã cho.

Mã 030 • Câu 3. (2 điểm)

Quy trình có trách nhiệm gồm tự phác việc cần làm, hỏi AI, đọc code, kiểm trên mẫu, rồi khai báo. Việc nào phải làm trước?

A. Chạy lại thấy không có thông báo lỗi là đủ.
B. Tự tính lại trên một mẫu nhỏ để đối chiếu với code.
C. Bỏ qua bước đọc code, chỉ cần tin vào AI viết code.
D. Khai báo AI, USAGE, mở xong rồi mới cần đọc code.

Đáp án: B. Tự tính lại trên một mẫu nhỏ để đối chiếu với code.

Giải thích. Trong quy trình nêu ở đề, trước khi chấp nhận kết quả chương trình cần có phép đối chiếu độc lập. Mẫu nhỏ tự tính được giúp kiểm tra cả công thức, bộ lọc, mẫu số và cách xử lý ô thiếu.

Code chạy không báo lỗi chỉ cho biết lần chạy đó chưa gặp lỗi thực thi; một công thức sai vẫn có thể chạy trơn tru. Bỏ đọc code để tin AI hoặc chỉ khai báo việc dùng AI không thay thế bước kiểm chứng kết quả.

Mở rộng / Cách nhớ. Một phép kiểm tra nên gồm dữ liệu đầu vào nhỏ, đáp án tự tính và kết quả chạy thật. Nếu dữ liệu có thiếu hoặc khóa không khớp, thêm tình huống đó vào mẫu. Cụm ``AI, USAGE, mở" trong D được giữ theo nguồn; nó không phải một hướng dẫn thao tác cho tài liệu này.

Mã 030 • Câu 4. (2 điểm)

```
ds = pd.Series(['wifi', '3g', 'wifi', '5g', 'wifi'])
ds = ['pocket wifi']
```

So sánh "wifi" in ds với pd.Series([new]).str.contains("wifi", case=False).

A. Phần tử thì True, tìm nền chuỗi cũng trả về True.
B. Phần tử thì True, tìm nền chuỗi JSON gốc thì là False.
C. Phần tử thì False, tìm nền chuỗi JSON gốc thì là True.
D. Cả hai đều True, vì cả hai cách đều thấy đoạn chữ wifi.

Đáp án: Theo đúng bản PDF: "wifi" in ds là False; biểu thức dùng new báo NameError nếu chạy độc lập. Không có phương án mô tả đúng cả hai.

Giải thích. Dòng đầu gán ds là một Series, nhưng dòng thứ hai gán lại cùng tên thành danh sách `['pocket wifi']`. Giá trị cũ không còn là giá trị của biến ds. Toán tử `in` trên danh sách tìm một phần tử bằng đúng `'wifi'`; danh sách chỉ có `'pocket wifi'`, nên kết quả là `False`.

Biểu thức thứ hai dùng biến `new`. Trong câu này không có dòng khai báo new, nên đánh giá nó trong phạm vi riêng của bài sẽ báo `NameError`. Không thể tự giả sử new là chuỗi JSON của ds. Vì vậy chưa được chọn C cho code gốc. **Một cách sửa minh họa, không phải nguyên văn đề:**

```
new = '["pocket wifi"]'
ds = ['pocket wifi']
print('wifi' in ds)  # False
print(pd.Series([new]).str.contains('wifi', case=False).iloc[0])
# True
```

Mở rộng / Cách nhớ. Sau khi *bổ sung giả định cụ thể* như trên, ý C mới phù hợp. Phép chứa phần tử trong danh sách khác phép tìm đoạn chữ trong một chuỗi. Dòng gán lại biến cũng khác với việc sửa phần tử của đối tượng cũ.

Mã 030 • Câu 5. (2 điểm)

extract có re.I và hai dấu □ in ra list nào?

```
ten = pd.Series(["80 N2", "12n2x"])
ten = ten.str.extract(r"(\d+)\s*(n2)\b", flags=re.I)[0].tolist()
```

A. ["80", "12"]
B. ["N", "n2"]
C. ["80", "12"]
D. [NaN, NaN]

Đáp án: Biến ten trở thành ['80', nan]; không có lựa chọn A--D đúng. Đoạn mã không có print nên không tự in danh sách.

Giải thích. Mẫu thực tế là `r"(\d+)\s*(n2)\b"`, có hai nhóm bắt: số và đơn vị n2. `flags=re.I` làm việc khớp không phân biệt hoa thường, nên `80 N2` khớp. Chọn cột nhóm `[0]` chỉ lấy phần số, tức chuỗi `'80'`.

Với `12n2x`, ranh giới cuối `\b` không tồn tại giữa 2 và x vì cả hai đều là ký tự từ. Hàng này không khớp, nên cho giá trị thiếu. Cuối cùng `tolist()` trả danh sách và phép gán đặt danh sách đó vào biến ten.

A và C giống hệt nhau nhưng đều sai ở phần tử thứ hai; B nhầm nội dung nhóm; D sai ở phần tử đầu. Ngoài ra, chữ trong câu hỏi nói ``hai dấu" nhưng code chỉ có một ranh giới từ ở cuối. Ô vuông trong đề là ký tự không hiển thị được của PDF nguồn; tài liệu không tự đoán thay nó bằng một token khác.

Mở rộng / Cách nhớ. Muốn hiển thị kết quả, thêm `print(ten)` sau phép gán. Muốn giữ ten là Series tên gốc, nên gán danh sách trích được sang biến mới, chẳng hạn `so_trich`, thay vì ghi đè tên biến.

Mã 030 • Câu 6. (2 điểm)

duckdb.sql("SELECT ...").df() trả về đối tượng nào?

A. Một chuỗi JSON nguyên văn của câu SQL.
B. Một file CSV tạm, chưa có cột.
C. Một DataFrame dữ liệu theo nội dung bảng.
D. Số hàng duy nhất, không còn bảng.

Đáp án: C. Một DataFrame dữ liệu theo nội dung bảng.

Giải thích. `duckdb.sql(...)` tạo kết quả quan hệ của truy vấn. Phương thức `.df()` lấy kết quả đó về dạng `pandas.DataFrame`, gồm các hàng và cột mà truy vấn trả ra.

Nó không tự xuất CSV, không biến câu SQL thành chuỗi JSON và không chỉ trả số hàng. Nếu truy vấn chỉ có một ô đếm thì `.df()` vẫn trả DataFrame một ô.

```
bang = duckdb.sql("SELECT 1 AS x").df()
print(type(bang).__name__)  # DataFrame
```

Mở rộng / Cách nhớ. Chọn API lấy kết quả theo nhu cầu: DataFrame để tiếp tục xử lý bằng pandas; `fetchone()` để lấy một hàng; `fetchall()` để lấy danh sách các hàng. Kiểu đầu ra phụ thuộc lời gọi lấy kết quả, không chỉ phụ thuộc số ô của truy vấn.

Mã 030 • Câu 7. (2 điểm)

Với mẫu r"p\d+p?X" cho str.match và str.fullmatch thế nào?

A. False và False
B. False và True
C. True và True
D. True và False

Đáp án: Thiếu chuỗi đầu vào để kiểm tra; không thể chọn duy nhất A, C hoặc D. B không thể xảy ra với cùng mẫu và cùng chuỗi.

Giải thích. Đề chỉ cung cấp mẫu `p\d+p?X`, không nói nó được áp dụng lên chuỗi nào. Mẫu nhận p thường, một hoặc nhiều chữ số, có thể thêm một p thường, rồi X hoa. Không có cờ bỏ phân biệt hoa thường.

`match` kiểm tra lần khớp từ đầu chuỗi; `fullmatch` yêu cầu khớp toàn chuỗi. Các ví dụ cho thấy đầu ra thay đổi theo chuỗi:

Chuỗi	match	fullmatch
`P7x`	False	False
`p7X`	True	True
`p7Xz`	True	False
`p7pX`	True	True

Như vậy A, C hoặc D đều có thể đúng với những chuỗi khác nhau. Nếu fullmatch đã True thì match cũng phải True, nên cặp False/True của B không thể là kết quả cùng điều kiện.

Mở rộng / Cách nhớ. Không được tự lấy chuỗi của một đề khác để điền vào chỗ thiếu. Cách sửa cần bổ sung rõ chuỗi nguồn hoặc một Series cụ thể; sau đó mới có đủ dữ kiện chọn đáp án.

Mã 030 • Câu 8. (2 điểm)

```
df = pd.DataFrame({
    'tên': ['Nguyễn cân', 'phòng răng', 'Nguyễn cân'],
    'giá': [100.0, 50.0, np.nan],
    'cọc': [200, 0, -1],
    'bmap': [True, False, True],
    'loc': ['a', 'b', 'c']
})
occ = bmap['giá']
occ.loc['a']
```

Sau các dòng này, giá trị trong bảng và trong cột là gì?

A. Cả hai đều 80.
B. Bảng là 80, cột là 100.
C. Bảng là 100, cột là 80.
D. Cả hai đều 100.

Đáp án: Chạy độc lập sẽ báo NameError tại bmap['giá']; không có đáp án A--D đúng.

Giải thích. DataFrame được gán vào biến df. Cột có tên `'bmap'` nằm *bên trong* df; việc tạo cột đó không khai báo một biến Python riêng tên bmap. Vì thế dòng `occ = bmap['giá']` dùng một tên chưa được định nghĩa.

Ngay cả nếu đoán thay bmap bằng df, bảng đang dùng chỉ mục mặc định 0, 1, 2. Cột `'loc'` chứa a, b, c không tự thành chỉ mục. Khi đó `occ.loc['a']` vẫn không tìm thấy nhãn a. Hơn nữa, dòng cuối chỉ lấy giá trị, không có phép gán 80 ở đâu.

Vì vậy bài này không thể được giải bằng cách dựa vào quy tắc view/copy rồi chọn một trong các cặp 100 và 80; code đã lỗi trước bước đó.

Mở rộng / Cách nhớ. Một phiên bản minh họa có đủ ý nghĩa cần chỉ rõ chỉ mục, biến được lấy cột và phép gán:

```
bang = df.set_index('loc').copy()
occ = bang['giá']
occ.loc['a'] = 80
```

Đây là code đã sửa. Với pandas 3.x Copy-on-Write, bang vẫn giữ 100 còn occ là 80; với hành vi cũ tắt Copy-on-Write, thay đổi có thể truyền về bang. Không lấy kết quả phiên bản sửa để gọi là đầu ra của code gốc.

Mã 030 • Câu 9. (2 điểm)

```
df = pd.DataFrame({
    'tên': ['Nguyễn cân', 'Phòng răng', 'Nguyễn cân'],
    'giá': [100.0, 50.0, np.nan],
    'cọc': [200, 0, -1],
    'loc': ['a0', 'b0', 'c1']
})
print(df['giá'].groupby(df['index']).sum().loc['c1'])
```

A. NaN
B. 1
C. 2
D. 3

Đáp án: KeyError: 'index'; không có đáp án đúng trong A--D.

Giải thích. Bảng có các cột `'tên'`, `'giá'`, `'cọc'`, `'loc'`. Không có cột `'index'`. Khi tính đối số `df['index']` của groupby, pandas báo KeyError; phép nhóm, cộng và lấy `'c1'` chưa được thực hiện.

Chỉ mục hàng mặc định 0, 1, 2 cũng không được truy cập bằng `df['index']`. Thuộc tính `df.index` và cột có tên `'index'` là hai thứ khác nhau. **Nếu ý định là nhóm theo cột loc, một cách sửa là:**

```
print(df['giá'].groupby(df['loc']).sum().loc['c1'])
# 0.0 với sum mặc định
```

Giá của c1 duy nhất là NaN. Sum mặc định có `min_count=0`, nên nhóm toàn thiếu có tổng 0.0. Nếu muốn giữ thiếu khi nhóm không có giá nào:

```
print(df['giá'].groupby(df['loc']).sum(min_count=1).loc['c1'])
# nan
```

Mở rộng / Cách nhớ. Cả sửa tên cột và chọn chính sách tổng nhóm thiếu đều phải được nói rõ. Không tự chuyển tổng nhóm toàn thiếu thành một kết quả khác rồi gán cho code đề.

Mã 030 • Câu 10. (2 điểm)

Cột khu là Categorical với các mức A, B, C, nhưng bảng chỉ có hàng A và B. groupby(..., observed=True) và observed=False khác nhau thế nào?

A. Cả hai cách đều thêm khu C với trung bình 0.
B. Cả hai cách đều loại khu C.
C. Cả hai cách đều bỏ khu C vì không có hàng.
D. Cả hai kết quả đều là NaN.

Đáp án: Không có lựa chọn đúng. observed=True chỉ có nhóm đã xuất hiện A, B; observed=False còn giữ mức C chưa có quan sát.

Giải thích. Với khóa Categorical, danh sách mức cho phép và các mức thật sự có hàng là hai tập khác nhau. `observed=True` trả nhóm có quan sát; `observed=False` còn đưa mức chưa xuất hiện vào kết quả.

Đề không ghi phép tổng hợp cụ thể sau groupby. Nếu tính `mean` như các lựa chọn nhắc đến, nhóm C không có giá nào nên trung bình là NaN, không phải 0. Hai nhóm A, B vẫn có kết quả bình thường. Vì vậy A sai, B và C sai, D cũng sai.

```
df = pd.DataFrame({
    'khu': pd.Categorical(['A', 'B'], categories=['A', 'B', 'C']),
    'gia': [10.0, 20.0],
})
print(df.groupby('khu', observed=True)['gia'].mean())
# A: 10.0, B: 20.0
print(df.groupby('khu', observed=False)['gia'].mean())
# A: 10.0, B: 20.0, C: NaN
```

Mở rộng / Cách nhớ. Giá trị ở nhóm rỗng phụ thuộc phép tổng hợp: mean thường cho NaN; size có thể cho 0. Đừng dùng kết quả của phép đếm để suy ra trung bình của nhóm rỗng bằng 0.

Mã 030 • Câu 11. (2 điểm)

Tệp ma.csv chỉ có nội dung sau, không khai báo dtype:

```
id
007
```

```
df = pd.read_csv("ma.csv")
print(raw.loc[0, "id"])
```

Giá trị in ra là gì?

A. Số 7, kiểu số nguyên.
B. Chuỗi "7", còn dấu số 0.
C. Chuỗi có hai ký tự.
D. Giá trị thiếu, vì 007 không hợp lệ.

Đáp án: Theo code nguyên văn: NameError vì raw chưa được khai báo. Nếu sửa raw thành df ở dòng print, chọn A: số nguyên 7.

Giải thích. Tệp được đọc vào biến df nhưng dòng in lại dùng raw. Khi xét bài độc lập, không có định nghĩa raw nên chương trình dừng tại dòng print; A--D không có phương án báo lỗi này.

Nếu chỉ sửa cho thống nhất tên biến:

```
df = pd.read_csv("ma.csv")
print(df.loc[0, "id"])  # 7
```

Không khai báo dtype, pandas suy luận `007` là biểu diễn của số nguyên 7. Giá trị số không giữ hai số 0 ở đầu; nó cũng không tự trở thành chuỗi `'7'`.

Mở rộng / Cách nhớ. Mã định danh cần giữ độ dài nên được đọc bằng dtype chuỗi:

```
df = pd.read_csv("ma.csv", dtype={"id": "string"})
print(df.loc[0, "id"])  # 007
```

Hai việc cần tách riêng: sửa tên biến để lệnh chạy được, và lựa chọn dtype để giữ đúng ý nghĩa mã.

Mã 030 • Câu 12. (2 điểm)

Tệp ma.csv chỉ có nội dung sau, không khai báo dtype:

```
khu = pd.Series([' Ba Lát', 'BA LÁT', 'bà lạt', 'Da Lat'])
khu = khu.str.strip().str.lower().nunique()
```

Đoạn mã in ra gì?

A. 4
B. 2
C. 3
D. 1

Đáp án: Code không có print nên không in gì. Sau hai dòng, biến khu bằng 3; nếu bổ sung print(khu) thì chọn C.

Giải thích. Dòng thứ hai tính số tên phân biệt rồi *gán lại* vào khu. Phép gán không tự in kết quả, kể cả trong một ô notebook thông thường. Nội dung giới thiệu tệp CSV ở đầu câu cũng không có thao tác đọc tệp tương ứng trong code.

Các chuỗi sau strip và lower là:

```
['ba lát', 'ba lát', 'bà lạt', 'da lat']
```

Hai chuỗi đầu trùng nhau, nhưng `'ba lát'`, `'bà lạt'` và `'da lat'` là ba cách viết khác nhau. Lower không sửa dấu hoặc chính tả. Vì vậy nunique tính ra 3.

Mở rộng / Cách nhớ. Nếu muốn hiển thị con số vừa tính, thêm `print(khu)`. Không thay các tên viết sai trong nguồn thành một tên chuẩn rồi mới đếm, vì thao tác chuẩn hóa đó chưa có trong đoạn mã đã cho.

Mã 030 • Câu 13. (2 điểm)

Hai nhóm mà extract lấy từ "4 bedrooms" là gì?

```
m = pd.Series(['4 bedrooms'])
print(m.str.extract(r'(\d+)\s*([Bb]edrooms)').iloc[0].tolist())
```

A. ['4', 'bedrooms']
B. ['4', 'Bedrooms']
C. ['4', 'bedrooms']
D. ['4', 'bedroom']

Đáp án: A và C cùng đúng vì hai lựa chọn bị trùng hệt nhau: ['4', 'bedrooms'].

Giải thích. Nhóm thứ nhất `(\d+)` lấy chuỗi số 4. Phần `\s*` nhận khoảng trắng. Nhóm thứ hai `([Bb]edrooms)` nhận chữ b hoa hoặc thường theo sau bởi `edrooms`; trong chuỗi đầu vào, nó nhận đúng `'bedrooms'`.

`extract` trả hai cột nhóm. `iloc[0]` lấy hàng đầu, rồi `tolist()` cho `['4', 'bedrooms']`. Nó không đổi chữ b thành hoa, cũng không tự cắt s cuối. Do A và C giống nhau, đề không có duy nhất một đáp án trắc nghiệm.

Mở rộng / Cách nhớ. Nếu chỉ muốn một cột số, dùng nhóm không bắt cho đơn vị:

```
pat = r'(\d+)\s*(?:[Bb]edrooms)'
```

Một cặp ngoặc thường là một nhóm được trả ra; `(?:...)` chỉ gom mẫu mà không thêm cột kết quả.

Mã 030 • Câu 14. (2 điểm)

```
A = np.array([[10, 12, 11],
              [20, 21, 24],
              [30, 33, 31],
              [40, 44, 42]], dtype=np.int64)
R = A.reshape(2, 6)
print(R.itemsize, R.nbytes)
```

Hai số in ra là gì?

A. 4 96
B. 8 48
C. 8 96
D. 16 96

Đáp án: C. 8 96.

Giải thích. Dtype `np.int64` dùng 64 bit, tức 8 byte cho mỗi phần tử. Vì vậy `R.itemsize = 8`. A có $4 \times 3 = 12$ phần tử; reshape thành 2×6 không đổi số phần tử hay dtype.

$$\texttt{R.nbytes} = \texttt{R.size} \times \texttt{R.itemsize} = 12 \times 8 = 96 \text{ byte}.$$

Mở rộng / Cách nhớ. Reshape chỉ đổi cách tổ chức các phần tử. Với mảng liên tục như ví dụ, R có thể dùng chung dữ liệu với A; nbytes vẫn mô tả dung lượng phần tử của R chứ không khẳng định có thêm 96 byte mới được cấp phát. Nó cũng không gồm toàn bộ bộ nhớ quản lý đối tượng Python.

Mã 030 • Câu 15. (2 điểm)

18.534 chỉ có 9.712 host, mỗi host một trung vị giá. Câu nào đúng khi đưa số này vào báo cáo?

A. Size theo host_id đếm số bản trùng bản ghi.
B. Bảng host vẫn có 18.534 hàng.
C. Trung vị đó chính là trung vị của 18.534 host.
D. Phải nói rõ số này là trung vị theo host, không phải theo chỗ ở.

Đáp án: D. Phải nói rõ số này là trung vị theo host, không phải theo chỗ ở.

Giải thích. Sau khi gom các chỗ ở theo host và mỗi host có một trung vị giá, đơn vị quan sát đã chuyển thành host. Không còn một hàng cho từng chỗ ở ban đầu. Vì thế phải ghi rõ cấp tổng hợp trong báo cáo.

`size` theo host đếm số hàng của mỗi nhóm, không tự đếm số bản ghi trùng. Trung vị của các giá đại diện theo host cũng không đồng nhất với trung vị trên toàn bộ các chỗ ở. Bản PDF có câu chữ rút thiếu ở đầu, nhưng D vẫn mô tả đúng yêu cầu báo cáo.

Mở rộng / Cách nhớ. Ví dụ host H1 có giá 10, 10, 10 còn H2 có giá 100. Trung vị mỗi host là 10 và 100, trung vị hai giá đó là 55; trung vị của bốn chỗ ở ban đầu là 10. Mỗi host có một trọng số khác với mỗi chỗ ở có một trọng số.

Mã 030 • Câu 16. (2 điểm)

Đánh giá cuối cùng trong tháng 8 là ngày 18/08/2026. Nhãn tháng 8 của resample("ME") là ngày nào?

A. 2026-08-18
B. 2026-08-31
C. 2026-09-01
D. 2026-08-01

Đáp án: B. 2026-08-31.

Giải thích. Tần suất `ME` dùng nhãn ngày cuối tháng. Tháng 8/2026 kết thúc vào 31/08, nên nhóm tháng 8 có nhãn đó, dù đánh giá mới nhất trong nhóm là ngày 18/08.

Nhãn kỳ là một mốc quy ước theo lịch, không phải thời điểm quan sát cuối và không xác nhận nguồn đã thu đủ cả tháng. Không đổi nhãn sang 18/08 hoặc 01/09 chỉ vì ngày dữ liệu mới nhất.

Mở rộng / Cách nhớ. Khi so sánh tổng tháng, cần kiểm tra kỳ có đủ độ phủ quan sát hay chưa. Câu 24 dùng chính tình huống tháng cuối chưa đủ và yêu cầu loại kỳ có nhãn sau ngày quan sát cuối.

Mã 030 • Câu 17. (2 điểm)

DataFrame đã đọc đúng: cột khu còn chuỗi "NA", cột giá có NaN. Đưa bảng này vào DuckDB. Câu nào đúng?

A. DuckDB nhìn NaN như số 0 và nhóm "NA" biến mất.
B. "NA" thành NULL, còn NaN của giá được giữ để tính trung bình.
C. "NA" là giá trị văn bản của cột khu.
D. NaN của giá thành NULL và AVG bỏ qua; "NA" vẫn là một mã khu.

Đáp án: D mô tả đầy đủ; C cũng đúng. Vì vậy câu hỏi có hai lựa chọn đúng về mặt logic.

Giải thích. Đề khẳng định bảng đã đọc đúng và `'NA'` vẫn là chuỗi trong cột khu. Do đó C đúng ngay từ giả thiết: đây là giá trị văn bản, không phải giá trị thiếu.

Khi DuckDB đọc bảng pandas này, NaN của cột giá được xử lý như SQL NULL; phép AVG bỏ qua các giá thiếu. Chuỗi `'NA'` vẫn là mã khu hợp lệ. Đó là toàn bộ nội dung đúng của D. A sai vì không tự thay thiếu bằng 0; B sai vì không tự biến mã chuỗi NA thành NULL trong tình huống đã cho.

Mở rộng / Cách nhớ. Nếu bài bắt buộc chọn một phương án *đầy đủ nhất*, chọn D. Nếu chỉ hỏi ``câu nào đúng'' như văn bản hiện tại, phải thừa nhận cả C và D. Quy tắc nhận ký hiệu thiếu khi đọc CSV là bước khác; không áp lại nó lên chuỗi đã được giữ đúng trong DataFrame.

Mã 030 • Câu 18. (2 điểm)

```
df = pd.DataFrame({
    'loại': ['Nguyễn cân', 'Phòng riêng', 'Nguyễn cân'],
    'giá': [100.0, 50.0, np.nan],
    'index': ['a0', 'b0', 'c1']
})
mot = df.loc[['a0', 'b0'], 'giá']
hai = df.loc[['a0', 'b0'], 'giá']
print(type(mot).__name__, type(hai).__name__)
```

Hai tên kiểu in ra là gì?

A. Series DataFrame
B. Series Series
C. DataFrame DataFrame
D. DataFrame Series

Đáp án: KeyError ở phép lấy mot; không có đáp án A--D đúng với code gốc.

Giải thích. Tạo một cột tên `'index'` không đặt nó thành chỉ mục hàng. Bảng đang có chỉ mục mặc định 0, 1, 2. `loc[['a0', 'b0'], ...]` tìm các nhãn a0, b0 trong chỉ mục, không tìm trong cột index, nên báo KeyError ngay ở dòng gán mot. Dòng in tên kiểu chưa được chạy.

Còn một lỗi độc lập: cả hai dòng mot và hai đều chọn cột đơn `'giá'`. Sau khi sửa chỉ mục, cả hai mới là Series, tương ứng B; không có dòng nào dùng danh sách một cột để giữ DataFrame. **Bản sửa minh họa nếu muốn phân biệt Series và DataFrame:**

```
df = df.set_index('index')
mot = df.loc[['a0', 'b0'], 'giá']
hai = df.loc[['a0', 'b0'], ['giá']]
print(type(mot).__name__, type(hai).__name__)
# Series DataFrame
```

Mở rộng / Cách nhớ. Bản sửa này cho A nhưng đã thay *hai* điểm của nguồn: đặt chỉ mục và đổi bộ chọn cột của hai. Không được gọi A là đầu ra của đoạn mã nguyên văn.

Mã 030 • Câu 19. (2 điểm)

```
doc = pd.Series(["18/08/2026", "31/06/2026", None])
goc = pd.to_datetime(doc, format="%d/%m/%Y", errors="coerce")
print(int(goc.notna().sum() & doc.isna().sum()))
```

Dòng in ra bao nhiêu?

A. 1
B. 0
C. 3
D. 2

Đáp án: A. 1, nhưng chương trình đang AND hai số đếm, không đếm lỗi đọc ngày theo từng hàng.

Giải thích. Biến doc là nguồn chuỗi, goc là kết quả đọc ngày. Ngày 18/08/2026 hợp lệ; 31/06/2026 không hợp lệ; None vốn thiếu. Vì thế:

```
goc.notna().sum()  # 1: chỉ một ngày đọc thành công
doc.isna().sum()   # 1: một ô nguồn vốn thiếu
```

Hai lời gọi sum được thực hiện *trước* toán tử &. Toán tử này tác động lên hai số nguyên: $1 \,\&\, 1 = 1$. Ép int rồi print cho 1.

Kết quả đúng số 1 không chứng minh công thức đang đếm ngày không đọc được. Đây chỉ là sự trùng hợp trên bộ dữ liệu này. Nếu hai số đếm là 2 và 1, AND bit cho $2 \,\&\, 1 = 0$.

Mở rộng / Cách nhớ. Muốn đếm nguồn có giá trị nhưng chuyển ngày thất bại, dùng mặt nạ theo từng hàng rồi mới cộng:

```
loi_doc = doc.notna() & goc.isna()
print(int(loi_doc.sum()))  # 1: hàng 31/06/2026
```

Phép toán trên các số đếm và phép giao điều kiện theo hàng có ý nghĩa khác nhau dù dùng cùng ký hiệu &.

Mã 030 • Câu 20. (2 điểm)

```
s = pd.Series([10, 30], index=pd.to_datetime(["2026-01-31", "2026-03-31"]))
print(s.asfreq("ME").shift(1).loc["2026-03-31"])
```

Ô in ra là gì?

A. Đúng bằng giá trị của tháng 1, tháng lên trước trên lịch.
B. Số 0, vì tháng 2 không có hàng trong Series gốc.
C. Bằng giá trị của tháng 3, vì shift không đổi hàng.
D. NaN, vì tháng 2 vừa được thêm đang thiếu.

Đáp án: D. NaN, vì tháng 2 vừa được thêm đang thiếu.

Giải thích. `asfreq('ME')` chèn đầy đủ các mốc cuối tháng, nên từ hai hàng 31/01 và 31/03 tạo thành ba hàng. 28/02 không có dữ liệu nguồn, được gán NaN. Sau đó shift dịch giá trị một hàng, giữ các nhãn:

Ngày	Sau asfreq	Sau shift
31/01	10	NaN
28/02	NaN	10
31/03	30	NaN

Vì hàng ngay trước tháng 3 trên lịch đã được thêm là tháng 2 với giá thiếu, ô tháng 3 sau dịch nhận NaN. Lệnh print hiển thị `nan`.

Mở rộng / Cách nhớ. Nếu không chèn lịch mà gọi `shift(1)` trực tiếp trên bảng hai hàng, ô tháng 3 sẽ lấy 10 từ tháng 1. Kỳ trước theo hàng khác kỳ trước theo tháng khi dữ liệu thiếu mốc thời gian.

Phần II: Tự luận

Mã 030 • Câu 21. (10 điểm)

Bảng có khu và giá.

```
df = pd.DataFrame({"khu": ["A", "A", "B"], "gia": [10, 30, 40]})
tb = df.groupby("khu")["gia"].mean()
tb.to_csv("tb.csv", index=False)
```

a) (5 điểm) Tệp tb.csv còn những gì? Vì sao không còn tên khu?

b) (5 điểm) Viết dòng ghi tệp để còn cột khu và cột trung bình, nhưng không ghi thêm một cột chỉ mục số.

Đáp án: a) CSV chỉ còn cột gia và hai trung bình 20.0, 40.0; tên khu nằm ở index đã bị bỏ. b) Chuyển index thành cột rồi ghi với index=False.

a) `groupby("khu")["gia"].mean()` trả một Series: tên Series là `gia`, còn A và B nằm ở chỉ mục có tên `khu`. Giá trung bình là:

$$\overline{g}_A = \frac{10+30}{2} = 20, \qquad \overline{g}_B = 40.$$

`to_csv(index=False)` bỏ chỉ mục khi ghi. Vì mã khu nằm trong chỉ mục, chúng cũng bị bỏ. Nội dung tệp còn:

```
gia
20.0
40.0
```

Giá trị vẫn còn nhưng tệp không còn nói trung bình nào thuộc khu nào.

b) Chuyển chỉ mục khu thành một cột thông thường trước khi ghi:

```
tb.reset_index().to_csv("tb.csv", index=False)
```

Tệp mới là:

```
khu,gia
A,20.0
B,40.0
```

`reset_index()` đưa mã khu về cột `khu`; `index=False` lúc này chỉ ngăn ghi thêm chỉ mục số mới 0, 1. Nếu muốn tên cột thể hiện rõ trung bình:

```
tb.rename("gia_trung_binh").reset_index().to_csv("tb.csv", index=False)
```

Mở rộng / Cách nhớ. Có thể tránh đưa khóa vào chỉ mục ngay từ đầu:

```
tb = df.groupby("khu", as_index=False)["gia"].mean()
```

Trước khi xuất bảng, luôn kiểm tra các khóa định danh đang là cột hay chỉ mục; bỏ index chỉ an toàn nếu index không chứa thông tin cần giữ.

Mã 030 • Câu 22. (10 điểm)

Tệp gia_theo_khu.csv. NA là mã một khu, ô trống là chưa có giá, 0 là giá đã ghi.

```
khu,gia
NA,200
TB,50
NA,
TB,20
NA,40
NA,0
```

```
df = pd.read_csv(
    "gia_theo_khu.csv",
    dtype={"gia": "float64"},
    keep_default_na=False,
    na_values={"gia": [""]},
)
print(df.loc[df["khu"] == "NA", "gia"].mean())
```

a) (5 điểm) Dòng in ra bao nhiêu? Ô giá trống và giá 0 của khu NA được xử lý thế nào?

b) (5 điểm) Bỏ keep_default_na=False thì cùng dòng in ra gì? Nếu vẫn giữ cờ đó nhưng cột có chuỗi "0" là thiếu, trung bình khu NA thành bao nhiêu?

Đáp án: a) 80.0. Ô trống là NaN và bị bỏ qua; giá 0 vẫn được tính. b) Bỏ keep_default_na=False: NaN. Giữ cờ nhưng coi cả "0" là thiếu: 120.0.

a) `keep_default_na=False` tắt bộ ký hiệu thiếu mặc định, nhờ đó mã khu `"NA"` vẫn là chuỗi. Tham số `na_values` trong đề chỉ khai báo ô rỗng ở cột giá là thiếu. Dtype cột giá được khóa là `float64`.

Bốn hàng khu NA có các giá `[200.0, NaN, 40.0, 0.0]`. `mean` bỏ qua giá thiếu nhưng giữ số 0 hợp lệ:

$$\bar{g}_{NA} = \frac{200 + 40 + 0}{3} = 80.$$

Ô trống không được tính vào mẫu số, còn giá 0 có đóng góp vào mẫu số. Đây là lý do không thể coi cả hai là cùng một loại dữ liệu.

b) Nếu bỏ `keep_default_na=False`, giá trị mặc định là `True`; chuỗi `"NA"` ở cột khu sẽ bị nhận là thiếu. Điều kiện `df["khu"] == "NA"` không còn chọn được các hàng đó. Trung bình của tập rỗng là `NaN`, lệnh print hiển thị `nan`, không phải 0.

Nếu giữ cờ nhưng khai báo thêm `"0"` là thiếu ở cột giá:

```
df = pd.read_csv(
    "gia_theo_khu.csv",
    dtype={"gia": "float64"},
    keep_default_na=False,
    na_values={"gia": ["", "0"]},
)
```

Các giá được tính ở khu NA chỉ còn 200 và 40, nên trung bình trở thành $(200 + 40)/2 = 120$. Đó là kết quả của quy ước thay đổi; theo ý nghĩa nguồn đã nêu, coi 0 là thiếu là sai và làm trung bình tăng từ 80 lên 120.

Mở rộng / Cách nhớ. Nên khai báo ký hiệu thiếu theo từng cột. Một chuỗi có thể là mã hợp lệ trong cột này nhưng là ký hiệu thiếu trong cột khác. Sau đọc, kiểm tra số hàng, số giá thiếu và sự tồn tại của các mã quan trọng trước khi tổng hợp.

Mã 030 • Câu 23. (10 điểm)

Hai lần ghi giá của cùng ba mã. Lần 2 chưa có giá của c.

```
lan1 = pd.Series({"a": 100, "b": 200, "c": 300})
lan2 = pd.Series({"a": 130, "b": 210, "c": np.nan})
```

a) (5 điểm) lan2.mean() - lan1.mean() bằng bao nhiêu? Vì sao số này chưa phản ánh mức thay đổi của cùng một tập chỗ ở?

b) (5 điểm) Viết code tính hiệu trung bình lần 2 trừ trung bình lần 1 chỉ trên những mã có giá ở cả hai lần. Kết quả là bao nhiêu?

Đáp án: a) -30.0, do hai trung bình dùng hai tập mã khác nhau. b) Trên các mã có giá ở cả hai lần, hiệu trung bình là +20.0.

a) Lần 1 có đủ a, b, c nên:

$$\overline{g}_1 = \frac{100 + 200 + 300}{3} = 200.$$

Lần 2 chỉ có a và b có giá, vì mean bỏ qua NaN:

$$\overline{g}_2 = \frac{130 + 210}{2} = 170, \qquad \overline{g}_2 - \overline{g}_1 = 170 - 200 = -30.$$

Hiệu âm này không mô tả mức giảm của cùng một tập: lần 1 có mã c giá cao 300, lần 2 mã c không được tính. Thay đổi thành phần quan sát có thể làm trung bình giảm dù cả a và b đều tăng giá.

b) Căn theo mã rồi chỉ giữ các mã có dữ liệu ở cả hai lần. Một cách rõ ràng:

```
cap = pd.concat([lan1.rename("lan1"), lan2.rename("lan2")], axis=1)
cap = cap.dropna(subset=["lan1", "lan2"])
hieu_tb = cap["lan2"].mean() - cap["lan1"].mean()
print(hieu_tb)  # 20.0
```

Hai mã được giữ là a và b. Trung bình lần 1 là $(100 + 200)/2 = 150$, lần 2 là 170; hiệu $170 - 150 = 20$.

Cũng có thể tính trung bình của các hiệu ghép đúng mã:

```
print((cap["lan2"] - cap["lan1"]).mean())  # 20.0
```

Mở rộng / Cách nhớ. Trên cùng một tập cặp quan sát và cùng trọng số, hiệu hai trung bình bằng trung bình các hiệu: $\overline{y} - \overline{x} = \overline{y - x}$. Ở đây hai mức tăng là 30 và 10, trung bình 20. Không điền giá thiếu của c bằng 0 nếu nguồn chưa xác nhận giá thực sự bằng 0.

Mã 030 • Câu 24. (10 điểm)

Mỗi ngày từ 01/07/2026 đến 18/08/2026 có đúng một đánh giá. resample("ME") gắn nhãn 31/07 cho tháng 7 và nhãn 31/08 cho tháng 8.

```
ngay = pd.date_range("2026-07-01", "2026-08-18", freq="D")
s = pd.Series(1, index=ngay)
theo = s.resample("ME").sum()
```

a) (5 điểm) Hai tổng trong theo là bao nhiêu? Vì sao không kết luận tháng 8 ít đánh giá hơn tháng 7 khi so cùng độ dài một tháng?

b) (5 điểm) Viết biểu thức chỉ giữ những kỳ có nhãn không vượt quá ngày đánh giá cuối cùng. Còn lại kỳ nào và tổng bao nhiêu?

Đáp án: a) Kỳ 31/07 có 31, kỳ 31/08 có 18. Tháng 8 mới được quan sát 18 ngày. b) Chỉ giữ kỳ 31/07/2026, tổng bằng 31.

a) `date_range` với hai mốc và tần suất ngày tạo đủ các ngày từ 01/07 đến hết 18/08. Mỗi ngày có giá trị 1, nên tổng tháng bằng số ngày có dữ liệu:

Nhãn kỳ	Phạm vi quan sát thật	Tổng
31/07/2026	01/07--31/07: 31 ngày	31
31/08/2026	01/08--18/08: 18 ngày	18

Không thể từ $18 < 31$ kết luận lượng đánh giá của *một tháng đầy đủ* tháng 8 thấp hơn tháng 7. Hai tổng có độ dài quan sát khác nhau; nhãn 31/08 không làm xuất hiện dữ liệu từ 19/08 đến 31/08. Với số liệu đã cho, tốc độ trung bình trong những ngày có dữ liệu của cả hai kỳ đều là 1 đánh giá/ngày.

b) Lấy ngày quan sát cuối rồi so với nhãn kỳ:

```
du_ky = theo.loc[theo.index <= s.index.max()]
print(du_ky)
# 2026-07-31    31
```

Ngày cuối là 18/08/2026. Nhãn 31/07 không vượt mốc đó nên được giữ; 31/08 vượt nên bị loại. Tổng của phần giữ lại là 31.

Mở rộng / Cách nhớ. Quy tắc này phù hợp ở đây vì nguồn bắt đầu đúng 01/07 và mỗi ngày trong khoảng đều có một đánh giá. Với nguồn bắt đầu giữa tháng, chỉ kiểm tra nhãn không vượt ngày cuối chưa đủ chứng minh kỳ đầu đầy đủ; cần kiểm tra cả mốc bắt đầu và độ phủ quan sát.

Mã 030 • Câu 25. (10 điểm)

Hai tên căn lấy số mét vuông đứng thành một từ riêng.

```
ten = pd.Series(["18m2", "18m2x"])
print(ten.str.extract(r"\b(\d+)m2\b")[0].tolist())
```

a) (5 điểm) Dòng in ra list nào? Lệnh có đúng vì lỗi không? Vì sao 18m2 không cho số 18?

b) (5 điểm) Viết lại mẫu bằng chuỗi thô để ranh giới từ còn hiệu lực. List mới là gì?

Đáp án: a) ['18', nan]; lệnh chạy bình thường. Tiền đề ``18m2 không cho số 18" không đúng với code in trong PDF. b) Mẫu đã là chuỗi thô; kết quả giữ nguyên.

a) Bản PDF dùng `r"\b(\d+)m2\b"`, *đã có* tiền tố r. Do đó các dấu gạch chéo được giữ để bộ regex hiểu hai `\b` là ranh giới từ; chúng không bị Python biến thành backspace.

`18m2` bắt đầu và kết thúc tại ranh giới từ, nên khớp toàn mẫu và nhóm số trả chuỗi `'18'`. Với `18m2x`, giữa 2 và x không có ranh giới từ vì cả hai đều là ký tự từ. Hàng này không khớp, cho NaN. Đầu ra là `['18', nan]`, không phải `[nan, nan]`.

Câu chữ ``Lệnh có đúng vì lỗi không?" và ``Vì sao 18m2 không cho số 18?" được giữ nguyên, nhưng không dùng chúng để phủ nhận kết quả thực thi đã kiểm chứng.

b) Không cần thêm r lần nữa; có thể viết lại đúng mẫu hiện tại:

```
pat = r"\b(\d+)m2\b"
print(ten.str.extract(pat)[0].tolist())
# ['18', nan]
```

Mở rộng / Cách nhớ. Nếu *bỏ* tiền tố r, hai `\b` của chuỗi thường sẽ thành backspace, khi đó mới có thể cho `[nan, nan]` trên hai tên này. Đây là phân tích biến thể để giải thích lỗi có thể đã xảy ra khi soạn đề, không phải thay đổi code nguồn. Kết quả nhóm bắt là chuỗi, cần chuyển sang số nếu muốn tính toán.

Mã 030 • Câu 26. (10 điểm)

Hai lượt ở. Số đêm ở đây là số ngày vượn, phần giờ lẻ không làm tròn thành thêm một ngày.

```
luot = pd.DataFrame({
    "vao": pd.to_datetime(["2026-03-01 20:00", "2026-03-10 08:00"]),
    "ra": pd.to_datetime(["2026-03-03 02:00", "2026-03-10 20:00"]),
})
hieu = luot["ra"] - luot["vao"]
print(luot.loc[0, "vao"], hieu.dt.days.to_list())
print(hieu.dt.days.to_list())
```

a) (5 điểm) Số ngày vượn của lượt đầu là bao nhiêu? Lượt đó dài 1 ngày 6 giờ, vì sao không phải 2?

b) (5 điểm) hieu.dt.days có thể âm nào? List số ngày vượn của cả hai lượt là gì, và tổng là bao nhiêu?

Đáp án: a) Lượt đầu có 1 ngày trọn. b) Code cho [1, 0], tổng 1; trong dữ liệu này không có số âm.

a) Cụm ``ngày vượn'' trong PDF có dấu hiệu lỗi chữ; dựa vào quy ước được nêu ngay sau đó, bài yêu cầu số ngày trọn, không làm tròn phần giờ lẻ lên thêm một ngày. Lượt đầu từ 01/03 lúc 20:00 đến 03/03 lúc 02:00 là 30 giờ, tức 1 ngày 6 giờ:

$$\left\lfloor \frac{30}{24} \right\rfloor = 1.$$

Chạm tới ngày 03/03 không đồng nghĩa đã ở đủ 2 lần 24 giờ. Vì vậy số ngày trọn là 1.

b) hieu là Series các khoảng thời gian; `hieu.dt.days` hợp lệ và lấy phần ngày của từng khoảng. Lượt thứ hai dài 12 giờ cùng ngày, nên có 0 ngày trọn. `to_list()` và `tolist()` đều chuyển Series thành danh sách.

Hai dòng in của code nguyên văn là:

```
2026-03-01 20:00:00 [1, 0]
[1, 0]
```

Dòng đầu in *ngày vào* của lượt đầu cùng cả danh sách ngày trọn; nó không in riêng hiệu thời gian hay số ngày của lượt đầu. Dòng hai in danh sách một lần nữa. Có thể lấy tổng bằng:

```
print(hieu.dt.days.sum())  # 1
```

Về câu hỏi có thể âm: các thời điểm trong đề đều có ngày ra sau ngày vào nên không âm. Nói chung, nếu ra trước vào, Timedelta sẽ âm và phần days có thể âm; ví dụ khoảng -12 giờ được biểu diễn thành -1 ngày cộng 12 giờ, nên phần days là -1.

Mở rộng / Cách nhớ. Trước khi tính số đêm cần kiểm tra thứ tự thời gian. Nếu muốn số ngày thực:

```
so_ngay_thuc = hieu.dt.total_seconds() / 86400
# [1.25, 0.5]
```

Còn dt.days cho phần ngày theo biểu diễn Timedelta. Không cộng giờ dư giữa các lượt riêng biệt để làm tròn lại nếu yêu cầu là đếm ngày trọn cho từng lượt.

Mã đề 001 (luyện tập)

Đáp án nhanh phần trắc nghiệm

Câu	1	2	3	4	5	6	7	8	9	10
Đáp án	B	C	B	B	D	A	D	A	D	C
Câu	11	12	13	14	15	16	17	18	19	20
Đáp án	C	A	B	A	B	C	C	C	D	D

Đề luyện tập có điểm số 1 điểm/câu trắc nghiệm, 5 điểm/câu tự luận. Code và yêu cầu khoảng 6 dòng được giữ nguyên; phần tự luận có thêm mẫu trả lời ngắn bên cạnh giải thích đầy đủ.

Đi đến từng câu Câu 1 • Câu 2 • Câu 3 • Câu 4 • Câu 5 • Câu 6 • Câu 7 • Câu 8 • Câu 9 • Câu 10 • Câu 11 • Câu 12 • Câu 13

Câu 14 • Câu 15 • Câu 16 • Câu 17 • Câu 18 • Câu 19 • Câu 20 • Câu 21 • Câu 22 • Câu 23 • Câu 24 • Câu 25 • Câu 26

Đã có đủ câu 1--26. Phần đề nằm trước hộp đáp án; các bản sửa code chỉ nằm trong lời giải.

Phần I: Trắc nghiệm

Mã 001 • Câu 1. (1 điểm)

Minh làm bài tập lớn tại nhà (chế độ mở). Minh nhờ AI viết hàm đọc và làm sạch cột giá, tự viết ca thử, phát hiện hàm đổi ô trống thành 0 nên sửa lại khoảng một nửa số dòng. Theo chính sách AI của môn, nhận định nào đúng?

A. Phải xin phép giảng viên trước khi dùng, vì để AI viết một hàm cho bài tập lớn là gian lận.
B. Phải khai báo đã dùng AI vào việc gì; khi vấn đáp vẫn phải giải thích được từng dòng.
C. Không cần khai báo việc dùng AI, vì Minh đã tự kiểm thử và tự sửa lại phần code sai.
D. Khai báo đầy đủ trong AI_USAGE.md là đủ, khi vấn đáp sẽ không bị hỏi phần do AI viết.

Đáp án: B. Phải khai báo đã dùng AI vào việc gì và vẫn phải giải thích được từng dòng khi vấn đáp.

Giải thích. Tình huống là bài tập lớn tại nhà ở chế độ mở. Theo chính sách được câu hỏi viện dẫn, dùng AI có thể được chấp nhận khi minh bạch phạm vi hỗ trợ và người học hiểu, kiểm tra, chịu trách nhiệm về sản phẩm. Minh đã tự viết ca thử và sửa lỗi, nhưng việc đó không xóa đi sự thật đã dùng AI.

A tự thêm điều kiện phải xin phép trước mà đề không đặt ra. C sai vì tự sửa code không thay thế nghĩa vụ khai báo. D sai vì khai báo không miễn việc hiểu và trả lời vấn đáp phần code có AI hỗ trợ.

Mở rộng / Cách nhớ. Lỗi điền ô trống thành 0 trong tình huống này làm thay đổi ý nghĩa dữ liệu: chưa biết giá khác với khuyến mãi 0. Khi khai báo, nên nêu AI đã hỗ trợ phần nào và cách đã kiểm chứng. Đây là phân tích chính sách trong đề, không phải một yêu cầu mới về quy trình dùng tài liệu ôn tập.

Mã 001 • Câu 2. (1 điểm)

Repo bài tập lớn có requirements.txt (tạo bằng pip freeze) và tệp .python-version ghi 3.12.3. Máy của Hà chỉ cài Python 3.11 và không dùng công cụ quản lý phiên bản Python nào. Hà chạy:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Nhận định nào đúng?

A. Nhờ có .python-version, lệnh tạo venv tự tải và dùng Python 3.12.3.
B. Không cần quan tâm phiên bản Python, vì requirements.txt đã ghi đủ phiên bản thư viện.
C. Venv dùng Python 3.11 sẵn có; cần cài Python 3.12.3 rồi tạo lại venv để khớp.
D. Lệnh pip install -r sẽ tự cài Python 3.12.3 trước, rồi mới cài các thư viện.

Đáp án: C. Venv dùng Python 3.11 sẵn có; cần cài Python 3.12.3 rồi tạo lại venv bằng đúng bản đó.

Giải thích. `python -m venv` chạy module venv của chính trình thông dịch mà lệnh python đang trỏ tới. Máy chỉ có Python 3.11 và không có công cụ quản lý phiên bản, nên môi trường tạo ra dựa trên 3.11.

Tệp `.python-version` chỉ là thông tin mà một số công cụ quản lý phiên bản có thể đọc; bản thân venv không tự tải Python theo tệp ấy. `requirements.txt` ghi phiên bản thư viện, không thay thế việc cài trình thông dịch. Lệnh pip install cũng không tự nâng Python lên 3.12.3.

```
python3.12 -m venv .venv
source .venv/bin/activate
python --version
python -m pip install -r requirements.txt
```

Ví dụ trên giả sử đã cài và chọn đúng Python 3.12.3; tên lệnh cụ thể tùy hệ điều hành và cách cài.

Mở rộng / Cách nhớ. Môi trường tái lập cần cả phiên bản Python và thư viện. `pip freeze` ghi các gói đang cài, không bảo đảm mọi gói đều tương thích với mọi phiên bản Python hay mọi hệ điều hành.

Mã 001 • Câu 3. (1 điểm)

Sau bước làm sạch giá, None là giá không đọc được, còn 0.0 là giá khuyến mãi hợp lệ.

```
v = [1200.0, None, 0.0, 600.0]
da_biet = [x for x in v if x]
print(sum(da_biet) / len(da_biet))
```

Đoạn code in ra gì?

A. Báo lỗi TypeError
B. 900.0
C. 450.0
D. 600.0

Đáp án: B. 900.0.

Giải thích. Điều kiện `if x` kiểm tra truthiness, không chỉ kiểm tra thiếu. Cả None và số 0.0 đều có giá trị logic False. Hai phần tử được giữ là `[1200.0, 600.0]`, nên:

$$\frac{1200 + 600}{2} = 900.$$

Code không bị TypeError vì None đã bị loại trước khi cộng. Tuy nhiên, nó cũng loại nhầm giá khuyến mãi 0.0 hợp lệ. **Nếu muốn trung bình trên các giá đã biết:**

```
da_biet = [x for x in v if x is not None]
print(sum(da_biet) / len(da_biet))  # 600.0
```

Lúc này mẫu số là 3, gồm 1200, 0 và 600.

Mở rộng / Cách nhớ. Dùng điều kiện đúng với ý nghĩa dữ liệu. Một bộ lọc truthiness còn loại cả chuỗi rỗng và các đối tượng rỗng; nó không phải bộ nhận dạng mọi loại dữ liệu thiếu. Nếu không còn giá nào, cần xử lý trước phép chia cho độ dài 0.

Mã 001 • Câu 4. (1 điểm)

Tệp luot_o.csv:

```
ma,so_dem
L1,40
L2,5
L3,9
L4,6
L5,15
```

```
import csv
with open("luot_o.csv", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
dai = sorted(rows, key=lambda r: r["so_dem"], reverse=True)
print(dai[0]["ma"])
```

Dòng cuối in ra gì?

A. L5
B. L3
C. L1
D. L2

Đáp án: B. L3.

Giải thích. `csv.DictReader` đọc các ô thành chuỗi. Vì chưa chuyển kiểu, khóa sắp xếp là các chuỗi `'40'`, `'5'`, `'9'`, `'6'`, `'15'`. Thứ tự giảm dần theo chuỗi là:

```
['9', '6', '5', '40', '15']
# Mã tương ứng: L3, L4, L2, L1, L5
```

Ký tự '9' lớn hơn '6', '5', '4', '1' trong so sánh này, nên L3 đứng đầu. Đây không phải sắp xếp theo số đêm thực tế.

Mở rộng / Cách nhớ. Chuyển khóa sang số khi sắp xếp:

```
dai = sorted(rows, key=lambda r: int(r["so_dem"]), reverse=True)
print(dai[0]["ma"])  # L1, vì 40 là số lớn nhất
```

Với tệp thực tế, cần kiểm tra cả ô trống hoặc chuỗi không đọc được trước khi ép int.

Mã 001 • Câu 5. (1 điểm)

Đoạn code sau in ra gì?

```
G = np.array([[4, 7, 1],
              [6, 2, 8]])
h = G[1, 1:]
m = G[G < 5]
h[1] = 6
m[0] = 9
print(G.sum())
```

A. 33
B. 31
C. 28
D. 26

Đáp án: D. 26.

Giải thích. h được tạo bằng basic slicing nên là một view vào hàng thứ hai của G, gồm hai phần tử 2 và 8. Lệnh `h[1] = 6` sửa phần tử 8 trong G thành 6.

Ngược lại, `G[G < 5]` dùng Boolean indexing, tạo mảng độc lập m với các giá trị ban đầu `[4, 1, 2]`. Sửa `m[0] = 9` chỉ đổi m, không đổi 4 trong G. Cuối cùng:

```
G = [[4, 7, 1],
     [6, 2, 6]]
```

Tổng là $4 + 7 + 1 + 6 + 2 + 6 = 26$.

Mở rộng / Cách nhớ. Có thể kiểm tra quan hệ bộ nhớ bằng `np.shares_memory(G, h)` và với m. Nếu muốn sửa G theo điều kiện, gán trực tiếp `G[G < 5] = ...`; việc tạo một mảng lọc rồi sửa mảng đó là thao tác khác.

Mã 001 • Câu 6. (1 điểm)

Mảng T có dtype int32, T.shape là (6, 5) và T.strides là (20, 4). Xét:

```
V = T[1::2, ::2]
```

V.shape và V.strides lần lượt là gì?

A. (3, 3) và (40, 8)
B. (2, 3) và (40, 8)
C. (3, 3) và (20, 8)
D. (2, 3) và (20, 8)

Đáp án: A. V.shape = (3, 3), V.strides = (40, 8).

Giải thích. Trục hàng có 6 vị trí 0--5. Lát `1::2` lấy hàng 1, 3, 5: ba hàng. Trục cột có 5 vị trí 0--4; `::2` lấy cột 0, 2, 4: ba cột. Vì vậy shape mới là (3, 3).

Strides cho biết số byte cần dịch trong bộ nhớ để tăng một vị trí ở mỗi trục. Bước hàng mới gấp 2 bước hàng cũ, nên $20 \times 2 = 40$ byte. Bước cột mới cũng gấp 2, nên $4 \times 2 = 8$ byte.

Mở rộng / Cách nhớ. Với basic slicing, bước nhảy thay đổi strides mà không cần dồn các phần tử thành vùng liên tục mới. Dtype vẫn int32, itemsize vẫn 4 byte; một bước theo cột của view có thể vượt hơn một phần tử của mảng gốc.

Mã 001 • Câu 7. (1 điểm)

Với x = np.arange(1_000_000) (kiểu int64) và ds = list(range(1_000_000)), phép x * 2 thường nhanh hơn nhiều so với [v * 2 for v in ds]. Giải thích nào đúng?

A. NumPy lưu mỗi số thành một đối tượng riêng, nhờ vậy Python truy cập từng số nhanh hơn.
B. NumPy dùng SIMD để chia một triệu phép nhân cho nhiều lõi CPU chạy song song.
C. NumPy nén các số theo dtype, nên thực tế chỉ phải nhân ít hơn một triệu số.
D. Phần lặp chạy bằng mã máy đã biên dịch sẵn cho int64, đọc các số nằm liền nhau.

Đáp án: D. Phần lặp chạy bằng mã máy đã biên dịch sẵn cho int64, đọc các số nằm liền nhau.

Giải thích. NumPy lưu dữ liệu int64 trong vùng số đồng nhất; phép nhân mảng được triển khai bằng vòng lặp mức thấp trong mã đã biên dịch. Python chỉ phát lệnh toán trên mảng, tránh điều khiển và xử lý một đối tượng Python riêng ở từng lượt như list comprehension.

A mô tả sai cách lưu mảng số. C sai vì vẫn có một triệu giá trị cần nhân, không có phép nén làm giảm số phép nhân như nói trong đáp án. B nhầm SIMD với việc chia công việc cho nhiều lõi CPU: SIMD là một lệnh thao tác trên nhiều phần tử và không tự đồng nghĩa chạy đa lõi.

Mở rộng / Cách nhớ. Lợi thế còn phụ thuộc kích thước, dtype và bố trí bộ nhớ. `x * 2` thường tạo mảng kết quả mới; không nên suy từ việc nhanh hơn rằng nó không có vòng lặp, không cấp phát bộ nhớ hoặc luôn dùng mọi lõi CPU.

Mã 001 • Câu 8. (1 điểm)

Đoạn code sau in ra gì?

```
df = pd.DataFrame({"gia": [100, 50, 80, 120, 60]})
r = df[df["gia"] < 90]
print(r.loc[1:4, "gia"].sum(), r.iloc[1:4, 0].sum())
```

A. 190 140
B. 130 190
C. 130 140
D. 190 190

Đáp án: A. 190 140.

Giải thích. Sau khi lọc giá dưới 90, r giữ chỉ mục gốc:

Vị trí trong r	Nhãn hàng	Giá
0	1	50
1	2	80
2	4	60

`loc[1:4]` cắt theo nhãn và lấy cả nhãn cuối, nên giữ cả ba hàng: tổng $50 + 80 + 60 = 190$. `iloc[1:4]` cắt theo vị trí, lấy từ vị trí 1 đến trước 4; r chỉ có ba hàng nên lấy 80 và 60, tổng 140.

Mở rộng / Cách nhớ. Phép lọc không tự đánh lại chỉ mục. Nếu muốn nhãn mới 0, 1, 2, phải gọi `reset_index(drop=True)`. Đừng dùng một con số trong loc và iloc như thể nó có cùng ý nghĩa.

Mã 001 • Câu 9. (1 điểm)

Bảng df có hai cột khu và gia. Chạy:

```
tk = df.groupby("khu")["gia"].agg(
    trung_vi="median", so_gia="count")
tk.to_csv("tong_hop.csv", index=False)
lai = pd.read_csv("tong_hop.csv")
```

Nhận định nào đúng về bảng lai?

A. Có ba cột Unnamed: 0, trung_vi, so_gia; tên khu nằm ở cột đầu.
B. read_csv báo lỗi, vì tệp thiếu cột chỉ mục để đọc lại bảng.
C. Có ba cột khu, trung_vi, so_gia, vì chỉ mục có tên nên vẫn được ghi.
D. Chỉ có hai cột trung_vi, so_gia; không còn biết mỗi hàng là khu nào.

Đáp án: D. Chỉ có hai cột trung_vi, so_gia; mất thông tin mỗi hàng thuộc khu nào.

Giải thích. Theo mặc định của groupby, khóa khu trở thành chỉ mục của bảng tổng hợp tk. Hai thống kê là cột `trung_vi` và `so_gia`. Khi ghi với `index=False`, chỉ mục khu không được xuất dù nó có tên.

Đọc CSV không báo lỗi vì tệp chỉ có hai cột dữ liệu vẫn là CSV hợp lệ. Cột `Unnamed: 0` thường do một chỉ mục không tên đã được ghi ra; trường hợp này lại chủ động không ghi chỉ mục, nên không có cột ấy.

Mở rộng / Cách nhớ. Đưa khu về cột trước khi ghi:

```
tk.reset_index().to_csv("tong_hop.csv", index=False)
```

Kết quả có ba cột khu, trung_vi, so_gia và không ghi thêm chỉ mục số. Khóa định danh cần được giữ cùng các giá trị tổng hợp để bảng có thể dùng tiếp.

Mã 001 • Câu 10. (1 điểm)

Đoạn code sau in ra gì?

```
ks = pd.DataFrame({"gia": [120, 90, 75]},
                  index=["K1", "K2", "K3"])
thue = pd.Series([9, 12, 6], index=["K2", "K4", "K1"])
ks["tong"] = (ks["gia"] + thue).to_numpy()
print(ks["tong"].tolist())
```

A. [129, 102, 81]
B. [126.0, 99.0, nan]
C. Báo lỗi ValueError
D. [126.0, 99.0, 75.0]

Đáp án: C. Báo lỗi ValueError ở lệnh gán cột tong.

Giải thích. Phép cộng hai Series căn theo nhãn trước khi gọi to_numpy. Hợp các nhãn là K1, K2, K3, K4:

Nhãn	Giá	Thuế	Tổng
K1	120	6	126
K2	90	9	99
K3	75	NaN	NaN
K4	NaN	12	NaN

`to_numpy()` tạo mảng dài 4. Bảng ks chỉ có 3 hàng, nên không thể gán mảng dài 4 thành một cột dài 3. Chương trình báo ValueError trước dòng print.

Mở rộng / Cách nhớ. Nếu giữ Series ở vế phải, pandas căn về nhãn hàng của ks khi gán:

```
ks["tong"] = ks["gia"] + thue
# Trên K1, K2, K3: [126.0, 99.0, NaN]
```

Đây là code đã sửa. Không được chọn B cho nguyên văn vì nguyên văn đã chuyển sang mảng và bị lỗi độ dài.

Mã 001 • Câu 11. (1 điểm)

Đoạn code sau in ra gì?

```
d = pd.DataFrame({"loai": ["Studio", "Căn hộ", "Nhà nghỉ",
                           "Studio", "Căn hộ"]})
d["phu_phi"] = d["loai"].map({"Studio": 8, "Căn hộ": 15})
print(d["phu_phi"].sum(), d["phu_phi"].count(), len(d))
```

A. nan 5 5
B. nan 4 5
C. 46.0 4 5
D. 46 5 5

Đáp án: C. 46.0 4 5.

Giải thích. map tra phụ phí theo loại. Bốn hàng Studio hoặc Căn hộ được ánh xạ thành 8, 15, 8, 15. Nhà nghỉ không có trong từ điển nên cho NaN. Cột là `[8.0, 15.0, NaN, 8.0, 15.0]`.

Sum mặc định bỏ NaN nên tổng $8 + 15 + 8 + 15 = 46.0$. Count đếm số giá trị không thiếu nên bằng 4. Len đếm số hàng nên vẫn bằng 5.

Mở rộng / Cách nhớ. Không có khóa trong từ điển là một dạng chưa xác định kết quả ánh xạ, không tự động là phụ phí 0. Nếu nghiệp vụ muốn Nhà nghỉ miễn phụ phí, cần khai báo rõ trong từ điển; khi đó count mới tăng thành 5.

Mã 001 • Câu 12. (1 điểm)

Bảng dat ghi các lượt đặt phòng, bảng km ghi mã khuyến mãi của từng khách sạn. Đoạn code sau in ra gì?

```
dat = pd.DataFrame({"ma": ["D1", "D2", "D3", "D4", "D5"],
                    "ks": ["K1", "K2", "K1", "K3", "K2"]})
km = pd.DataFrame({"ks": ["K1", "K2", "K2"],
                   "ma_km": ["HE10", "LE5", "VIP"]})
m = dat.merge(km, on="ks", how="left")
print(len(m), m["ma_km"].isna().sum())
```

A. 7 1
B. 5 1
C. Báo lỗi MergeError
D. 6 0

Đáp án: A. 7 1.

Giải thích. K1 có một mã khuyến mãi nên mỗi đơn đặt K1 tạo một hàng. K2 có hai mã nên mỗi đơn đặt K2 tạo hai hàng. K3 không có mã nhưng left join vẫn giữ đơn đặt của nó, với ma_km thiếu:

Đơn đặt	Khách sạn	Số hàng sau ghép
D1	K1	1
D2	K2	2
D3	K1	1
D4	K3	1
D5	K2	2

Tổng $1 + 2 + 1 + 1 + 2 = 7$ hàng; chỉ D4 có ma_km thiếu, nên số ô NaN là 1. Merge không tự báo lỗi chỉ vì khóa ở bảng phải lặp nếu không có yêu cầu validate tương ứng.

Mở rộng / Cách nhớ. Nếu mục tiêu là mỗi đơn đặt chỉ một hàng, cần xác định cách xử lý nhiều khuyến mãi trước khi ghép, chẳng hạn chọn một mã theo quy tắc. Không được xóa ngẫu nhiên một trong hai mã hoặc giả sử left join luôn giữ nguyên số hàng.

Mã 001 • Câu 13. (1 điểm)

Năm 2026, giá của mọi chỗ ở đều tăng thêm đúng 15 USD/đêm so với năm 2025. Mỗi năm, nhóm phân tích chia giá thành ba nhóm bằng pd.qcut(gia, 3, labels=["thấp", "giữa", "cao"]). Nhận định nào đúng?

A. Một số chỗ ở nhóm "giữa" chuyển lên nhóm "cao", vì giá của chúng đã tăng thêm 15 USD.
B. Mỗi chỗ ở vẫn ở đúng nhóm năm 2025, vì qcut chỉ dựa vào thứ tự giá mà thứ tự không đổi.
C. Hai mốc chia nhóm của năm 2026 giữ nguyên như năm 2025, vì qcut khoá cỡ của ba nhóm.
D. Cỡ ba nhóm vẫn bằng nhau, nhưng có thể có chỗ ở đổi nhóm vì qcut tính lại mốc cho mỗi năm.

Đáp án: B. Mỗi chỗ ở vẫn ở đúng nhóm năm 2025.

Giải thích. Với cùng tập chỗ ở và mọi giá đều cộng đúng 15, thứ tự và các quan hệ bằng nhau của giá không đổi. Các phân vị chia nhóm cũng tăng thêm 15, nên vị trí tương đối của mỗi giá so với hai mốc chia vẫn giữ nguyên.

Gọi giá cũ là x, mốc cũ là q. Sau khi tăng đồng loạt, điều kiện $x \le q$ tương đương $x + 15 \le q + 15$. Vì vậy việc qcut tính lại mốc không làm chỗ ở đổi nhóm. C sai vì các mốc số không giữ nguyên; A và D sai vì dự đoán có thể đổi nhóm trong phép tịnh tiến đồng loạt này.

Mở rộng / Cách nhớ. Không khái quát kết quả cho trường hợp thêm/bớt chỗ ở hoặc mỗi chỗ tăng khác nhau. Nếu giá có quá nhiều giá trị trùng, qcut có thể gặp mốc không phân biệt; câu hỏi đang xét tình huống phép chia ba nhóm đã thực hiện được.

Mã 001 • Câu 14. (1 điểm)

Tệp diem.csv, trong đó NA là mã tỉnh hợp lệ:

```
ma,tinh,diem
007,NA,8
012,HN,
015,NA,9
```

```
df = pd.read_csv("diem.csv")
print(df["ma"].iloc[0], df["tinh"].isna().sum())
```

Dòng cuối in ra gì?

A. 7 2
B. 007 2
C. 007 0
D. 7 0

Đáp án: A. 7 2.

Giải thích. Không khai báo dtype, cột ma được suy luận là số nguyên nên 007 thành số 7. Các số 0 đầu không được giữ trong biểu diễn số.

Mặc định read_csv nhận chuỗi NA là ký hiệu thiếu. Vì vậy hai ô tỉnh NA đều trở thành giá trị thiếu, dù theo nghiệp vụ NA là mã tỉnh hợp lệ. Dòng print in mã số đầu tiên 7 và số tỉnh thiếu 2.

Mở rộng / Cách nhớ. Một cách đọc đúng quy ước nguồn là:

```
df = pd.read_csv("diem.csv", dtype={"ma": "string"},
                 keep_default_na=False, na_values={"diem": [""]})
```

Cách này giữ 007, giữ tỉnh NA, đồng thời nhận ô điểm rỗng là thiếu. Dùng quy ước thiếu theo từng cột để tránh làm mất mã hợp lệ.

Mã 001 • Câu 15. (1 điểm)

DataFrame df có cột id, khu với bốn hàng (001, A), (002, B), (003, C), (004, A). DataFrame khu_phi có cột khu, phi với hai hàng (A, 20), (B, 5).

```
kq = duckdb.sql("""
    SELECT df.id, khu_phi.phi
    FROM df LEFT JOIN khu_phi ON df.khu = khu_phi.khu
    WHERE khu_phi.phi < 10
""").df()
```

Bảng kq có bao nhiêu hàng?

A. 2
B. 1
C. Báo lỗi
D. 4

Đáp án: B. 1 hàng, là id 002 ở khu B với phí 5.

Giải thích. Left join trước hết cho phí 20 ở hai hàng khu A, phí 5 ở hàng khu B, và NULL ở hàng khu C không ghép được. Sau đó WHERE chỉ giữ hàng có điều kiện `phi < 10` là TRUE.

Phí 20 không đạt; phí 5 đạt; NULL so với 10 cho UNKNOWN chứ không phải TRUE, nên hàng khu C cũng bị loại. Kết quả chỉ còn id 002.

Mở rộng / Cách nhớ. Điều kiện trên cột bảng phải đặt trong WHERE có thể loại các hàng không ghép được của left join. Nếu muốn giữ mọi hàng trái nhưng chỉ ghép các phí nhỏ hơn 10, phải đặt điều kiện đó trong ON; nếu muốn giữ thêm hàng phí thiếu khi lọc, phải thêm điều kiện IS NULL phù hợp mục tiêu.

Mã 001 • Câu 16. (1 điểm)

Code do AI viết gọi API dự báo thời tiết, trong đó có một tên tham số không có trong tài liệu của dịch vụ:

```
r = requests.get(url, params=tham_so, timeout=20)
du_lieu = r.json()
print(r.status_code, sorted(du_lieu))
```

Dòng cuối in 400 ['error', 'reason']. Nhận định nào đúng?

A. Chỉ cần tăng timeout, vì mã 400 cho biết máy chủ không kịp trả lời trong vòng 20 giây đã đặt.
B. Mã 400 chỉ là cảnh báo; raise_for_status() chỉ dừng chương trình khi máy chủ gặp sự cố (mã 5xx).
C. Phản hồi báo lỗi cũng có thể là JSON; gọi r.raise_for_status() trước r.json() thì code dừng tại đó.
D. Yêu cầu vẫn thành công vì r.json() đọc được thành dict; chỉ là dịch vụ không có dữ liệu cho tham số đó.

Đáp án: C. Phản hồi lỗi cũng có thể là JSON; raise_for_status() trước json() sẽ ném lỗi HTTP ở mã 400.

Giải thích. json() giải mã phần thân phản hồi. Một thông báo lỗi dạng JSON vẫn có thể được đọc thành dict bình thường; điều đó không biến yêu cầu thành thành công. Mã HTTP 400 thuộc nhóm lỗi yêu cầu phía khách, không phải thông báo timeout.

raise_for_status() kiểm tra trạng thái lỗi 4xx và 5xx. Với phản hồi đã nêu, nó ném HTTPError. Nếu không bắt ngoại lệ, chương trình dừng tại đó trước khi chạy json(). Vì vậy B sai khi nói chỉ dừng với 5xx; D sai khi lấy việc đọc được dict làm tiêu chí thành công.

```
r = requests.get(url, params=tham_so, timeout=20)
r.raise_for_status()
du_lieu = r.json()
```

Mở rộng / Cách nhớ. Cần sửa tên tham số theo tài liệu dịch vụ và xem nội dung error/reason khi xử lý lỗi. Tăng timeout không sửa một tên tham số không hợp lệ. Kiểm tra HTTP và kiểm tra cấu trúc dữ liệu là hai bước riêng.

Mã 001 • Câu 17. (1 điểm)

Đoạn code sau trích số phòng tắm từ tên chỗ ở:

```
import re
ten = pd.Series(["Căn 2 bath riêng", "Nhà 1wc", "Villa3baths",
                 "Phòng 2 bathroom"])
pat = r"\b(\d+)\s*(?:WC|baths?)\b"
so = ten.str.extract(pat, flags=re.I, expand=False)
```

Những hàng nào của so nhận được giá trị (không thiếu)? Chỉ mục hàng là 0, 1, 2, 3.

A. Hàng 0, 2 và 3
B. Hàng 0, 1 và 3
C. Hàng 0 và 1
D. Hàng 0 và 2

Đáp án: C. Hàng 0 và 1.

Giải thích. Mẫu bắt số ở ranh giới từ, cho phép không có khoảng trắng, nhận WC hoặc bath/baths và yêu cầu ranh giới từ sau đơn vị. re.I bỏ phân biệt hoa thường.

Hàng	Kết quả	Lý do
0	'2'	Có số 2 ở ranh giới từ và đơn vị bath hợp lệ.
1	'1'	Khoảng trắng không bắt buộc; wc khớp WC nhờ re.I.
2	NaN	Trong Villa3baths, a và 3 đều là ký tự từ; thiếu ranh giới trước số.
3	NaN	Bath là tiền tố của bathroom nhưng không có ranh giới ngay sau bath.

Mở rộng / Cách nhớ. Muốn nhận bathroom hoặc số dính sau tên Villa thì phải chủ động đổi mẫu theo yêu cầu nguồn. Một mẫu có thể tìm đúng phần số nhưng vẫn bị loại vì ràng buộc trước hoặc sau đơn vị; không chỉ nhìn vào chữ số có xuất hiện hay không.

Mã 001 • Câu 18. (1 điểm)

Nguồn ghi giá theo dạng $2,400; ô None là giá thiếu từ nguồn.

```
gia = pd.Series(["$85", None, "$2,400", "$1,050", "thoả thuận"])
so = pd.to_numeric(gia.str.replace("$", "", regex=False),
                   errors="coerce")
loi = gia.notna() & so.isna()
print(loi.sum())
```

Dòng cuối in ra gì?

A. 4
B. 1
C. 3
D. Báo lỗi ValueError

Đáp án: C. 3.

Giải thích. Lệnh replace chỉ xóa dấu đô la, chưa xóa dấu phẩy ngăn cách hàng nghìn. Các giá được thử chuyển thành số lần lượt là `'85'`, thiếu, `'2,400'`, `'1,050'`, `'thoả thuận'`.

85 chuyển được; hai chuỗi có dấu phẩy và chuỗi thoả thuận không chuyển được nên thành NaN. None vốn thiếu từ nguồn, bị điều kiện gia.notna() loại khỏi nhóm lỗi. Ba hàng được đếm là chỉ mục 2, 3, 4; tổng bằng 3.

Mở rộng / Cách nhớ. Để đọc đúng các giá tiền theo quy ước đã cho:

```
sach = gia.str.replace("$", "", regex=False).str.replace(
    ",", "", regex=False)
so = pd.to_numeric(sach, errors="coerce")
```

Khi ấy chỉ thoả thuận còn là chuỗi nguồn có giá trị nhưng không đọc được số. Không dùng coerce để che mất lỗi làm sạch mà không kiểm tra những hàng chuyển thành thiếu.

Mã 001 • Câu 19. (1 điểm)

Mỗi phần tử của thoi_diem là lúc một đánh giá được gửi. Đoạn code sau in tỷ lệ thay đổi của ba ngày đầu. Kết quả là gì?

```
thoi_diem = pd.to_datetime([
    "2026-10-01 08:00", "2026-10-01 19:30", "2026-10-02 10:15",
    "2026-10-04 07:00", "2026-10-04 12:40", "2026-10-04 21:05"])
dem = pd.Series(1, index=thoi_diem).resample("D").size()
print(dem.pct_change().head(3).tolist())
```

A. [nan, -0.5, 2.0]
B. [nan, -50.0, 200.0]
C. [nan, -50.0, -100.0]
D. [nan, -0.5, -1.0]

Đáp án: D. [nan, -0.5, -1.0].

Giải thích. Resample theo ngày tạo cả ngày không có đánh giá. Số hàng mỗi ngày từ 01/10 đến 04/10 là `[2, 1, 0, 3]`. Ngày 03/10 có 0 hàng, không bị bỏ khỏi dãy.

pct_change tính $x_t / x_{t-1} - 1$. Ba giá trị đầu là: ngày đầu không có mốc trước nên NaN; ngày 02 là $1/2 - 1 = -0.5$; ngày 03 là $0/1 - 1 = -1.0$.

Hàm trả tỷ lệ dạng thập phân, không tự nhân 100. Vì vậy -0.5 tương ứng -50%, còn -1.0 tương ứng -100%.

Mở rộng / Cách nhớ. Nếu xét thêm ngày 04, mốc trước bằng 0 nên mức thay đổi thông thường không xác định hữu hạn; phép tính có thể cho inf. Không nhầm 0 đánh giá với dữ liệu chưa được thu: trong bài này size đếm hàng của tập quan sát đã cho.

Mã 001 • Câu 20. (1 điểm)

Biết ngày 31/10/2026 là thứ Bảy.

```
t = pd.Timestamp("2026-10-31 20:30", tz="UTC")
vn = t.tz_convert("Asia/Ho_Chi_Minh")
print(vn.month, vn.dayofweek)
```

Dòng cuối in ra gì?

A. 10 6
B. 11 7
C. 10 5
D. 11 6

Đáp án: D. 11 6.

Giải thích. Mốc t đã có múi giờ UTC nên dùng tz_convert để hiển thị cùng thời điểm theo giờ Việt Nam. UTC+7 làm 31/10 lúc 20:30 thành 01/11 lúc 03:30:

```
2026-11-01 03:30:00+07:00
```

Tháng của vn là 11. Ngày 31/10 là thứ Bảy nên 01/11 là Chủ nhật. Quy ước dayofweek dùng thứ Hai 0 đến Chủ nhật 6, nên in 11 và 6, không phải 11 và 7.

Mở rộng / Cách nhớ. Một thời điểm có thể đổi cả ngày, tháng, năm khi đổi múi giờ. Muốn thống kê theo ngày địa phương, chuyển múi giờ trước khi lấy ngày/tháng hoặc gom nhóm theo lịch địa phương.

Phần II: Tự luận

Mã 001 • Câu 21. (5 điểm)

Mảng D lưu số lượt đặt phòng: mỗi hàng là một khách sạn, mỗi cột là một tuần.

```
D = np.array([[10, 30, 20, 40],
              [ 5,  5, 10, 30],
              [ 8, 12, 20, 10]])
```

Cần mảng ty_trong cùng shape với D, trong đó mỗi số được chia cho tổng lượt đặt của khách sạn chứa nó.

a) (1,5 điểm) Vì sao D / D.sum(axis=1) báo lỗi?

b) (2 điểm) Viết một dòng NumPy tạo ty_trong, không dùng vòng lặp.

c) (1,5 điểm) Một bạn viết D / D.sum(axis=0). Lệnh này có báo lỗi không, vì sao? Mỗi số trong kết quả cho biết điều gì? Trả lời ngắn gọn trong ô trả lời (khoảng 6 dòng).

Đáp án: a) Shape (3, 4) không broadcast được với (3,). b) D / D.sum(axis=1, keepdims=True). c) Chia theo tổng cột chạy được nhưng cho tỷ trọng khách sạn trong từng tuần.

Mẫu trả lời ngắn.

a) Tổng theo hàng có shape (3,), so từ chiều cuối thì 4 khác 3 nên lỗi.

b) Viết:

```
ty_trong = D / D.sum(axis=1, keepdims=True)
```

c) Tổng theo cột có shape (4,) nên broadcast được; mỗi ô là phần đóng góp của khách sạn vào tổng lượt của tuần đó.

Giải thích. Broadcasting so kích thước từ phải sang trái. Hai chiều hợp lệ nếu bằng nhau hoặc một chiều bằng 1. Trong ý a, D là (3, 4), tổng theo axis=1 là `[100, 50, 50]` có shape (3,); chiều cuối là 4 và 3 nên không tương thích.

keepdims=True giữ tổng thành cột (3, 1). Khi chia, mỗi tổng được dùng cho cả bốn cột cùng hàng. Kết quả mong muốn là:

$$\begin{pmatrix} 0.10 & 0.30 & 0.20 & 0.40 \\ 0.10 & 0.10 & 0.20 & 0.60 \\ 0.16 & 0.24 & 0.40 & 0.20 \end{pmatrix}.$$

Tổng mỗi hàng bằng 1. Trái lại, tổng theo axis=0 là `[23, 47, 50, 80]`; chia từng cột cho số này làm tổng mỗi cột bằng 1. Nó trả lời một câu hỏi khác.

Mở rộng / Cách nhớ. Code chạy được chưa bảo đảm chia theo đúng đơn vị quan sát. Với dữ liệu tổng quát, cần xử lý hàng có tổng 0 trước khi chia; ở D đã cho, mọi tổng hàng đều dương.

Mã 001 • Câu 22. (5 điểm)

Yêu cầu: đặt giá mọi phòng hạng Cao cấp thành 130 USD/đêm; cột hai_dem (tiền 2 đêm) phải khớp với giá mới.

```
ph = pd.DataFrame(
    {"hang": ["Cao cấp", "Tiêu chuẩn", "Cao cấp"],
     "gia": [150.0, 70.0, 110.0]},
    index=["P01", "P02", "P03"])
ph["hai_dem"] = ph["gia"] * 2
ph[ph["hang"] == "Cao cấp"]["gia"] = 130
print(ph.loc["P01", ["gia", "hai_dem"]].tolist())
```

Code chạy hết, in một cảnh báo và dòng cuối in [150.0, 300.0].

a) (2 điểm) Vì sao lệnh gán giá 130 không đổi giá trong ph?

b) (2 điểm) Viết lại lệnh đó thành một lệnh gán đúng, chỉ gán cột gia và chọn hàng theo cột hang.

c) (1 điểm) Sau khi sửa ở b), dòng print in ra gì? Đã đúng yêu cầu chưa? Nếu chưa, cần làm gì thêm?

Trả lời ngắn gọn trong ô trả lời (khoảng 6 dòng).

Đáp án: a) Gán chuỗi thao tác chọn vào đối tượng trung gian. b) Dùng ph.loc[mask, "gia"] = 130. c) Sau chỉ sửa giá, P01 là [130.0, 300.0]; tính lại hai_dem thì mới thành [130.0, 260.0].

Mẫu sửa ngắn.

```
ph.loc[ph["hang"] == "Cao cấp", "gia"] = 130
# Chỉ sửa dòng trên: print cho [130.0, 300.0], chưa đúng.
ph["hai_dem"] = ph["gia"] * 2
# Sau tính lại: P01 có [130.0, 260.0].
```

Giải thích. Biểu thức lọc hàng tạo đối tượng trung gian; tiếp tục chọn cột gia rồi gán vào đó là chained assignment. Nó không phải một phép gán trực tiếp xác định cả hàng và cột của ph. Kết quả không cập nhật bảng gốc như yêu cầu; pandas 3 với Copy-on-Write cũng không cho phép truyền thay đổi ngược theo cách này.

loc thực hiện chọn hàng theo hang và chọn đúng cột gia trong một lệnh gán lên ph. Sau sửa, giá P01 là 130, nhưng hai_dem đã được tính từ giá cũ 150 và lưu là 300. Cột này không tự chứa công thức liên kết để cập nhật theo gia như một ô bảng tính.

Tính lại hai_dem sau khi giá mới đã có sẽ cho P01 và P03 cùng 260; P02 vẫn là 140. Có thể kiểm tra bảng sau cùng:

Phòng	Giá	Hai đêm
P01	130	260
P02	70	140
P03	130	260

Mở rộng / Cách nhớ. Thứ tự phụ thuộc là dữ liệu gốc được sửa trước, cột dẫn xuất được tính sau. Chỉ sửa cảnh báo bằng cách tắt warning không khắc phục việc bảng chưa đổi và cột tiền đang cũ.

Mã 001 • Câu 23. (5 điểm)

Bảng dg có ba cột ma (mã chỗ ở), khu, diem; mỗi hàng là một đánh giá, một chỗ ở có thể có nhiều đánh giá; chỉ mục là 0, 1, 2, … và cột diem không thiếu. Cần bảng du gồm các đánh giá thuộc những khu có từ 3 chỗ ở khác nhau trở lên. Một bạn viết:

```
du = dg[dg.groupby("khu")["ma"].nunique() >= 3]
```

và nhận lỗi IndexingError.

a) (1,5 điểm) Vì sao lệnh trên báo lỗi?

b) (1,5 điểm) Sửa lệnh trên cho đúng.

c) (2 điểm) Viết lệnh thêm vào du cột lech bằng điểm của đánh giá trừ điểm trung bình của chính chỗ ở được đánh giá.

Trả lời ngắn gọn trong ô trả lời (khoảng 6 dòng).

Đáp án: a) Mặt nạ có index theo khu, không căn được với index hàng đánh giá. b) Dùng transform hoặc lọc theo danh sách khu. c) Dùng trung bình theo ma, không theo khu.

Mẫu trả lời ngắn.

```
mask = dg.groupby("khu")["ma"].transform("nunique") >= 3
du = dg.loc[mask].copy()
tb_ma = dg.groupby("ma")["diem"].transform("mean")
du["lech"] = du["diem"] - tb_ma.loc[du.index]
```

Giải thích. groupby(...).nunique() trả một số cho mỗi khu, với nhãn là tên khu. So sánh >=3 tạo Series Boolean cũng có nhãn khu. Bảng dg lại có chỉ mục hàng 0, 1, 2, ...; dùng Series đó lọc trực tiếp khiến pandas không căn được nhãn, dẫn đến IndexingError như đề nêu.

transform đưa số mã phân biệt của khu về từng hàng đánh giá của chính khu ấy. Mặt nạ sau đó có cùng chỉ mục với dg. Phải dùng nunique vì một chỗ ở có thể có nhiều đánh giá; count hoặc size sẽ đếm đánh giá và có thể giữ nhầm khu chưa đủ ba chỗ ở.

Với cột lech, đơn vị cần so là chỗ ở nên nhóm bằng ma. tb_ma được tính trên các đánh giá nguồn và chọn về đúng các hàng giữ trong du. Ví dụ một mã có điểm 2 và 4, trung bình 3, hai độ lệch là -1 và +1.

Mở rộng / Cách nhớ. Một cách lọc khác là lấy các khu đạt điều kiện rồi dùng `dg["khu"].isin(khu_dat)`. Dùng copy cho bảng sẽ thêm cột giúp việc gán rõ ràng; không trộn khóa nhóm khu với khóa nhóm ma trong hai yêu cầu khác nhau.

Mã 001 • Câu 24. (5 điểm)

DataFrame kh có các cột ma, khu, gia; mỗi hàng là một chỗ ở; gia có thể thiếu (thành NULL khi truy vấn bằng DuckDB).

a) (3 điểm) Viết câu SQL cho duckdb.sql(...) trả về, với mỗi khu có ít nhất 2 chỗ ở giá từ 50 USD trở lên: tên khu, số chỗ ở đó (đặt tên so_cho) và giá trung bình của chúng (đặt tên tb).

b) (2 điểm) Nếu bỏ yêu cầu giá từ 50 USD (chỉ giữ các khu có ít nhất 2 giá đã biết, so_cho là số giá đã biết) thì câu SQL ở ý a cần đổi những gì? Vì sao?

Trả lời ngắn gọn trong ô trả lời (khoảng 6 dòng).

Đáp án: a) WHERE lọc giá >=50 trước khi GROUP BY; HAVING giữ nhóm có ít nhất hai hàng đạt điều kiện. b) Bỏ điều kiện giá, dùng COUNT(gia) cho cả so_cho và HAVING.

SQL trả lời ý a.

```
kq = duckdb.sql("""
    SELECT khu, COUNT(*) AS so_cho, AVG(gia) AS tb
    FROM kh
    WHERE gia >= 50
    GROUP BY khu
    HAVING COUNT(*) >= 2
""").df()
```

SQL trả lời ý b.

```
kq = duckdb.sql("""
    SELECT khu, COUNT(gia) AS so_cho, AVG(gia) AS tb
    FROM kh
    GROUP BY khu
    HAVING COUNT(gia) >= 2
""").df()
```

Giải thích. Ý a cần đếm và tính trung bình *những chỗ có giá từ 50 trở lên*. WHERE làm việc ở cấp hàng trước khi nhóm. Giá NULL không làm điều kiện >=50 thành TRUE nên bị loại; với các hàng còn lại, COUNT(*) là đúng số chỗ đạt yêu cầu. HAVING lọc trên kết quả nhóm.

Ý b cho phép mọi mức giá đã biết, kể cả dưới 50 hoặc bằng 0. Nếu chỉ bỏ WHERE mà giữ COUNT(*), các hàng giá thiếu cũng bị đếm vào so_cho và điều kiện ít nhất hai giá biết có thể bị sai. COUNT(gia) bỏ NULL; AVG(gia) cũng bỏ NULL, nên hai thống kê cùng dùng tập giá đã biết.

Mở rộng / Cách nhớ. Có thể dùng WHERE gia IS NOT NULL và COUNT(*) cho ý b; kết quả tương đương. WHERE là điều kiện hàng, HAVING là điều kiện nhóm. Do đề không cung cấp các giá cụ thể trong kh, không suy ra một bảng số cuối cùng ngoài câu SQL tổng quát.

Mã 001 • Câu 25. (5 điểm)

Cột tiện nghi lưu mỗi danh sách thành một chuỗi JSON:

```
raw = pd.Series(['["Wifi", "Kitchen"]', '["wifi ", "Pool"]', '[]',
                 '["Kitchen", "Pocket wifi"]'],
                index=["P1", "P2", "P3", "P4"])
```

Cần tỷ lệ chỗ ở có tiện nghi wifi, gộp các nhãn chỉ khác hoa thường hoặc khoảng trắng ở hai đầu (Pocket wifi là tiện nghi khác). Kết quả đúng là 0,5.

a) (3 điểm) Viết code đọc mỗi ô thành một danh sách rồi tính tỷ lệ này (đã có import json).

b) (2 điểm) Một bạn tách mỗi danh sách thành từng hàng bằng explode, chuẩn hoá nhãn, rồi lấy trung bình của phép so sánh với “wifi” và được khoảng 0,29. Phép trung bình đó chia cho bao nhiêu, lẽ ra phải chia cho bao nhiêu? Vì sao?

Trả lời ngắn gọn trong ô trả lời (khoảng 6 dòng).

Đáp án: a) P1 và P2 có wifi, tỷ lệ 2/4 = 0.5. b) Trung bình sau explode đang chia cho 7 hàng, đúng ra phải chia cho 4 chỗ ở.

Mẫu code ngắn cho ý a.

```
ds = raw.map(json.loads)
co_wifi = ds.map(lambda items: any(
    t.strip().lower() == "wifi" for t in items))
ty_le = co_wifi.mean()  # 0.5
```

Giải thích. json.loads đọc từng chuỗi thành một danh sách thật. Mỗi tên tiện nghi được strip hai đầu và lower rồi so bằng đúng wifi. Kết quả theo chỗ ở là `[True, True, False, False]`. Danh sách rỗng ở P3 cho any bằng False; Pocket wifi ở P4 vẫn là một tên khác, không bị tính chỉ vì chứa đoạn chữ wifi.

Sau explode, hai tiện nghi của P1, hai của P2, một ô NaN giữ chỗ cho danh sách rỗng P3 và hai của P4 tạo tổng cộng $2 + 2 + 1 + 2 = 7$ hàng. Có hai hàng bằng wifi, nên trung bình phép so sánh cho $2/7 \approx 0.2857$, tức khoảng 0.29. Nó đang đo tỷ lệ ở cấp hàng sau tách, không phải tỷ lệ chỗ ở.

Mẫu số đúng là bốn chỗ ở ban đầu. Mỗi chỗ cần được tính một lần, dù có nhiều tiện nghi, có nhãn lặp hoặc có danh sách rỗng.

Mở rộng / Cách nhớ. Nếu đã explode, cần gom lại mặt nạ theo mã chỗ ở bằng any rồi reindex đủ các chỗ ở gốc trước khi lấy mean. Kể cả bỏ NaN rồi chia cho sáu hàng tiện nghi còn lại, kết quả vẫn không phải tỷ lệ theo bốn chỗ ở.

Mã 001 • Câu 26. (5 điểm)

Tệp danh_gia.csv có cột ngay ghi theo DD/MM/YYYY , chứa đánh giá từ tháng 01/2025 đến hết ngày 10/10/2026, không sắp theo ngày; dòng dữ liệu đầu tiên là 03/10/2026. AI viết đoạn code đếm số đánh giá theo tháng và tỷ lệ thay đổi so với tháng trước:

```
rv = pd.read_csv("danh_gia.csv")
rv["ngay"] = pd.to_datetime(rv["ngay"])
thang = rv.set_index("ngay").resample("M").size()
tang = thang.pct_change()
```

a) (2 điểm) Dòng 2 có vấn đề gì? Sửa dòng 2 (chỉ cần ghi phần thay đổi).

b) (2 điểm) Dòng 3 có vấn đề gì trong pandas 3? Sửa dòng 3 (chỉ cần ghi phần thay đổi).

c) (1 điểm) Khi code đã chạy đúng, có nên dùng giá trị tang của tháng 10/2026 để kết luận về xu hướng không? Vì sao? Trả lời ngắn gọn trong ô trả lời (khoảng 6 dòng).

Đáp án: a) Khóa format="%d/%m/%Y". b) Đổi tần suất M thành ME trong pandas 3. c) Không dùng tháng 10 mới có dữ liệu đến ngày 10 để kết luận thay đổi của cả tháng.

Mẫu trả lời ngắn.

```
rv["ngay"] = pd.to_datetime(rv["ngay"], format="%d/%m/%Y")
thang = rv.set_index("ngay").resample("ME").size()
```

Tháng 10/2026 chưa đầy đủ; chỉ so các tháng đã đủ phạm vi quan sát hoặc dùng cùng số ngày quan sát.

a) Ngày nguồn là ngày/tháng/năm nhưng to_datetime không được chỉ định format. Với dòng đầu 03/10/2026, mặc định có thể hiểu thành 10/03/2026 thay vì 03/10. Việc suy định dạng đó còn có thể làm các ngày khác bị đọc sai hoặc báo lỗi. Format nêu rõ hợp đồng nguồn và tránh để bộ phân tích tự đoán.

b) Trong pandas 3, alias M cho MonthEnd đã bị bỏ; phải dùng ME. Đây không phải lỗi do dữ liệu chưa sort: resample có thể gom các mốc thời gian chưa sắp theo thứ tự nguồn. Đề cho biết ngày không sắp để người đọc không được lấy hàng cuối làm ngày quan sát mới nhất.

c) Tháng 9 đầy đủ được so với tháng 10 chỉ có đến 10/10 sẽ có độ phủ khác nhau. Nhãn 31/10 được resample tạo ra không chứng minh có dữ liệu cả tháng. Dùng tỷ lệ đó để kết luận xu hướng cả tháng có thể gây hiểu sai.

```
thang_du = thang.loc[thang.index <= rv["ngay"].max()]
tang_du = thang_du.pct_change(fill_method=None)
```

Theo khoảng nguồn nêu trong đề, phần giữ lại kết thúc ở kỳ 30/09/2026. Không có dữ liệu chi tiết số đánh giá nên không thể tự tính các tỷ lệ số cụ thể.

Mở rộng / Cách nhớ. Nếu tháng gốc có 0 đánh giá, phần trăm thay đổi ở kỳ sau cần xử lý riêng vì phép chia cho 0 không có tỷ lệ hữu hạn thông thường. Tách ba kiểm tra: đọc đúng ngày, dùng API đúng phiên bản, và diễn giải thống kê đúng độ phủ dữ liệu.

Đối chiếu PDF 046 mới với ảnh 046 cũ

Kết luận về độ đầy đủ. Cả bộ ảnh cũ và PDF mới đều có đủ câu 1--26; không có câu nào bị thiếu trong PDF mới. PDF mới có 9 trang vật lý do bố trí lại, dù dòng cuối tự ghi ``5 trang". Việc đếm nội dung được làm theo câu, không dựa vào dòng ghi số trang đó.

Sau khi bỏ qua khác biệt xuống dòng/khoảng trắng trình bày, có ba câu khác nội dung:

Câu	Ảnh cũ	PDF mới và cách dùng trong phần chính
16	Phương án A kết thúc bằng ``trước khi hàng bị nhân im."	Thêm chữ ``lặng": ``trước khi hàng bị nhân im lặng." Ý nghĩa không đổi; đáp án A.
18	B là 2026-08-31; C là 2026-08-18.	B là 2026-08-18; C là 2026-08-31. Đáp án mới C, đáp án ảnh cũ B.
24	Chuỗi thường, không có tiền tố r; ý a hỏi vì sao 18m2 không cho 18.	Chuỗi thô có r; ý a hỏi vì sao 18m2x không cho 18. Đáp án theo code mới ['18', nan].

Không trộn hai mã đáp án.

Đáp án số ngày cuối tháng ở câu 18 vẫn là 31/08/2026; chỉ chữ cái lựa chọn đổi. Ở câu 24, bản thân chuỗi mẫu đổi nên đầu ra cũng đổi. Phần lời giải chính ghi theo PDF mới và nhắc rõ nguồn của khác biệt. **Toàn bộ lựa chọn câu 18 theo ảnh cũ**

Đánh giá cuối cùng trong tháng 8 là ngày 18/08/2026. Nhãn tháng 8 của `resample("ME")` là ngày nào?

A. 2026-09-01
B. 2026-08-31
C. 2026-08-18
D. 2026-08-01

Đáp án ảnh cũ là B. Phần chính của tài liệu đã chép đủ lựa chọn theo PDF mới và chọn C.

Câu 24 theo ảnh 046 cũ: giữ riêng biến thể đã gửi

Hai tên cần lấy số mét vuông đứng thành một từ riêng.

```
ten = pd.Series(["18m2", "18m2x"])
print(ten.str.extract("\b(\d+)m2\b")[0].tolist())
```

a) (5 điểm) Dòng in ra list nào? Lệnh có dừng vì lỗi không? Vì sao 18m2 không cho số 18?

b) (5 điểm) Viết lại mẫu bằng chuỗi thô để ranh giới từ còn hiệu lực. List mới là gì?

Đáp án: a) [nan, nan]; lệnh không dừng vì lỗi trong môi trường Python 3.12 chuẩn. b) Dùng raw string thì danh sách mới là ["18", nan].

a) Đề cố ý viết mẫu bằng chuỗi Python thường, không có tiền tố `r`. Python xử lý chuỗi trước khi bộ regex nhìn thấy nó. Trong chuỗi thường, `\b` là ký tự backspace, mã `U+0008`; nó không còn là hai ký tự gạch chéo và b để regex hiểu thành ranh giới từ.

Vì hai tên `18m2`, `18m2x` không chứa backspace trước và sau đoạn cần khớp, mẫu không tìm được lần khớp nào. Kết quả in là `[nan, nan]`, kể cả tên `18m2` nhìn bề ngoài đúng đơn vị.

`\d` là escape không được Python nhận biết trong chuỗi thường; trên Python 3.12 có thể xuất hiện `SyntaxWarning: invalid escape sequence`. Với cấu hình cảnh báo thông thường, đây không phải lỗi làm dừng chương trình, và chuỗi vẫn đến được bộ regex. Không nhầm cảnh báo này với nguyên nhân không khớp: nguyên nhân quyết định là `\b` đã thành backspace. Hành vi cảnh báo có thể khác theo phiên bản hoặc cấu hình biến cảnh báo thành lỗi.

b) Giữ các dấu gạch chéo bằng chuỗi thô:

```
print(ten.str.extract(r"\b(\d+)m2\b")[0].tolist())
# ['18', nan]
```

Tên `18m2` có ranh giới từ ở đầu và cuối nên bắt được chuỗi `"18"`. Tên `18m2x` không có ranh giới giữa 2 và x, nên vẫn không khớp.

Mở rộng / Cách nhớ. Cách tương đương bằng chuỗi thường phải nhân đôi từng gạch chéo:

```
pat = "\\b(\\d+)m2\\b"
```

Raw string thường dễ đọc hơn cho regex. `extract` trả chuỗi đã bắt; nếu cần số nguyên để tính toán, dùng `pd.to_numeric` và giữ rõ các giá trị không trích được.

Tài liệu đối chiếu và bảng kiểm cuối

Đã kiểm tra đủ nội dung

- Năm mã đề 047, 045, 046, 030, 001; mỗi mã đúng 26 câu, tổng 130 câu có lời giải.
- 100 câu trắc nghiệm với 400 lựa chọn A--D; 30 câu tự luận với đầy đủ các ý hỏi và code nguồn.
- Ba câu 045/24--26 đã bổ sung từ ảnh cuối. Đã cập nhật 046/18 và 046/24 theo PDF mới.
- Không gán đáp án giả cho các câu lỗi của 030; các giả định, cảnh báo phiên bản và bản sửa minh họa đều ghi rõ.

Tài liệu chính thức dùng đối chiếu

1. pandas.to_datetime và Time series / date functionality: cách đọc ngày, múi giờ, tần suất và lấy kỳ.
2. pandas: Indexing and selecting data và Migration Guides: căn nhãn, phép chọn và Copy-on-Write.
3. pandas.read_csv và DataFrame.groupby: kiểu, ký hiệu thiếu, thống kê nhóm.
4. pandas.DataFrame.merge và Series.explode: khóa ghép, nhân hàng và danh sách rỗng.
5. NumPy: Indexing và Broadcasting: view/copy, bước nhảy và tương thích shape.
6. Python re, Python json và Python venv: mẫu chuỗi, JSON và môi trường Python.
7. Requests Quickstart: trạng thái HTTP và giải mã JSON.
8. DuckDB: Export to Pandas và Conversion between DuckDB and Python: kết quả DataFrame và giá trị thiếu.
9. pandas 3.0 release notes: bỏ offset alias M để dùng ME trong resample.

Các kết quả tính toán then chốt được chạy lại trên môi trường ghi ở bìa. Tài liệu trực tuyến có thể phản ánh phiên bản mới hơn; nơi nào hành vi phụ thuộc phiên bản đã được ghi chú tại câu liên quan.

Hoàn tất một tài liệu: toàn bộ 5 đề chính, đủ 130 câu; phụ lục giữ các khác biệt của mã 046 cũ.